ΔΙΕΘΝΕΣ ΠΑΝΕΠΙΣΤΗΜΙΟ ΤΗΣ ΕΛΛΑΔΟΣ
ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ, ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΣΧΕΔΙΑΣΜΟΣ ΚΑΙ ΥΛΟΠΟΙΗΣΗΣ ΦΟΡΕΣΙΜΗΣ ΣΥΣΚΕΥΗΣ ΑΝΙΧΝΕΥΣΗΣ ΒΗΜΑΤΩΝ

Διπλωματική Εργασία του

Μιχαήλ Παπαϊωάννου (19093)

Επιβλέπων: Βουρβουλάκης Ιωάννης Επίκουρος καθηγητής

ΣΕΡΡΕΣ, ΟΚΤΩΒΡΙΟΣ 2025

<u>Υπεύθυνη Δήλωση</u> *: Βεβαιώνω ότι είμαι συγγραφέας αυτής της πτυχιακής εργασίας και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της, είναι πλήρως αναγνωρισμένη και αναφέρεται στην πτυχιακή εργασία. Επίσης έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επίσης, βεβαιώνω ότι αυτή η πτυχιακή εργασία προετοιμάστηκε από εμένα προσωπικά ειδικά για τις απαιτήσεις του προγράμματος σπουδών του Τμήματος Μηχανικών Πληροφορικής, Υπολογιστών και Τηλεπικοινωνιών του Διεθνούς Πανεπιστημίου της Ελλάδας.*

Παπαϊωάννου Μιχαήλ

ΕΥΧΑΡΙΣΤΙΕΣ

Η παρούσα πτυχιακή εργασία πραγματοποιήθηκε στο Διεθνές Πανεπιστήμιο της Ελλάδος πανεπιστημιούπολη Σερρών, στο τμήμα Μηχανικών Πληροφορικής, κατά το έτος 2024-2025. Θα ήθελα να εκφράσω τις ειλικρινείς μου ευχαριστίες στον επιβλέποντα καθηγητή μου Βουρβουλάκη Ιωάννη, για την καθοδήγηση, την υπομονή και την πολύτιμη συμβολή του καθ΄ όλη την διάρκεια εκπόνησης της παρούσας διπλωματικής εργασίας. Επίσης, ευχαριστώ θερμά τα μέλη της οικογένειάς μου και τους φίλους μου για τη στήριξη, την κατανόηση και την ενθάρρυνση που μου παρείχαν.

ΕΠΙΒΛΕΠΩΝ ΚΑΘΗΓΗΤΗΣ: ΒΟΥΡΒΟΥΛΑΚΗΣ ΙΩΑΝΝΗΣ

Πίνακας περιεχομένων

ΚΕΦΑΛΑΙΟ 1 7
Εισαγωγή 7
ΚΕΦΑΛΑΙΟ 2 8
2. Ανάλυση Περιφερειακών της Συσκευής 8
2.1 Εισαγωγή στους μικροελεγκτές 8
2.1.1 Αρχιτεκτονική ενός μικροελεγκτή 9
2.2 Ο μικροελεγκτής PIC18F57Q43 13
2.2.1 Γενικά χαρακτηριστικά του μικροελεγκτή 13
2.2.2 Η αρχιτεκτονική του PIC18F57Q43 14
2.2.3 Επικοινωνία I^2C 16
2.2.4 Μονάδα UART 18
2.3 PIC18F57Q43 Curiosity Nano Evaluation Kit 20
2.4 BNO055 22
2.4.1 Adafruit-BNO055 9-Axis Abs. Orientation 23
2.5 Οθόνη OLED SDD1306 128x64 24
2.6 CP2102 USB 2.0 25
ΚΕΦΑΛΑΙΟ 3 27
3. Σχεδιασμός και υλοποίησης κατασκευής 27
3.1 Σχεδίαση και υλοποίηση κυκλώματος 27
3.2 Σχεδίαση και υλοποίηση της θήκης 29
ΚΕΦΑΛΑΙΟ 4 38
4. Περιγραφή του Firmware της συσκευής ανίχνευσης βημάτων 38
4.1.1 Η λογική πίσω από το αρχείο OLED.C 38
4.1.2 Ανάλυση συναρτήσεων του αρχείου OLED.C 40
4.2.1 Ανάλυση του αρχείου BNO055.C 42
4.2.2 Συναρτήσεις του αρχείου BNO055.C 42
4.3.1 Ανάλυση του αρχείου save_data.c 43
4.4.1 Ανάλυση του αρχείου main.c 43
ΚΕΦΑΛΑΙΟ 5 46
5.1 Ανίχνευση Βημάτων μέσω Αλγορίθμου σε Python 46
5.1.1 Αλγόριθμος Ανίχνευσης Βημάτων 47
5.1.2 Αναγνώριση και προ επεξεργασία δεδομένων 48
5.1.3 Φιλτράρισμα Κορυφών 49

5.2.1 Αποτελέσματα Ανίχνευσης Βημάτων 50
ΚΕΦΑΛΑΙΟ 6 56
Συμπεράσματα 56
ΒΙΒΛΙΟΓΡΑΦΙΑ 57
Παράρτημα Α – Κώδικας Συσκευής ανίχνευσης βημάτων 58
OLED.C 58
BNO055.C 65
Save_data.C 69
Main.h 70
Main.c 70
Παράρτημα Β – Firmware αναγνώρισης βημάτων 73
StepTracker.py 73
Παράρτημα Γ – Συμπληρωματικό Υλικό 75

ΚΕΦΑΛΑΙΟ 1

Εισαγωγή

Η ταχεία πρόοδος της τεχνολογίας σε συνδυασμό με τις τάσεις στην αγορά κυρίως στους τομείς της υγείας και του αθλητισμού, έδωσαν την έμπνευση για την υλοποίηση φορητών συσκευών, γνωστές ως “wearable”. Αυτή η φορέσιμη τεχνολογία χρησιμοποιείται συνήθως για την μέτρηση της φυσικής δραστηριότητας του ατόμου, η οποία έχει αποκτήσει μεγάλη σημασία λόγω της αυξανόμενης έμφασης που δίνεται στην βελτίωση της ποιότητας ζωής και κυρίως στην υγεία του ανθρώπου.

Οι φορέσιμες συσκευές παρέχουν πολλές λειτουργίες με πιο κοινή την καταγραφή βημάτων, παρέχοντας πολύτιμες πληροφορίες για τον αριθμό των βημάτων που κάνει ένα άτομο σε καθημερινή βάση, καθώς και την απόσταση που διανύει. Οι πληροφορίες αυτές χρησιμοποιούνται ευρέως τόσο από τους αθλητές, όσο και από άτομα που παρακολουθούν την φυσική τους κατάσταση.

Όπως αναφέρεται παραπάνω η καταγραφή βημάτων και ο υπολογισμός της διανυθείσας απόστασης είναι ιδιαίτερα σημαντικά για τους αθλητές που επιθυμούν να έχουν ακριβή υπολογισμό αποστάσεων και βημάτων. Ωστόσο, οι υπάρχουσες φορέσιμες συσκευές καταγραφής βημάτων βασίζονται κυρίως σε αισθητήρες που τοποθετούνται στο χέρι, με αποτέλεσμα να μην αποδίδουν την ίδια ακρίβεια κατά την βάδιση ή το τρέξιμο. Η τοποθέτηση στο πόδι, όπου πραγματοποιείται η κύρια κίνηση κατά την βάδιση ή το τρέξιμο, μπορεί να προσφέρει πιο ακριβείς μετρήσεις και αξιόπιστα αποτελέσματα.

Η παρούσα διπλωματική εργασία στοχεύει στην ανάπτυξη μίας φορέσιμης στο πόδι συσκευής, όπου μπορεί να μετράει τα βήματα του χρήστη. Η wearable συσκευή περιλαμβάνει τον μικροελεγκτή **PIC18F57Q43**, έναν αισθητήρα επιταχυνσιόμετρο/γυροσκοπίου **BNO055** για την συλλογή και επεξεργασία των δεδομένων κίνησης. Επιπλέον, η συσκευή περιλαμβάνει μια **OLED οθόνη** η οποία χρησιμοποιείται για την άμεση εμφάνιση των μετρήσεων του επιταχυνσιόμετρου. Η οθόνη διευκολύνει σημαντικά την διεξαγωγή πειραμάτων, επιτρέποντας τη γρήγορη και οπτική επιβεβαίωση των δεδομένων. Η επικοινωνία της συσκευής με τον υπολογιστή πραγματοποιείται μέσω σειριακής σύνδεσης με το **CP2102 usb-to-serial**, διευκολύνοντας την μεταφορά δεδομένων και την περαιτέρω επεξεργασία τους. Εν συνεχεία, μέσω ευρετικών αλγορίθμων που εφαρμόζονται σε γλώσσα python γίνεται η αναγνώριση βημάτων για την απόσταση που διανύθηκε.

Η εργασία χωρίζεται στα εξής κεφάλαια: στο **Κεφάλαιο 2** αναλύονται τα τεχνικά χαρακτηριστικά των εξαρτημάτων της συσκευής. Στο **Κεφάλαιο 3** παρουσιάζεται η σχεδίαση και η υλοποίηση της κατασκευής. Το **Κεφάλαιο 4** επικεντρώνεται στο firmware της συσκευής, με έμφαση στην επικοινωνία του μικροελεγκτή με την οθόνη και το επιταχυνσιόμετρο. Στο **Κεφάλαιο 5** επεξηγείται ο αλγόριθμος ανίχνευσης βημάτων. Τέλος στο **Κεφάλαιο 6** γίνεται η τελική ανάλυση των αποτελεσμάτων και συζήτηση για μελλοντικές επεκτάσεις.

ΚΕΦΑΛΑΙΟ 2

2. Ανάλυση Περιφερειακών της Συσκευής

Σε αυτό το κεφάλαιο αναλύονται τα βασικά εξαρτήματα που χρησιμοποιήθηκαν για την κατασκευή της φορέσιμης συσκευής για την καταμέτρηση των βημάτων. Τα εξαρτήματα επιλέχθηκαν βάση των λειτουργικών απαιτήσεων της συσκευής και της ανάγκης για ακριβείς μετρήσεις.

Συγκεκριμένα, παρουσιάζονται τα εξής:

- **PIC18F57Q43:** Ο Pic18f57q43 είναι ο μικροελεγκτής που χρησιμοποιήθηκε σε αυτή την συσκευή και λειτούργησε ως «εγκέφαλος», αναλαμβάνει τον συντονισμό των υπόλοιπων περιφερειακών μονάδων και την επεξεργασία των δεδομένων από τους αισθητήρες.
- **BNO055:** Ο bno055 είναι το αισθητήριο επιταχυνσιόμετρου/γυροσκοπίου όπου είναι υπεύθυνο για την παροχή ακριβών μετρήσεων της επιτάχυνσης και την καταγραφή των κινήσεων στους τρεις άξονες x, y, z.
- **OLED οθόνη:** Η OLED οθόνη χρησιμοποιείται για την απεικόνιση των αποτελεσμάτων του αισθητήρα BNO055 και την διευκόλυνση της διεξαγωγής πειραμάτων, παρέχοντας άμεση ανατροφοδότηση στον χρήστη.
- **CP2102 USB:** Η μονάδα επικοινωνίας CP2102 USB επιτρέπει την σύνδεση του υπολογιστή με την συσκευή, μέσω σειριακής σύνδεσης, διευκολύνοντας την μεταφορά των δεδομένων.

Παρακάτω θα αναλυθεί κάθε μια από αυτές τις μονάδες, περιγράφοντας αναλυτικά τις λειτουργίες, τα βασικά χαρακτηριστικά τους και τον ρόλο τους στην αρχιτεκτονική της συσκευής.

2.1 Εισαγωγή στους μικροελεγκτές

Ο μικροελεγκτής (Microcontroller, MCU) γενικά είναι ένα ολοκληρωμένο ηλεκτρονικό κύκλωμα που έχει ενσωματωμένα, έναν επεξεργαστή, μνήμη (ROM, RAM, Flash memory), άλλες περιφερειακές μονάδες καθώς επίσης και εισόδους/εξόδους για την επικοινωνία με εξωτερικές συσκευές .

Ο μικροελεγκτής έχει σχεδιαστεί για να διαβάζει δεδομένα, πάνω στα οποία κάνει συγκεκριμένες πράξεις και με βάση τις πράξεις αυτές ελέγχει το περιβάλλον του. Η κύρια χρήση του είναι να ελέγχει τη λειτουργία εξωτερικών συσκευών χρησιμοποιώντας ένα πρόγραμμα που είναι αποθηκευμένο στην μνήμη ROM ή flash και συνήθως δεν αλλάζει σε όλη τη διάρκεια ζωής του συστήματος.

Η αρχιτεκτονική ενός μικροελεγκτή μοιάζει με την αρχιτεκτονική ενός μικροϋπολογιστή, η διαφορά ανάμεσα σε έναν μικροελεγκτή και σε έναν μικροεπεξεργαστή είναι, ότι ένας μικροελεγκτής χρησιμοποιεί πολύ μικρότερο σύνολο εντολών απ' ότι ένας μικροεπεξεργαστής.

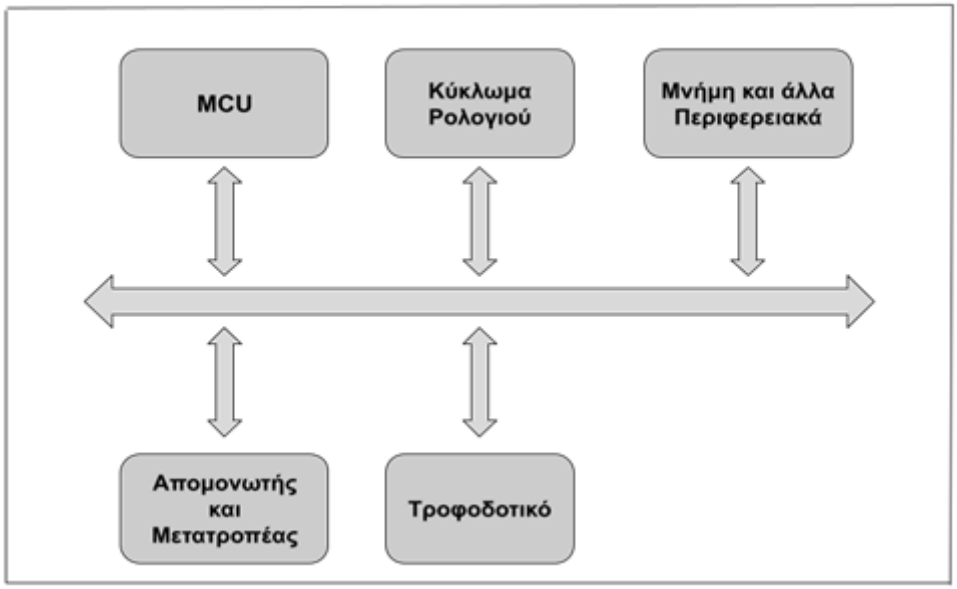
Οι μικροελεγκτές αν και διαθέτουν περιορισμένους πόρους προσφέρουν ορισμένα σημαντικά πλεονεκτήματα που τους καθιστούν ιδανικούς για ενσωματωμένα συστήματα. Η ενσωμάτωση σύνθετων περιφερειακών υποσυστημάτων, όπως για παράδειγμα οι μνήμες, κάνουν τους μικροελεγκτές να έχουν μια αυτονομία με αποτέλεσμα να μη χρειάζονται κανένα άλλο ολοκληρωμένο κύκλωμα για να λειτουργήσουν. Ένα ακόμη πλεονέκτημα των μικροελεγκτών είναι η χαμηλή κατανάλωση ισχύος, η οποία τους επιτρέπει να λειτουργούν αποδοτικά σε συσκευές οι οποίες έχουν περιορισμένες ενεργειακές απαιτήσεις. Επίσης, η ενσωμάτωση μνήμης, επεξεργαστή και διάφορων περιφερειακών μονάδων σε ένα κύκλωμα εξασφαλίζει το χαμηλό κόστος και το μικρότερο μέγεθος, με αποτέλεσμα να το κάνει ιδανικό για συστήματα που έχουν περιορισμένο χώρο. Τέλος, έχουν μειωμένη εκπομπή ηλεκτρομαγνητικών παρεμβολών και μικρότερη ευαισθησία σε αντίστοιχες παρεμβολές λόγω της χαμηλότερης ταχύτητας λειτουργίας. [1]

Οι μικροελεγκτές έχουν ευρεία εφαρμογή σε πολλούς κλάδους, λόγω της ευελιξίας τους, του μικρού μεγέθους και της ενεργειακής τους απόδοσης. Κάποια από τα πεδία εφαρμογής τους είναι τα εξής:

- Αυτοματισμοί και Ρομποτική
- Οικιακές συσκευές
- Αυτοκινητοβιομηχανία
- Ιατρικές συσκευές
- Τηλεπικοινωνίες
- Συστήματα ενέργειας
- Φορητές συσκευές και Wearables

2.1.1 Αρχιτεκτονική ενός μικροελεγκτή

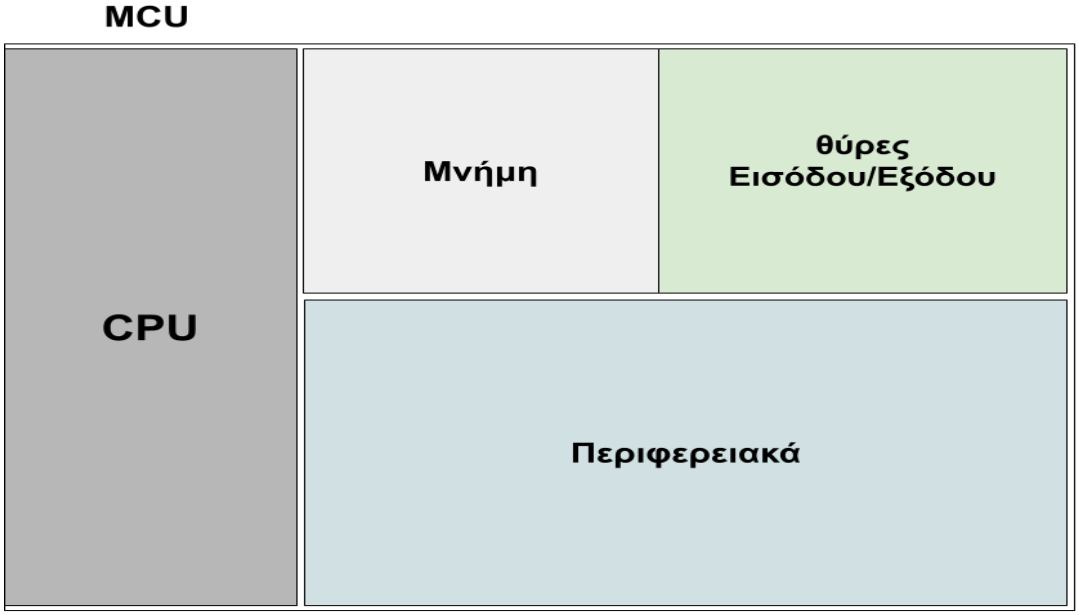
Οι μικροελεγκτές (MCU) είναι ειδικά σχεδιασμένοι, ώστε να επιτελούν συγκεκριμένες λειτουργίες με υψηλή απόδοση και μικρή κατανάλωση ενέργειας. Στο σχήμα 2.1.1 παρουσιάζεται σε μορφή διαγράμματος ένα σύστημα μικροελεγκτή με τα απαραίτητα κυκλώματα υποστήριξης.



Σχήμα 2.1.1 Γενικός σχεδιασμός ενός ενσωματωμένου συστήματος σε μορφή σχεδιαγράμματος.

Το **κύκλωμα ρολογιού** (Clock Circuit) παρέχει τον χρονισμό που απαιτείται για τον συγχρονισμό των λειτουργιών του μικροελεγκτή. Είναι συνήθως ένας κρύσταλλος, όπου δημιουργεί σήματα σταθερής συχνότητας. Αυτός ο κρύσταλλος είναι ο υπεύθυνος για τον σωστό χρονισμό του μικροελεγκτή.

Η **μονάδα μικροελεγκτή MCU** (MicroController Unit) αποτελείται από μια κεντρική μονάδα επεξεργασίας (CPU),τη μνήμη, τα περιφερειακά, τις θύρες εισόδου/εξόδου, και όλα τα ολοκληρωμένα κυκλώματα. Στο σχήμα 2.1.2 παρουσιάζεται μια απλή μορφή περιεχομένου ενός μικροελεγκτή (MCU).



Σχήμα 2.1.2 Μονάδα Μικροελεγκτή MCU

Η **κεντρική μονάδα επεξεργασίας CPU** (Central Processing Unit), είναι υπεύθυνη για την εκτέλεση εντολών και την σωστή επεξεργασία των δεδομένων. Τα κύρια δομικά στοιχεία του είναι ο μονάδα ελέγχου (Control Unit), η αριθμητική μονάδα ALU και κάποιοι εσωτερικοί καταχωρητές που βοηθούν στην επεξεργασία και στην αποθήκευση προσωρινών αποτελεσμάτων. Κάποιοι από τους καταχωρητές είναι ο καταχωρητής εντολών, ο αποκωδικοποιητής εντολών, ο καταχωρητής κατάστασης και πολλοί άλλοι.

Η **Αριθμητική Λογική Μονάδα** (Arithmetic Logi Unit - ALU), αυτή η μονάδα παρέχει στον επεξεργαστή την δυνατότητα να υλοποιεί λογικές και αριθμητικές πράξεις όπως:

- Πρόσθεση
- Αφαίρεση
- Λογικές λειτουργίες (AND, OR, NOT, NAND, NOR, XOR, XNOR)
- Μετατόπιση ή περιστροφή μιας λέξης

Μερικές αριθμητικές μονάδες κάνουν επιπλέον και πολλαπλασιασμό και διαίρεση. Σε αυτή την περίπτωση, οι πράξεις του πολλαπλασιασμού και της διαίρεσης γίνονται με διαδοχικές πράξεις πρόσθεσης και αφαίρεσης κάτω από μια ακολουθία βήμα προς βήμα, όπου αυτή η ακολουθία καθορίζεται ως υπορουτίνα, όπως αναφέρει και ο ΠΟΓΑΡΙΔΗΣ Δ. [2].

Η **Μονάδα ελέγχου** είναι πολύ σημαντικός για την λειτουργία ενός επεξεργαστή, γιατί παράγει τους παλμούς που χρειάζεται ο μικροελεγκτής για τον σωστό χρονισμό του. Επίσης, καλεί τις εντολές του προγράμματος από την μνήμη, καθοδηγεί την αριθμητική και λογική μονάδα ALU για την εκτέλεση αριθμητικών και λογικών πράξεων και τέλος ελέγχει τις θύρες εισόδου/εξόδου για την εισαγωγή και εξαγωγή πληροφοριών.

Η κεντρική μονάδα επεξεργασίας, περιέχει κάποιους εσωτερικούς καταχωρητές όπως αναφέρθηκε παραπάνω. Οι κύριοι τύποι καταχωρητών συνοψίζονται παρακάτω:

1) Ο Καταχωρητής Συσσώρευσης (Accumulator), είναι ο καταχωρητής στον οποίο αποθηκεύονται τα αποτελέσματα των αριθμητικών και λογικών πράξεων της ALU.

2) Ο Καταχωρητής μετρητής Προγράμματος (Program Counter), λειτουργεί ως δείκτης εντολών. Ελέγχει την σειρά με την οποία εκτελούνται οι εντολές που έρχονται από την μνήμη.

3) Ο Καταχωρητής κατάστασης (Status Register) ή Καταχωρητής σημαιών (Flag register), αποτελείται συνήθως από 8 ή περισσότερα ψηφία, καθένα από αυτά παίρνουν τιμές 0 ή 1. Αυτές οι σημαίες χρησιμοποιούνται από τον επεξεργαστή για να αποφασίσει αν πρέπει να γίνει διακλάδωση στην εκτέλεση του προγράμματος υπό συνθήκη. Μερικά από τα flag resister αναφέρονται παρακάτω:

- Δείκτης κρατουμένου(Carry flag)
- Δείκτης μηδενός (Zero flag)
- Δείκτης άρνησης (Negative flag)
- Δείκτης υπερχείλισης (Overflow flag)
- Δείκτης διακοπής (Interrupt flag)
- Δείκτης δεκαδικού (Decimal flag)
- Δείκτης σταματήματος (Brake flag)
- Δείκτης ισοτιμίας (Parity flag)

4) Καταχωρητής στοίβας (Stack) είναι ο καταχωρητής όπου αποθηκεύεται προσωρινά το περιεχόμενο του μετρητή προγράμματος και του καταχωρητή κατάστασης, συνήθως κατά την διάρκεια κλήσεων υπό προγραμμάτων ή διακοπών. Η στοίβα χρησιμοποιεί το σύστημα LIFO (Last In First Out), κατά το οποίο τα δεδομένα που εισέρχονται τελευταία στην στοίβα εξέρχονται πρώτα.

5) Καταχωρητής εντολών (Instruction Register), κρατάει την τρέχουσα εντολή που εκτελεί η CPU. Η εντολή αποκωδικοποιείται και καθορίζει τις επόμενες λειτουργίες που θα εκτελεστούν.

6) Καταχωρητής γενικής χρήσης (General Purpose Register), χρησιμοποιείται για προσωρινή εναποθήκευση πληροφοριών για να αποφευχθεί η χρονοβόρα αποθήκευση στην RAM και στην συνέχεια η ξανά ανάγνωσή τους για επόμενες πράξεις.

Με βάση τα παραπάνω δεδομένα, γίνεται φανερό, ότι οι μικροελεγκτές αποτελούν μια πολύπλευρη και αποτελεσματική λύση για την υλοποίηση ενσωματωμένων συστημάτων. Για την καλύτερη κατανόηση αυτών των αρχών στην πράξη, θα επικεντρωθούμε σε έναν από τους πλέον δημοφιλείς μικροελεγκτές, τον PIC18F57Q43, ο οποίος θα μας επιτρέψει να διερευνήσουμε μερικά από τα χαρακτηριστικά και τις δυνατότητες των MCU.

2.2 Ο μικροελεγκτής PIC18F57Q43

2.2.1 Γενικά χαρακτηριστικά του μικροελεγκτή

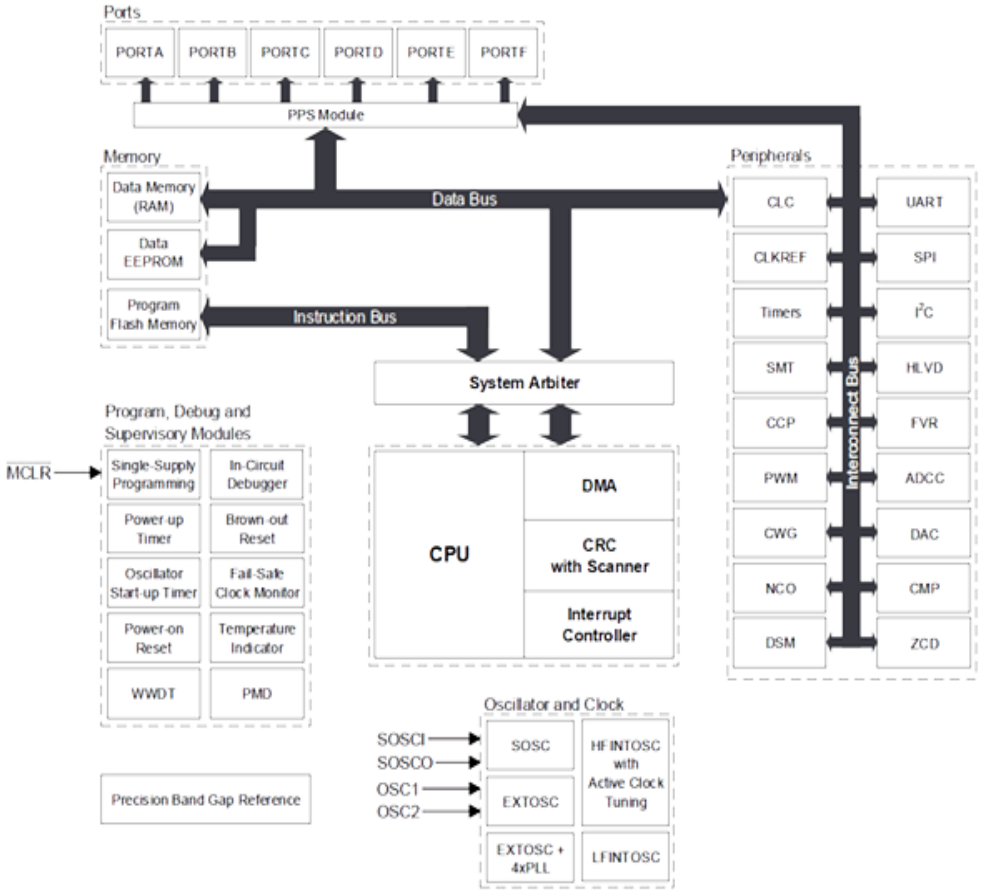
Σε αυτό το κεφάλαιο, θα παρουσιάσουμε τον PIC18F57Q43, ένα μικροελεγκτή της εταιρίας Microchip Technology, ο οποίος αποτελεί ιδανικό παράδειγμα της κατηγορίας 8-bit μικροελεγκτών που χρησιμοποιούνται ευρέως για ποικίλες εφαρμογές. Θα αναλύσουμε την αρχιτεκτονική του, τα χαρακτηριστικά του, και τις δυνατότητες που προσφέρει. Πλήρης και αναλυτική περιγραφή μπορεί να βρεθεί από το φύλλο προδιαγραφών (Datasheet) του μικροελεγκτή στο διαδικτυακό τρόπο της Microchip [3]. Στον Πίνακα 1 φαίνονται τα γενικά χαρακτηριστικά του μικροελεγκτή.

Γενικά Χαρακτηριστικά	PIC18F57Q43
Συχνότητα λειτουργίας	Η συχνότητα λειτουργίας φτάνει έως 64MHz
Flash Memory	128 Kbytes
Data SRAM Memory	8 Kbytes
Data EEPROM	1024 Bytes
Ενσωματωμένες Περιφερειακές Μονάδες	-Πολλαπλές **UART, I2C, SPI** για επικοινωνία.
ADC	12-bit μετατροπές αναλογικού σήματος σε ψηφιακό με επεξεργασία αποτελεσμάτων
DAC	8-bit μετατροπέας ψηφιακού σήματος σε αναλογικό
Comparators	Δύο συγκριτές τάσης
Input/Output Ports	PORTA (RA0-RA7) PORTB (RB0-RB3) PORTC (RC0-RC7) PORTD (RD0-RD7) PORTE (RE0-RE3) PORTF (RF0-RF7)

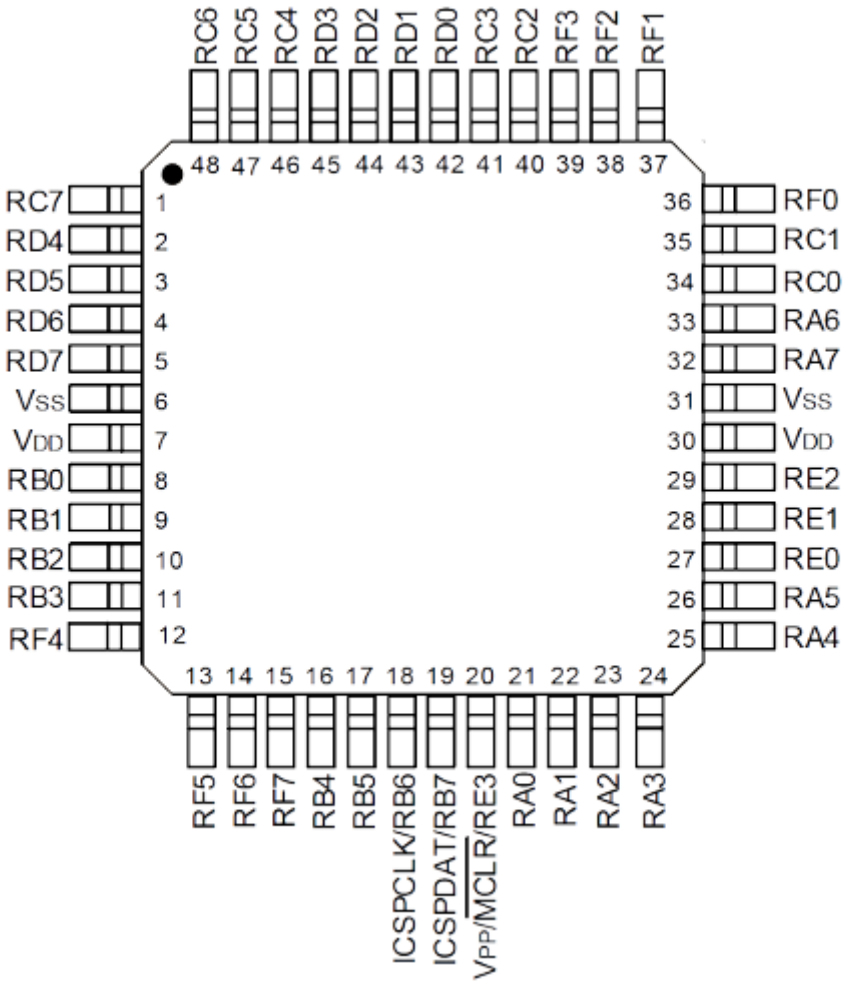
Πίνακας 1 Γενικά χαρακτηριστικά του PIC18F57Q43 [3].

2.2.2 Η αρχιτεκτονική του PIC18F57Q43

Η αρχιτεκτονική του PIC18F57Q43 παρουσιάζεται στην Εικόνα 2.2.1, διαθέτει ένα επεξεργαστή 8-bit αρχιτεκτονικής Harvard με δυνατότητα υποστήριξης 16-bit λειτουργιών για καλύτερη απόδοση. Εμπεριέχει ελεγκτή διακοπών (Interrupt Controller) και πληθώρα περιφερειακών με ελεγκτή για απευθείας πρόσβαση των περιφερειακών στην μνήμη. Η αρχιτεκτονική αυτή συνδυάζει χαμηλή κατανάλωση ενέργειας με υψηλή ταχύτητα εκτέλεσης και πολλά ενσωματωμένα περιφερειακά. Η αρχιτεκτονική Harvard επιτρέπει τη χρήση δύο διαύλων για την πρόσβαση στη μνήμη εντολών και στην μνήμη δεδομένων, αυξάνοντας έτσι την ταχύτητα εκτέλεσης.



Εικόνα 2.2.1 Το μπλοκ διάγραμμα της αρχιτεκτονικής του μικροελεγκτή PIC18F57Q43 [3].



Εικόνα 2.2.2 Διάγραμμα Pin ενός 48-pins TQFP [3].

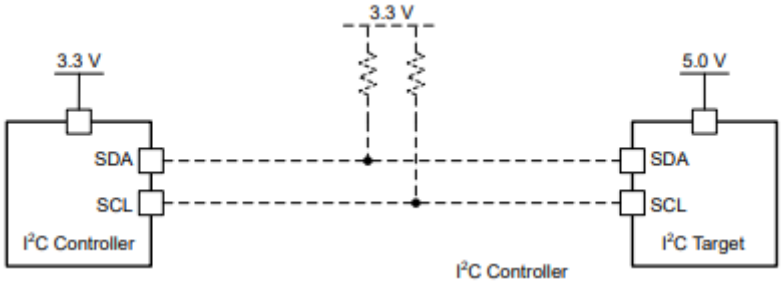
Για τον σχεδιασμό και την υλοποίηση της φορέσιμης συσκευής ανίχνευσης βημάτων που έγινε στα πλαίσια της διπλωματικής εργασίας χρησιμοποιήθηκαν οι παρακάτω μονάδες του μικροελεγκτή:

- I^2C για την επικοινωνία του μικροελεγκτή με τα περιφερειακά
- UART για την μεταφορά δεδομένων

2.2.3 Επικοινωνία I²C

Η I²C επιτρέπει τη σειριακή επικοινωνία με αισθητήρες και άλλα περιφερειακά μέσω δυο καλωδίων (Two-Wire Communication) και χρησιμοποιεί τη γραμμή δεδομένων SDA και τη γραμμή ρολογιού SCL. Το SCL χρησιμοποιείται για να συγχρονίζει την καταγραφή των δεδομένων εντός ή εκτός από την συσκευή ελεγκτή. Το σήμα SDA είναι η σειριακή γραμμή δεδομένων, η οποία χρησιμοποιείται για την μεταφορά των δεδομένων προς ή από την συσκευή προορισμού.

Ο PIC18F57Q43 υποστηρίζει την I²C επικοινωνία προσφέροντας μια ευελιξία. Αυτό σημαίνει, ότι είτε μπορεί να ελέγχει την ροή των δεδομένων ως (host), είτε να απαντάει σε αιτήματα δεδομένων από άλλες συσκευές ως (client). Επίσης, η I²C επικοινωνία παρέχει ένα σύστημα διευθύνσεων που επιτρέπει στον μικροελεγκτή να αναγνωρίζει διαφορετικές συσκευές στον ίδιο δίαυλο επικοινωνίας, καθώς κάθε περιφερειακό έχει μοναδική διεύθυνση. Στην Εικόνα 2.2.3 απεικονίζεται μια απλή σύνδεση με το πρωτόκολλο I²C. Παρατηρούμε, ότι ενδιάμεσα στον controller και στο target υπάρχουν δυο αντιστάσεις, αυτό το σύστημα αντιστάσεων είναι απαραίτητο για την επικοινωνία του I²C και ονομάζεται pull up. Οι αντιστάσεις αυτές χρησιμοποιούνται για να διασφαλίσουμε, ότι οι γραμμές SDA και SCL της I²C επικοινωνίας θα έχουν μια σταθερή υψηλή τάση, όταν δεν βρίσκονται σε χρήση από κάποια συσκευή στον δίαυλο.[5] Για τη συσκευή ανίχνευσης βημάτων χρησιμοποιήθηκε η αναπτυξιακή πλακέτα PIC18F57Q43 Curiosity Nano και σε αυτή την πλακέτα εμπεριέχονται οι pull up αντιστάσεις.



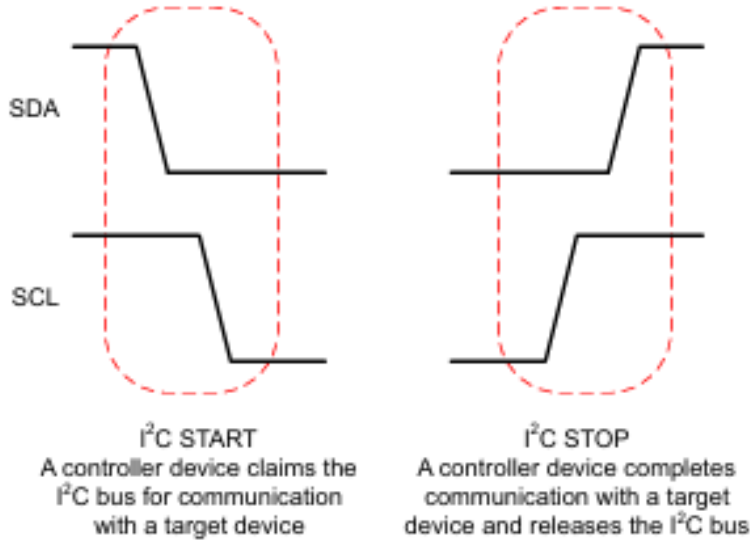
Εικόνα 2.2.3 Σύνδεση για επικοινωνία με το πρωτόκολλο I²C [5]

Το πρωτόκολλο I²C χωρίζεται σε πλαίσια (frames). Για να ξεκινήσει η επικοινωνία με το πρωτόκολλο I²C και να στείλει ένα πλαίσιο διεύθυνσης πρέπει να μπει σε μια κατάσταση από την συσκευή ελέγχου σε **I2C START**. Το πλαίσιο διεύθυνσης

ακολουθείται από ένα ή και περισσότερα πλαίσια δεδομένων (data frames), καθένα από αυτά αποτελείται από ένα byte. Κάθε πλαίσιο περιλαμβάνει επίσης ένα bit αναγνώρισης (ACK-NACK) για να ενημερώνει τον ελεγκτή ότι η συσκευή ή ο ίδιος ο ελεγκτής έλαβε την επικοινωνία.

Από αυτή την κατάσταση διεκδικεί ο ελεγκτής την επικοινωνία μέσω του διαύλου. Για να διεκδικήσει τον δίαυλο ο ελεγκτής, η συσκευή ελέγχου δίνει στη γραμμή SDA χαμηλή τάση τη στιγμή που η γραμμή SCL βρίσκεται σε υψηλή τάση. Με αυτόν τον τρόπο η συσκευή ελέγχου απομονώνει τον δίαυλο I^2C για επικοινωνία με άλλες συσκευές ελέγχου.

Όταν θα σταματήσει την επικοινωνία η συσκευή ελέγχου, θα απελευθερώσει την SCL πρώτα και θα την θέσει με υψηλή τάση και στην συνέχεια την γραμμή SDA αντίστοιχα με υψηλή τάση. Αυτό θα δηλώσει μια κατάσταση που ονομάζεται **I2C STOP**, η οποία δηλώνει, ότι ο δίαυλος είναι ελεύθερος για να επικοινωνήσει με άλλες συσκευές ελέγχου ή ο ίδιος ελεγκτής να ξεκινήσει επικοινωνία με άλλη συσκευή. Στην Εικόνα 2.2.4 παρουσιάζονται οι καταστάσεις I2C START-STOP.



Εικόνα 2.2.4 παρουσίαση καταστάσεων I2C START-STOP

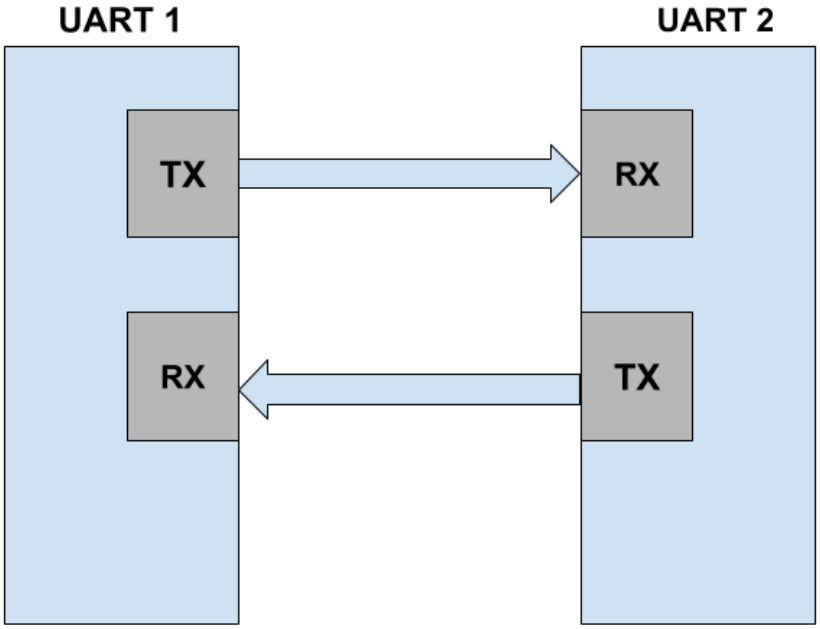
Κάθε συσκευή στο I^2C έχει μια μοναδική διεύθυνση. Όταν ξεκινάει η επικοινωνία με μια συγκεκριμένη συσκευή, ο ελεγκτής χρησιμοποιεί τη διεύθυνση της για να στείλει ή να λάβει δεδομένα στα επόμενα πλαίσια I^2C. Η διεύθυνση αποτελείται από 7 bits, και κάθε συσκευή στο bus έχει μια μοναδική διεύθυνση. Η διεύθυνση αποστέλλεται μέσω της γραμμής SDA ως δεδομένο και η SCL λειτουργεί ως σειριακό ρολόι. Έπειτα το 8^{ο} bit που ακολουθεί την διεύθυνση είναι το bit read/write (R/W). Αν το bit είναι 1, τότε ο ελεγκτής ζητά να διαβάσει δεδομένα από την συσκευή ενώ αν το bit είναι 0 τότε ο ελεγκτής ζητάει να γράψει δεδομένα στην συσκευή.

Μετά από κάθε byte επικοινωνίας χρησιμοποιείται ένα 9ο bit για να επιβεβαιώσει ότι η επικοινωνία ήταν επιτυχείς. Στο τέλος της αποστολής του byte διεύθυνσης, η συσκευή τραβάει την γραμμή SDA χαμηλά και κατά τον παλμό της SCL για να δείξει ότι έλαβε τη διεύθυνση. Αυτό είναι το bit ACK, αν το bit παραμείνει υψηλό, καμιά συσκευή δεν έλαβε την διεύθυνση και η επικοινωνία απέτυχε. Σε αυτή την περίπτωση μιλάμε για NACK (no Acknowledge). Το ACK είναι πολύ χρήσιμο και για αποσφαλμάτωση (debugging). Η απουσία του δείχνει είτε ότι συσκευή δεν έλαβε τη σωστή διεύθυνση, είτε ότι ο ελεγκτής δεν έλαβε τα αναμενόμενα δεδομένα.

2.2.4 Μονάδα UART

Η μονάδα UART (Universal Asynchronous Receiver-Transmitter) είναι ένα από τα πιο βασικά περιφερειακά του μικροελεγκτή και χρησιμοποιείται για ασύγχρονη σειριακή επικοινωνία ανάμεσα σε δύο συσκευές. Η επικοινωνία με την χρήση της UART είναι αμφίδρομη (Full duplex : και οι δύο συσκευές στέλνουν και δέχονται σήματα την ίδια στιγμή).

Η σύνδεση πομπού και δέκτη με την χρήση UART γίνεται μέσω δύο καλωδίων, τα οποία είναι το TD (Transmitter Data) και το RD (Receiver Data), και θα πρέπει το TD της μίας συσκευής, να συνδέεται με το RD της άλλης συσκευής. Στην εικόνα 2.2.5 παρουσιάζεται μια απλή σύνδεση πομπού δέκτη με την χρήση UART.



Εικόνα 2.2.5 Επικοινωνία δύο UART .

Στην ασύγχρονη επικοινωνία ο πομπός και ο δέκτης δεν έχουν κοινό σήμα ρολογιού, αυτό απλοποιεί αρκετά το πρωτόκολλο επικοινωνίας UART. Βέβαια, η ασύγχρονη επικοινωνία έχει μία ιδιαιτερότητα όσον αφορά το Baud Rate ανάμεσα στον πομπό και στον δέκτη. Το Baud Rate πρέπει να είναι ίδιο στον πομπό και στον δέκτη για την σωστή επικοινωνία με το πρωτόκολλο UART. Στον Πίνακα 2.2 απεικονίζονται κάποια βασικά Baud Rate που χρησιμοποιούνται σήμερα.

Baud Rate (bps)	
1	2400
2	9600
3	19200
4	38400
5	57600
6	115200
7	230400
8	460800
9	961600

Πίνακας 2.2 τα βασικά baud rate για την επικοινωνία με την μονάδα UART

2.2.4.1 Πλαίσιο χαρακτήρων

Η επικοινωνία με την UART γίνεται με την αποστολή πλαισίων δεδομένων (data frames), όπου το καθένα περιέχει από έναν χαρακτήρα. Το μέγεθος του πλαισίου χωρίς parity bit μπορεί να είναι 10 bits. Από αυτά τα 10 bits υπάρχουν δύο σημαντικά bits τα οποία είναι το start bit και το stop bit, τα οποία παίζουν καθοριστικό ρόλο για την επικοινωνία πομπού και δέκτη. Λόγω της ασύγχρονης, επικοινωνίας ο δέκτης πρέπει να ειδοποιηθεί από τον πομπό ότι θα αποσταλούν δεδομένα, το οποίο επιτυγχάνεται με την βοήθεια του start bit. Όταν η γραμμή αποστολής δεδομένων TD από λογικό ΄1΄ γίνει λογικό ΄0΄ τότε ο δέκτης αντιλαμβάνεται, ότι θα ακολουθήσουν κάποια bit δεδομένων, τα οποία μπορεί να είναι 5 bits έως 9 bits, ανάλογα την περίπτωση. Στο παράδειγμα τις εικόνας 2.2.6 τα bits δεδομένων είναι 8. Στην συνέχεια ακολουθεί το stop bit, το οποίο δηλώνει στον δέκτη ότι σταματάει η εισροή δεδομένων. Πολλές φορές πριν το stop bit υπάρχει ακόμη ένα bit, το οποίο ονομάζεται bit ισοτιμίας (parity bit) και βοηθάει στο να ανιχνεύονται τυχόν εσφαλμένα bits. Αξίζει εδώ να αναφερθεί, ότι στην περίπτωση που έχουμε 2 ανεστραμμένα bits δεν μπορεί να τα ανιχνεύσει. Το bit ισοτιμίας λειτουργεί κατά τρόπο, όπου αν το πλήθος των άσσων στο bit δεδομένων είναι άρτιο τότε το parity bit είναι 0, ενώ αν το πλήθος των άσσων είναι περιττός, τότε

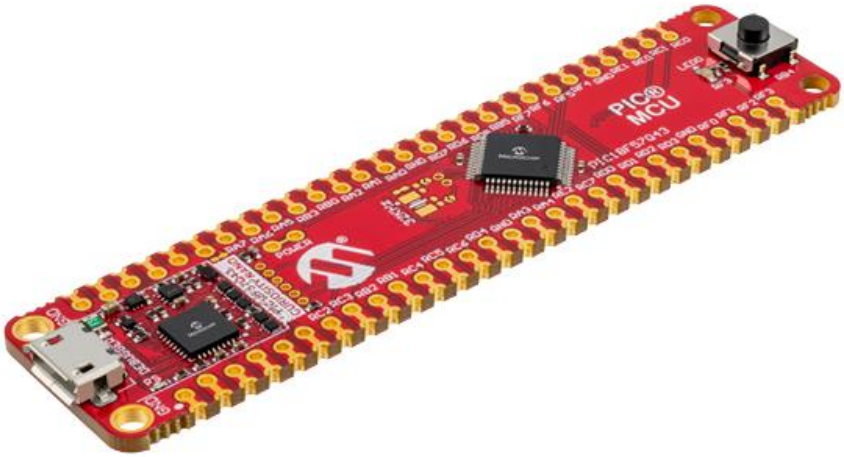
το bit ισοτιμίας θα είναι 1. Με αυτό τον τρόπο μπορεί ο δέκτης να καταλάβει αν έχει συμβεί περιττός αριθμός σφαλμάτων.

Αριθμός Bit	1	2	3	4	5	6	7	8	9	10
	Start Bit	5-8 data bits								STOP Bit
	Start	Data 0	Data 1	Data 2	Data 3	Data 4	Data 5	Data 6	Data 7	Stop

Εικόνα 2.2.6 Πλαίσιο χαρακτήρων 10bits.

2.3 PIC18F57Q43 Curiosity Nano Evaluation Kit

Όπως αναφέρθηκε και στο κεφάλαιο 2.2.3, για την εκπόνηση της εργασίας χρησιμοποιήθηκε η αναπτυξιακή πλακέτα Curiosity Nano Evaluation Kit της Microchip. Η οποία ενσωματώνει τον μικροελεγκτή PIC18F57Q43 που αναλύθηκε στα προηγούμενα κεφάλαια.



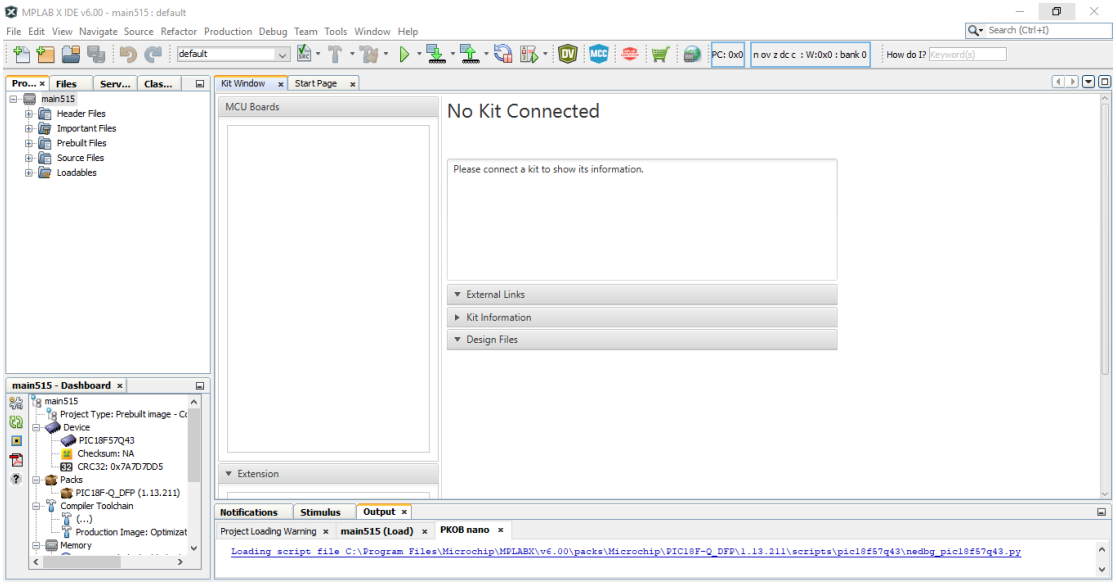
Εικόνα 2.3.1 Pic18F57Q43 Curiosity Nano Evaluation Kit. [4]

Το Curiosity Nano Evaluation Kit της Microchip είναι ένα σετ που φτιάχτηκε για την απλή και άμεση μάθηση και ανάπτυξη εφαρμογών με μικροελεγκτή PIC από την σειρά 18. Το kit αυτό περιλαμβάνει τον μικροελεγκτή PIC18F57Q43, που δίνει έναν πλήρη και οικονομικό τρόπο δοκιμών και ανάπτυξη εφαρμογών. Έχει σύνδεση με USB για εύκολη επικοινωνία με τον υπολογιστή, ενώ υποστηρίζεται από το περιβάλλον MPLAB X IDE καθώς και από το MCC (Microchip Code Configuration), που επιτρέπει γρήγορη μορφοποίηση και ανάπτυξη λογισμικού.

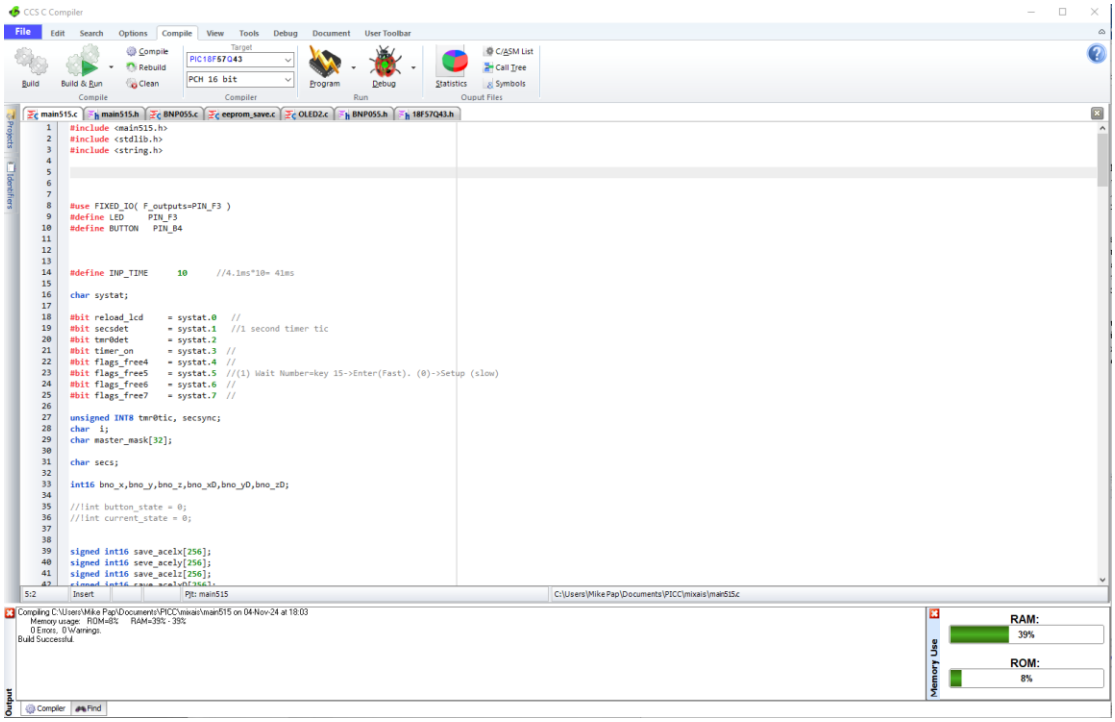
Το set αυτό δίνει πλήρη πρόσβαση σε όλους τους I/O ακροδέκτες του μικροελεγκτή και περιλαμβάνει μέρη, όπως το UART,I2C,SPI και άλλες συσκευές. Για όλα τα

παραπάνω, το σετ αυτό θεωρείται πολύ χρήσιμο ως προς την μάθηση και εφαρμογή ενσωματωμένων συστημάτων και ηλεκτρονικών εργασιών. Επιπλέον, διαθέτει κύκλωμα για debugger/programmer για τον μικροελεγκτή, επιτρέποντας στους χρήστες να κάνουν άμεσες δοκιμές στον προγραμματισμό και να παρακολουθούν τη συμπεριφορά του μικροελεγκτή στο αναπτυξιακό τους στάδιο. Τέλος, η αναπτυξιακή πλακέτα περιλαμβάνει μια δίοδο LED και ένα Push-button, όπου συνδέεται με τους ακροδέκτες RF3,RB4, τα οποία χρησιμοποιήθηκαν στην παρούσα διπλωματική.

Όσον αφορά στην ανάπτυξη του κώδικα στην παρούσα εργασία, χρησιμοποιήθηκε ο CCS C Compiler σε συνδυασμό με το MPLAB X IDE. Η διαδικασία ήταν η ακόλουθη: Μετά την εγγραφή του προγράμματος στην πλατφόρμα CCS γινόταν εξαγωγή του αρχείου .hex και εισαγόταν στην εφαρμογή MPLAB X για να γίνει η μεταφορά του προγράμματος στην αναπτυξιακή πλακέτα Curiosity nano. Αξίζει να αναφερθεί εδώ, ότι η πλατφόρμα CCS επιλέχτηκε λόγω του φιλικότερου interface και της καλύτερης εμφάνισης των διάφορων λαθών κατά την διάρκεια εγγραφής του κώδικα. Στην Εικόνα 2.3.2 και Εικόνα 2.3.3 παρουσιάζονται τα interfaces των δύο λογισμικών.



Εικόνα 2.3.2 Περιβάλλον λογισμικού MPLAB X IDE.



Εικόνα 2.3.3 Περιβάλλον λογισμικού CCS C Complier.

2.4 BNO055

Ο BNO055 είναι ένας αισθητήρας της εταιρίας BOSCH ο οποίος διαθέτει τριών αξόνων 14-bit επιταχυνσιόμετρο (accelerometer), τριών αξόνων 16-bit γυροσκόπιο (gyroscope) και τριών αξόνων μαγνητόμετρο (magnetometer). Εκτός από τις βασικές λειτουργίες επιτάχυνσης, γωνιακής ταχύτητας και μαγνητικού πεδίου, ο αισθητήρας προσφέρει πέντε διαφορετικές λειτουργίες σύντηξης δεδομένων (sensor fusion). Αυτές οι λειτουργίες παρουσιάζονται στον παρακάτω Πίνακα 2.3.

Λειτουργίες BNO055	
IMU (Inertial Measurement Unit - Μονάδα Μέτρησης Αδράνειας)	Σύντηξη των δεδομένων από το γυροσκόπιο και το επιταχυνσιόμετρο
Compass (Πυξίδα)	Σύντηξη των δεδομένων από επιταχυνσιόμετρο και μαγνητόμετρο για τον υπολογισμό κατεύθυνσης και προσανατολισμού

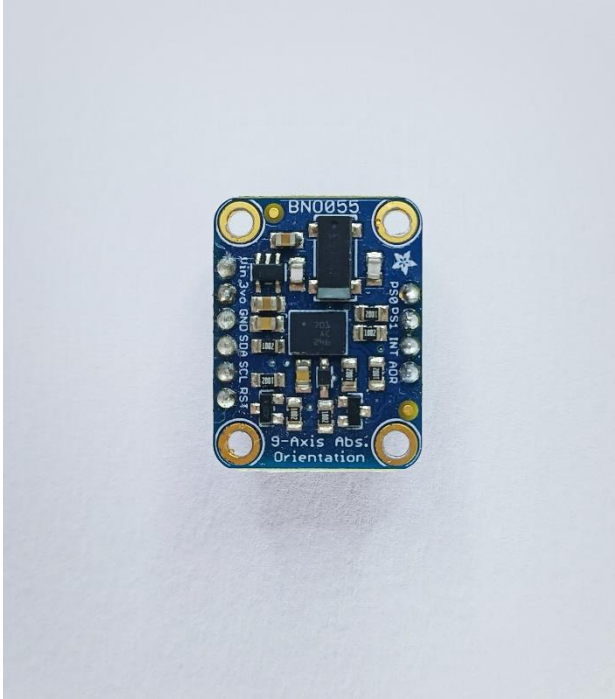
M4G (Magnet for Gyroscope)	Με χαμηλότερη κατανάλωση ενέργειας πραγματοποιεί την ίδια λειτουργία με την IMU
NDOD_FMC_OFF	Σύντηξη του επιταχυνσιόμετρου, του γυροσκόπιού και του μαγνητόμετρου, παρέχοντας 9 βαθμούς ελευθερίας (Nine Degrees of Freedom - NDOF)
NDOF	Η λειτουργία της είναι παρόμοια με την NDOF_FMC_OFF, με την μόνη διαφορά ότι εδώ η δυνατότητα γρήγορης μαγνητικής βαθμονόμησης (Fast Magnetic Calibration - FMC) είναι ενεργοποιημένη.

Πίνακας 2.3 Πρόσθετες λειτουργίες του BNO055.

2.4.1 Adafruit-BNO055 9-Axis Abs. Orientation

Για την εκπόνηση της διπλωματικής εργασίας χρησιμοποιήθηκε η αναπτυξιακή πλακέτα της Adafruit-BNO055 9-Axis που ενσωματώνει τον αισθητήρα BNO055 της BOSCH όπου αναφέρεται στο Κεφάλαιο 2.4. Όπως παρατηρούμε και στο όνομα της πλακέτας, διαθέτει ένα σύστημα 9 αξόνων το οποίο περιλαμβάνει ένα επιταχυνσιόμετρο τριών αξόνων, ένα γυροσκόπιο τριών αξόνων και ένα μαγνητόμετρο τριών αξόνων για την παρακολούθηση της κίνησης και του προσανατολισμού. Η επικοινωνία αυτής της αναπτυξιακής πλακέτας με κάποιον μικροελεγκτή γίνεται με το πρωτόκολλο I2C,UART εξασφαλίζοντας την αξιοπιστία. Σύμφωνα με το άρθρο [7], η αναπτυξιακή πλακέτα δεν λειτουργεί σωστά με βάση το πρωτόκολλο I2C με ορισμένες οικογένειες chip, όπως Espressif ESP32, ESP32-S3 και NXP i.MXRT1011. Επίσης αναφέρεται, ότι υπάρχουν κάποιες δυσλειτουργίες όταν χρησιμοποιούνται πολυπλέκτες I2C. Στην παρούσα διπλωματική εργασία η επικοινωνία της αναπτυξιακής πλακέτας PIC18F57Q43 Curiosity Nano και της αναπτυξιακής πλακέτας Adafruit-BNO055 έγινε με το πρωτόκολλο I2C χωρίς να χρησιμοποιηθεί πολυπλεξία του πρωτοκόλλου. Επίσης, δεν παρουσιάστηκε κάποιο πρόβλημα στην επικοινωνία των δύο αναπτυξιακών πλακετών. Εν συνεχεία, για την επικοινωνία των δύο αναπτυξιακών πλακετών με το πρωτόκολλο I2C δεν χρειάστηκαν pull up αντιστάσεις, γιατί η αναπτυξιακή πλακέτα Adafruit-BNO055 στα Pin SDA και SCL, με τα οποία γίνεται η επικοινωνία του πρωτόκολλο I2C, διαθέτει pull up αντιστάσεις 10K σε καθένα από αυτά.

Στην εικόνα 2.4.1 παρουσιάζεται η αναπτυξιακή πλακέτα Adafruit-BNO055.



Εικόνα 2.4.1 Η αναπτυξιακή πλακέτα Adafruit-BNO055 9-Axis

2.5 Οθόνη OLED SDD1306 128x64

Η OLED (Organic Light Emitting Diode) οθόνη είναι μία υψηλής ευκρίνειας οθόνη που χρησιμοποιείται ευρέως σε εφαρμογές ενσωματωμένων συστημάτων. Οι OLED οθόνες λειτουργούν με την χρήση οργανικών ενώσεων που εκπέμπουν φως όταν διαρρέονται από ηλεκτρικό ρεύμα. Αυτό βελτιώνει την κατανάλωση ενέργειας και τις κάνει αποδοτικότερες από τις κλασικές LCD οθόνες, γιατί δεν χρειάζονται οπίσθιο φωτισμό. Στα πλαίσια της διπλωματικής εργασίας χρησιμοποιήθηκε η OLED οθόνη ανάλυσης 128x64 pixels που βασίζεται στον ελεγκτή SSD1306, ο οποίος είναι υπεύθυνος για την διαχείριση των pixel της οθόνης και την σύνδεση με τον μικροελεγκτή. Το SSD 1306 διαθέτει έλεγχο αντίθεσης, μνήμη RAM και ταλαντωτή, ο οποίος μειώνει τον αριθμό των εξωτερικών εξαρτημάτων και την κατανάλωση της ενέργειας.

Η επικοινωνία της OLED οθόνης με την αναπτυξιακή πλακέτα PIC18F57Q43 Curiosity nano έγινε με το πρωτόκολλο I2C όπου και σε αυτή την περίπτωση δεν χρειαστήκαν οι pull up αντιστάσεις στα pins SCL και SDA της οθόνης, γιατί η σύνδεση έγινε στα κατάλληλα pins του PIC18F57Q43 Curiosity nano, όπου διαθέτουν τις 10k pull up αντιστάσεις. Στην Εικόνα 2.5.1 που ακολουθεί απεικονίζεται μια οθόνη OLED SSD1306 128x64. Επίσης, η OLED οθόνη διαθέτει 4 pins, τα οποία περιγράφονται στον Πίνακα 2.4 παρακάτω.

Εικόνα 2.5.1 Οθόνη OLED SDD1306 128x64 [8]

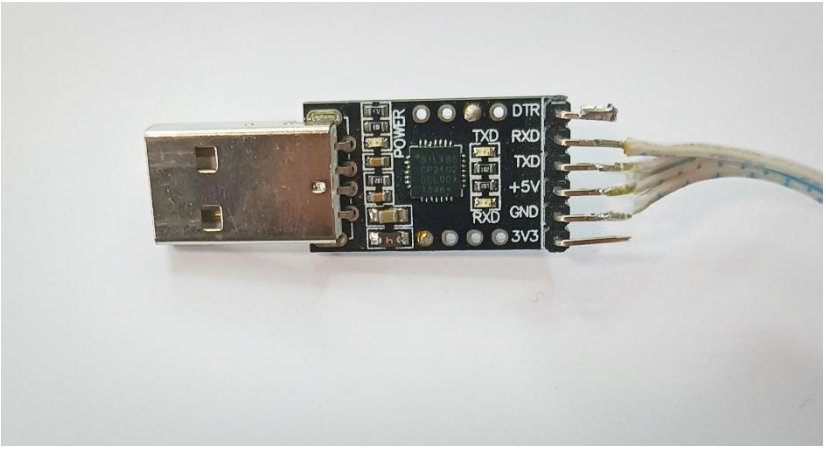
Pinout	Περιγραφή
VCC	Ακίδα τροφοδοσίας από (1,65V – 5V)
GND	Ακίδα Γείωση
SCL	Εκτελεί σήμα ρολογιού
SDA	Λαμβάνει τα δεδομένα από την συσκευή ελέγχου

Πίνακας 2.4 Ακίδες της OLED SSD1306 οθόνης.

2.6 CP2102 USB 2.0

Ο CP2102 είναι ένα ολοκληρωμένο κύκλωμα, το οποίο μετατρέπει μια θύρα USB σε UART, προσφέροντας μια αποδοτική και απλή λύση για την αναβάθμιση συστημάτων από RS232 σε USB. Αυτό διευκολύνει την επικοινωνία ενσωματωμένων συστημάτων με τον υπολογιστή ή με άλλες συσκευές που υποστηρίζουν θύρα USB.

Για να αναγνωριστεί από το λειτουργικό σύστημα που χρησιμοποιήθηκε (Windows 10) χρειάστηκε ένας οδηγός, ο οποίος αντλήθηκε από το [9]. Στην Εικόνα 2.6.1 που ακολουθεί, απεικονίζεται ο CP2102 USB 2.0. Στα πλαίσια της διπλωματικής εργασίας, ο CP2102 USB χρησιμοποιήθηκε για την μεταφορά των δεδομένων από την συσκευή μέτρησης βημάτων στον υπολογιστή. Θα μπορούσε να είχε χρησιμοποιηθεί η θύρα USB της αναπτυξιακής πλακέτας PIC18F57Q43 Curiosity nano για την αποστολή δεδομένων στον υπολογιστή, αλλά αυτή η θύρα χρησιμοποιήθηκε για την τροφοδοσία της συσκευής με power bank, έτσι ώστε να είναι αυτόνομη.



Εικόνα 2.6.1 CP2102 μετατροπές USB σε UART.

Ο CP2102 USB 2.0 διαθέτει 14 pinout από τα οποία χρησιμοποιήθηκαν τα 4. Τα pins αυτά αναλύονται στο Πίνακα 2.5.

Pinout	CP2102 USB 2.0
GND	Ακίδα Γείωση
5V	Ακίδα τροφοδοσίας
RXD (Receiver Data)	Είσοδος δεδομένων, που επιτρέπει στην συσκευή να λαμβάνει πληροφορία από την USB θύρα
TRD (Transmitter Data)	Έξοδος δεδομένων, που επιτρέπει την μετάδοση δεδομένων από την συσκευή προς τον υπολογιστή σε άλλη συσκευή.

Πίνακας 2.5 Pinout που χρησιμοποιήθηκαν από τον CP2102 USB 2.0

ΚΕΦΑΛΑΙΟ 3

3. Σχεδιασμός και υλοποίησης κατασκευής

3.1 Σχεδίαση και υλοποίηση κυκλώματος

Σε αυτό το κεφάλαιο εξηγούνται αναλυτικά οι συνδέσεις και ο τρόπος κατασκευής της φορέσιμης συσκευής ανίχνευσης βημάτων. Αρχικά, στην Εικόνα 3.1.1 απεικονίζεται ο σχεδιασμός του κυκλώματος στο EasyEDA. Πρώτα έγινε η σύνδεση ανάμεσα στον μικροελεγκτή PIC18F57Q43 με την αναπτυξιακή πλακέτα BNO055. Η σύνδεση των pins βρίσκεται και στον Πίνακα 3.1. Για αυτή την σύνδεση επιλέχθηκαν τα pin RC3,RC4 του μικροελεγκτή PIC18F57Q43, γιατί όπως αναφέρθηκε και στο Κεφάλαιο 2.4.1, ο BNO055 διαθέτει στα pin SDA,SCL pull up αντιστάσεις, οπότε δεν χρειάστηκε η επιλογή των Pins του μικροελεγκτή, όπου διαθέτουν pull up αντιστάσεις.

BNO055	PIC18F57Q43
VDD	VDD
GND	GND
SDA	RC4
SCL	RC3

Πίνακας 3.1.1 Σύνδεση BNO055 και PIC18F57Q43

Από την άλλη, η σύνδεση με την OLED οθόνη, χρειάστηκε να γίνει στα κατάλληλα pin του μικροελεγκτή, τα οποία είναι τα RB1,RB2 και διαθέτουν pull up αντιστάσεις. Αυτό έγινε, γιατί όπως αναφέρθηκε στο Κεφάλαιο 2.5, η OLED οθόνη δεν διαθέτει στα pins SDA,SCL pull up αντιστάσεις.

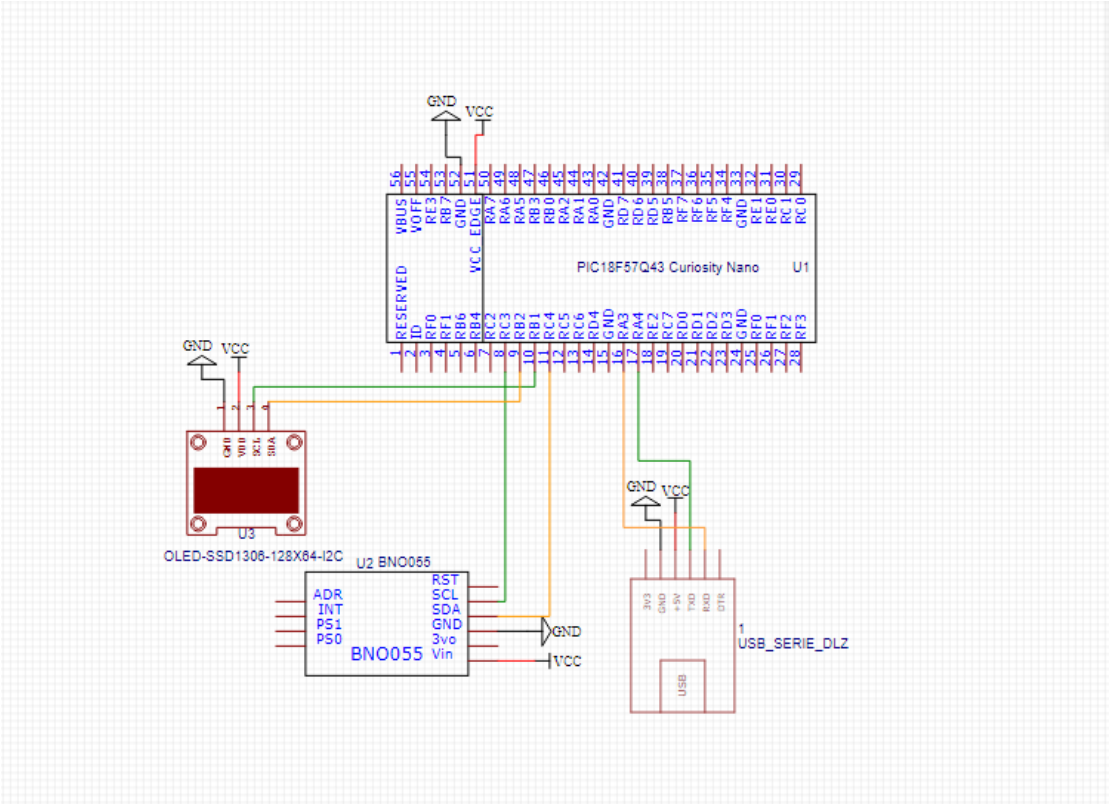
OLED Οθόνη	PIC18F57Q43
VDD	VDD
GND	GND
SDA	RB1
SCL	RB2

Πίνακας 3.1.2 Σύνδεση OLED Οθόνη και PIC18F57Q43

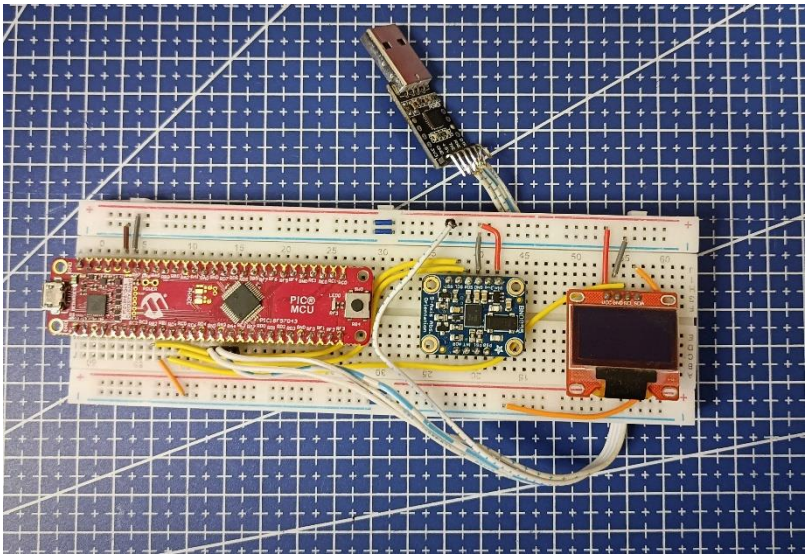
Τέλος όσο αφορά στην σύνδεση του CP2102 USB 2.0 με τον μικροελεγκτή, έγινε στα pins RA3,RA4, τα οποία είναι τα κατάλληλα για την επικοινωνία με την UART3 του μικροελεγκτή.

CP2102 USB 2.0	PIC18F57Q43
VDD	VDD
GND	GND
RXD	RA3
TXD	RA4

Πίνακας 3.1.3 Σύνδεση OLED Οθόνη και PIC18F57Q43



Εικόνα 3.1.1 Σχεδίαση του κυκλώματος στο EasyEDA



Εικόνα 3.1.2 Το κύκλωμα κατασκευασμένο πάνω σε breadboard.

Το κύκλωμα όπως απεικονίζεται και στην Εικόνα 3.1.2 υλοποιήθηκε πάνω σε raster για την εκπόνηση της παρούσας διπλωματικής εργασίας. Επίσης, η τροφοδοσία του όλου συστήματος έγινε με power bank μέσω της θύρας micro USB της αναπτυξιακής πλακέτας PIC18F57Q43 Curiosity nano. Οι ενεργειακές απαιτήσεις της συσκευής ήταν χαμηλές, μετά από μετρήσεις η συσκευή καταναλώνει σε λειτουργεία μέσο όρο 50mA και αντίστοιχα 0.25mW οπότε η ενέργεια ενός power bank κάλυπτε τις ανάγκες του συστήματος. Οι δύο οριζόντιες γραμμές στο πάνω μέρος του raster χρησιμοποιήθηκαν ως γραμμές τροφοδοσίας +5V και γείωσης για τα περιφερειακά BNO055 , OLED οθόνης και CP2102 USB2.0. Αυτές οι δυο γραμμές έχουν τους συμβολισμούς (+) και (-) και είναι χρωματισμένες με κόκκινο και μπλε. Η γραμμή με το κόκκινο χρώμα και το σύμβολο (+) συνδέθηκε με τα +5V της αναπτυξιακής πλακέτας Curiosity nano για την παροχή ρεύματος και στα υπόλοιπα περιφερειακά. Αντίστοιχα, η μπλε γραμμή με το σύμβολο (-) συνδέθηκε στην γείωση GND της αναπτυξιακής πλακέτας Curiosity nano για την παροχή γείωσης στα υπόλοιπα περιφερειακά.

3.2 Σχεδίαση και υλοποίηση της θήκης

Για την ολοκλήρωση της φορέσιμης συσκευής και την δυνατότητα στερέωσης της στο πόδι του χρήστη, σχεδιάστηκε και κατασκευάστηκε μια ειδική θήκη. Η θήκη αυτή εξυπηρετεί την στερέωση της συσκευής μέτρησης βημάτων, προστατεύοντας το κύκλωμα, το οποίο είναι συναρμολογημένο στο raster.

Η σχεδίαση της θήκης πραγματοποιήθηκε στο ελεύθερο λογισμικό Freecad, το οποίο παρέχει εργαλεία για την τρισδιάστατη μοντελοποίηση. Αφού ολοκληρώθηκε ο σχεδιασμός, η θήκη εκτυπώθηκε σε εκτυπωτή 3D Printer, χρησιμοποιώντας υλικό PLA. Η διαδικασία σχεδιασμού και κατασκευής επέτρεψε τη δημιουργία μιας θήκης που είναι ανθεκτική, ελαφριά και προσαρμοσμένη στις διαστάσεις της συσκευής, ενώ

ταυτόχρονα διαθέτει ανοίγματα για την σωστή λειτουργία των αισθητήρων και την παροχή τροφοδοσίας.

Η θήκη είναι κατασκευασμένη με τέτοιο τρόπο, ώστε να επιτρέπει τη σταθερή στερέωση στο πόδι με τη χρήση δυο ταινιών Velcro, εξασφαλίζοντας την σταθερότητα της συσκευής ακόμη και σε συνθήκες κίνησης.

3.2.1 Σχεδίαση της θήκης στο Freecad

Η σχεδίαση της θήκης, όπως αναφέρθηκε και παραπάνω έγινε στο λογισμικό Freecad, ένα λογισμικό ανοιχτού κώδικα για εύκολη τρισδιάστατη σχεδίαση, όπου παρέχει όλα τα απαραίτητα εργαλεία για τον σχεδιασμό αντικειμένων κατάλληλων για εκτύπωση σε 3D Printer.

Πρώτα πάρθηκαν κάποιες προδιαγραφές για την θήκη οι οποίες ήταν:

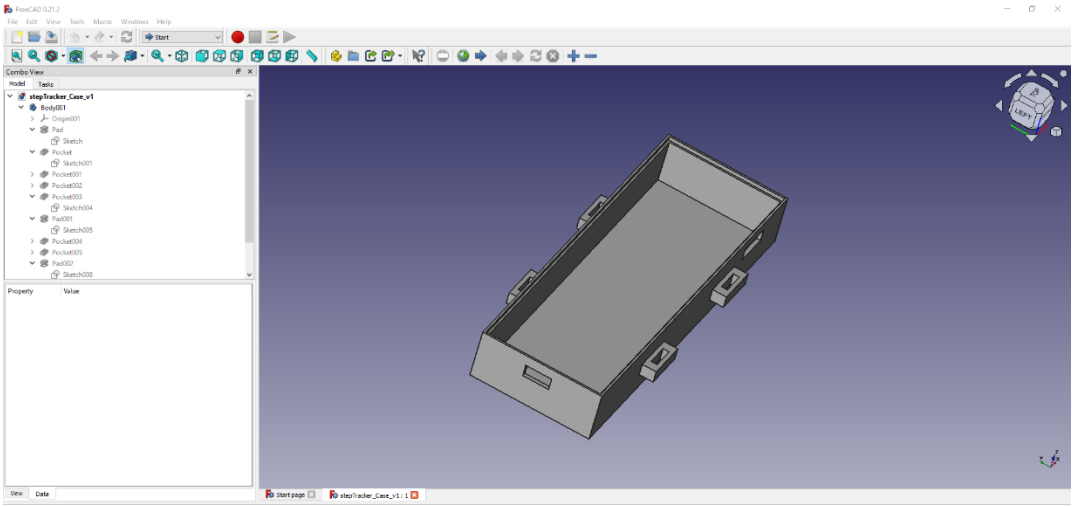
- **Διαστάσεις**: Η θήκη έπρεπε να προσαρμοστεί στις διαστάσεις του raster και των υπόλοιπων ηλεκτρονικών εξαρτημάτων.
- **Λειτουργικότητα**: Έπρεπε να διαθέτει κατάλληλα ανοίγματα για την σύνδεση καλωδίων, όπως την φόρτιση και την έξοδο του καλωδίου cp2102, το οποίο είναι για την μεταφορά των δεδομένων στον υπολογιστή.
- **Φορητότητα**: Να σχεδιαστεί με τέτοιο τρόπο, ώστε να επιτρέπει τη σωστή στερέωση της συσκευής στο πόδι του χρήστη.
- **Ασφάλεια**: Να προστατεύει τα ηλεκτρονικά εξαρτήματα από χτυπήματα.

Στην συνέχεια, έγινε η δημιουργία του μοντέλου στο Freecad. Η σχεδίαση της θήκης έγινε σε δύο στάδια. Αρχικά σχεδιάστηκε η βασική θήκη που φιλοξενεί το raster και εν συνεχεία το καπάκι της θήκης. Τα εργαλεία που χρησιμοποιήθηκαν από το λογισμικό FreeCad ήταν τα εξής:

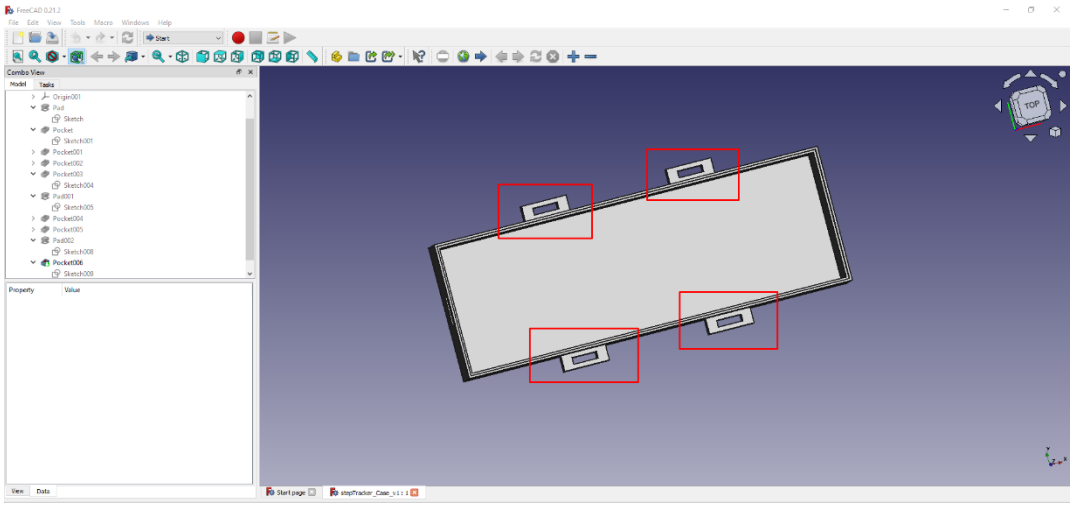
- Part Design: Παρέχει λειτουργίες για πλήρη παραμετρική μοντελοποίηση στερεών (Solid Modelling), όπως συγχώνευση (fuse), αφαίρεση (Cut), καμπυλώματα (fillets), δημιουργία οπών (holes), σχισμών (slots) και εγκοπών (rips) βασισμένες σε σκίτσα (Sketches).
- Sketcher: Με αυτό το εργαλείο γίνεται η κατασκευή σημείων, γραμμών, τόξων και καμπυλών ελεύθερης μορφής πάνω σε ένα επίπεδο δυσδιάστατο (2D), με τη δυνατότητα ορισμού περιορισμών (constrains) μεταξύ τους, όπως διαστάσεις, παραλληλισμός και άλλα.

3.2.2 1ο Στάδιο σχεδιασμού θήκης

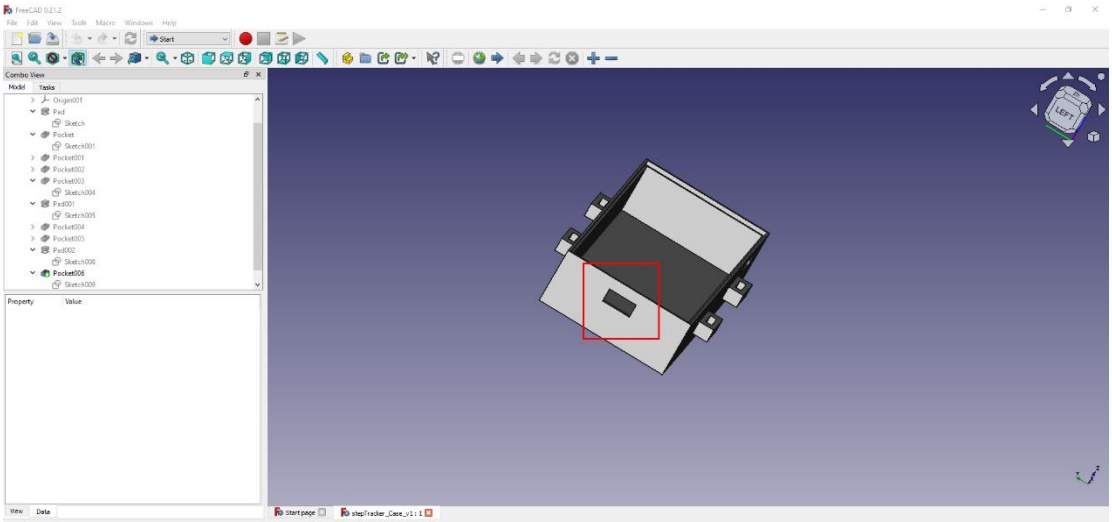
Κατά το πρώτο στάδιο σχεδίασης της θήκης υλοποιήθηκε το βασικότερο κομμάτι, το οποίο φιλοξενεί τον raster μαζί με όλα τα ηλεκτρονικά εξαρτήματα. Αρχικά, επιλέχθηκε ο χώρος εργασίας Part Design του FreeCad για την δημιουργία του βασικού τρισδιάστατου μοντέλου. Η θήκη σχεδιάστηκε με ορθογώνιο σχήμα, έτσι ώστε να προσαρμοστεί στις διαστάσεις του raster. Οι διαστάσεις προσαρμόστηκαν με ακρίβεια, χρησιμοποιώντας εργαλεία, όπως το Sketcher για την δημιουργία παραμετρικών σχεδίων. Ακολουθούν 5 εικόνες όπου φαίνονται κατά σειρά, η βασική θήκη στης συσκευής (Εικόνα 3.2.1), ο σχεδιασμός των 4 ειδικών υποδοχών στις οποίες τοποθετούνται οι ταινίες Velcro για την στερέωση της συσκευής στο πόδι, (εικόνα 3.2.2.) , το άνοιγμα για την παροχή τροφοδοσίας στο σύστημα (εικόνα 3.2.3), το άνοιγμα για την εξαγωγή του καλωδίου CP2102 (εικόνα 3.2.4.) και τέλος στην εικόνα 3.2.5 απεικονίζονται οι ακριβείς διαστάσεις της θήκης.



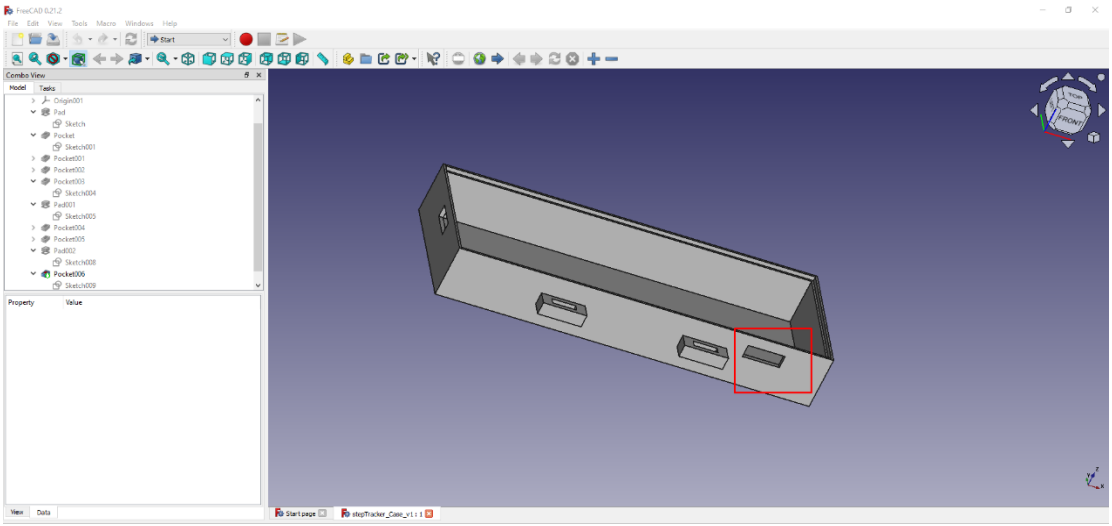
Εικόνα 3.2.1 Βασική θήκη της συσκευής.



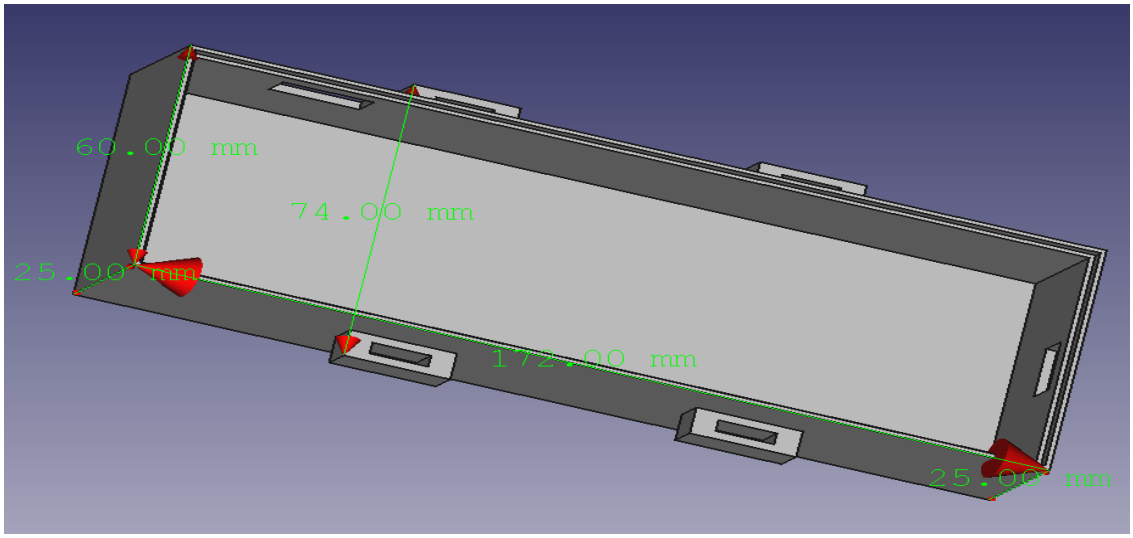
Εικόνα 3.2.2 Βάσεις για την εισαγωγή των Velcro



Εικόνα 3.2.3 Άνοιγμα για την τροφοδοσία της συσκευής.



Εικόνα 3.2.4 Άνοιγμα για την έξοδο του καλωδίου cp2102.

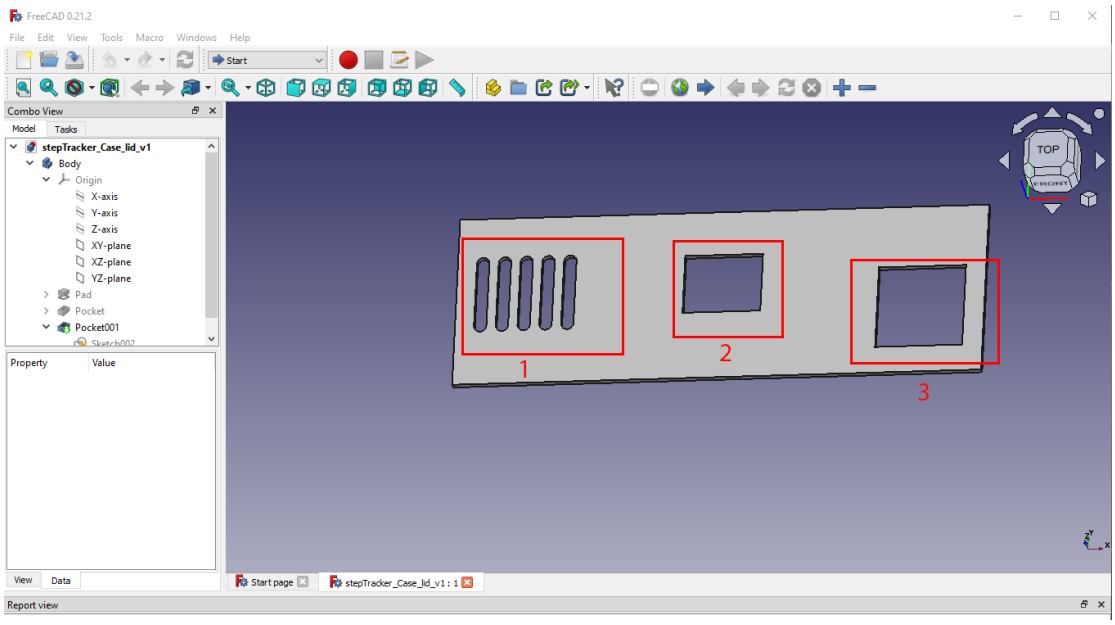


Εικόνα 3.2.5 Διαστάσεις πρώτου μέρους της θήκης.

3.2.3 2ο Στάδιο σχεδιασμού θήκης

Στο δεύτερο στάδιο υλοποίησης της θήκης σχεδιάστηκε το καπάκι στης θήκης. Το καπάκι προσφέρει προστασία και σταθερότητα στα ηλεκτρονικά εξαρτήματα που είναι τοποθετημένα πάνω στο raster κατά την χρήση της συσκευής. Ο σχεδιασμός του καπακιού έγινε με τέτοιο τρόπο, ώστε να έχει την δυνατότητα ο χρήστης να έχει οπτική επαφή με την οθόνη OLED και να μπορεί να αλληλοεπιδρά με το button της συσκευής.

Η διαδικασία σχεδίασης πραγματοποιήθηκε και αυτή στο λογισμικό FreeCad, με σκοπό την τέλεια εφαρμογή του, πάνω στη βάση, που σχεδιάστηκε στο πρώτο μέρος της θήκης. Για την σχεδίαση του καπακιού χρησιμοποιήθηκαν τα εργαλεία του Part Design, όπως έγινε και κατά την σχεδίαση της βάσης της θήκης, όπως επίσης με το εργαλείο Sketcher υλοποιήθηκαν και όλα τα σχέδια πάνω στο καπάκι που απεικονίζονται στην Εικόνα 3.2.6.



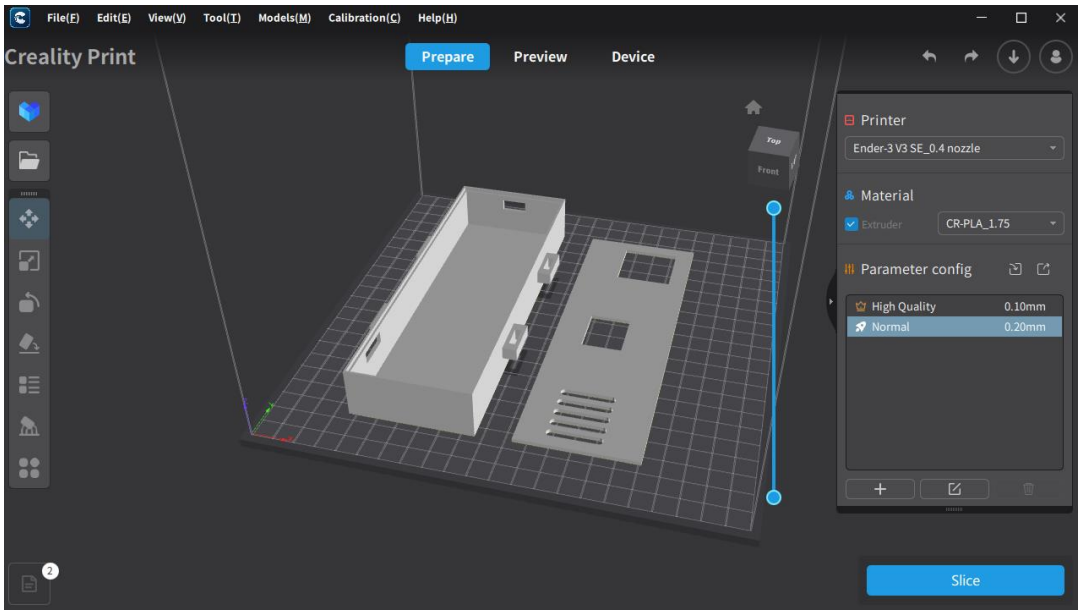
Εικόνα 3.2.6 Το καπάκι της θήκης της συσκευής μέτρησης βημάτων.

Πιο συγκεκριμένα, όπως φαίνεται και στην Εικόνα 3.2.6 απεικονίζονται τρία κόκκινα πλαίσια. Το πρώτο πλαίσιο δείχνει τις γρίλιες που σχεδιάστηκαν στο καπάκι, για να φαίνεται η led λυχνία της αναπτυξιακής πλακέτας Curiosity nano, ώστε να διακρίνεται η κατάσταση λειτουργίας της. Το δεύτερο πλαίσιο δημιουργήθηκε για να μπορεί ο χρήστης να αλληλοεπιδρά με το button της αναπτυξιακής πλακέτας Curiosity nano και τέλος το τρίτο πλαίσιο σχεδιάστηκε για να μπορεί ο χρήστης να έχει οπτική επαφή με την OLED οθόνη.

3.2.4 Τελικό στάδιο εκτύπωσης

Μετά από τα δυο στάδια σχεδίασης, έγινε η εξαγωγή των σχεδίων της θήκης και του καπακιού σε μορφή STL από το λογισμικό FreeCad. Το STL αρχείο περιγράφει ένα τρισδιάστατο αντικείμενο, το οποίο περιέχει μόνο γεωμετρικές πληροφορίες, με την απουσία των δεδομένων χρώματος ή υφής.

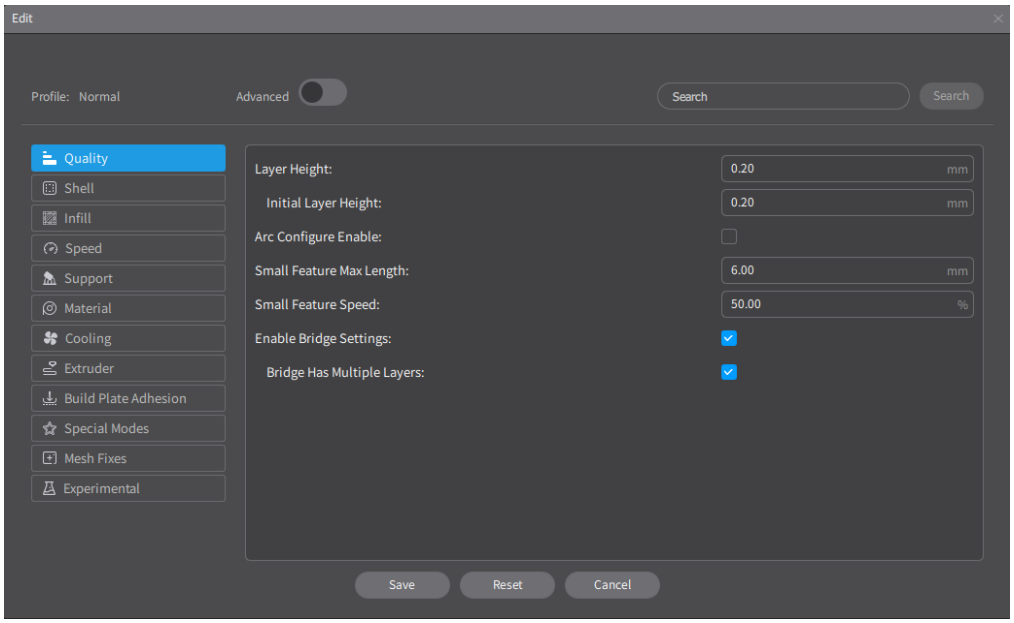
Τα δύο STL αρχεία που δημιουργήθηκαν από το FreeCad μεταφέρθηκαν σε ένα λογισμικό της εταιρίας Creality, η οποία ονομάζεται Creality Print. Το λογισμικό αυτό χρησιμοποιείται για την προετοιμασία τρισδιάστατων μοντέλων σε μορφές, όπως STL,OBJ για εκτύπωση σε 3D εκτυπωτές της Creality. Ο κύριος ρόλος του λογισμικού είναι η μετατροπή του τρισδιάστατου αρχείου STL σε ένα αρχείο G-Code, το οποίο έχει τις οδηγίες εκτύπωσης για τον τρισδιάστατο εκτυπωτή, δηλαδή για το πώς ο εκτυπωτής θα δημιουργήσει το φυσικό αντικείμενο. Στην Εικόνα 3.2.7 απεικονίζεται το λογισμικό Creality Print, στο οποίο έχουν εισαχθεί τα δύο αρχεία STL που δημιουργήθηκαν από το FreeCad, πριν γίνει η διαδικασία Slice.



Εικόνα 3.2.7 Λογισμικό Creality Print με τα αρχεία STL.

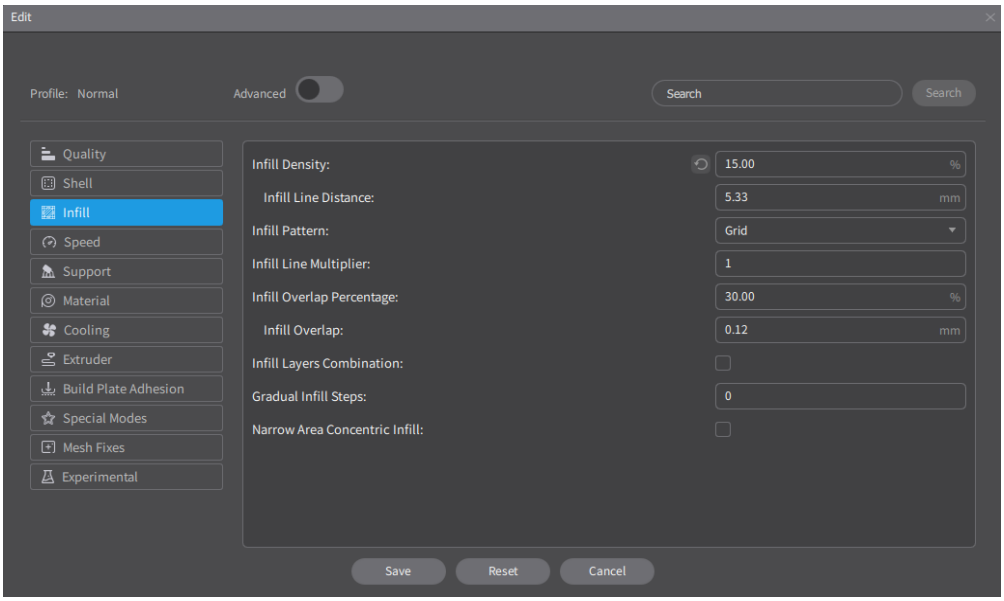
Το λογισμικό της Creality ανάλογα με τον εκτυπωτή που έχεις δηλώσει ότι χρησιμοποιείς, δίνει από μόνο του κάποιες ρυθμίσεις για την εκτύπωση, αλλά είναι εφικτό να γίνουν κάποιες παραμετροποιήσεις. Για την εκτύπωση της θήκης, δόθηκαν οι ρυθμίσεις που απεικονίζονται στις παρακάτω Εικόνες.

Στη Εικόνα 3.2.8 αναδεικνύεται η ρύθμιση Quality, με την οποία μπορούμε να ελέγξουμε το ύψος στρώματος (Layer Height), δηλαδή την ακρίβεια της εκτύπωσης. Στην συγκεκριμένη ρύθμιση χρησιμοποιήθηκαν οι ρυθμίσεις που έθεσε από μόνο του το λογισμικό.



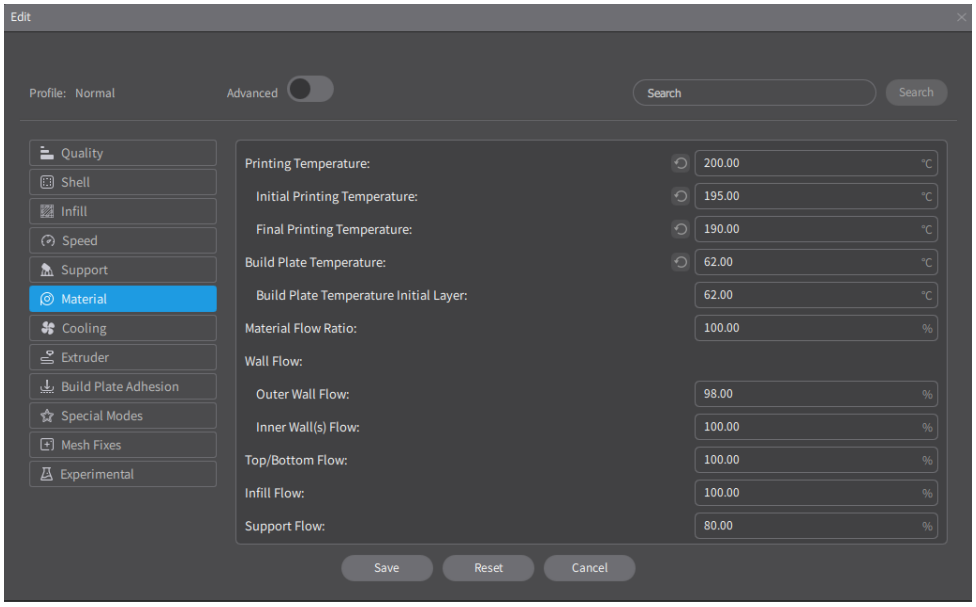
Εικόνα 3.2.8 Ρύθμιση Quality.

Στην Εικόνα 3.2.9 απεικονίζεται η ρύθμιση πυκνότητας (Infill), με την οποία ελέγχουμε το πόσο γεμάτο θα είναι στο εσωτερικό του το αντικείμενο που εκτυπώνεται. Στην περίπτωση της θήκης δεν χρειάστηκε μεγάλη πυκνότητα, καθώς σχεδιάστηκε με τρόπο ώστε η κατασκευή να είναι ελαφριά, οπότε το ποσοστό που επιλέχθηκε ήταν 15%.



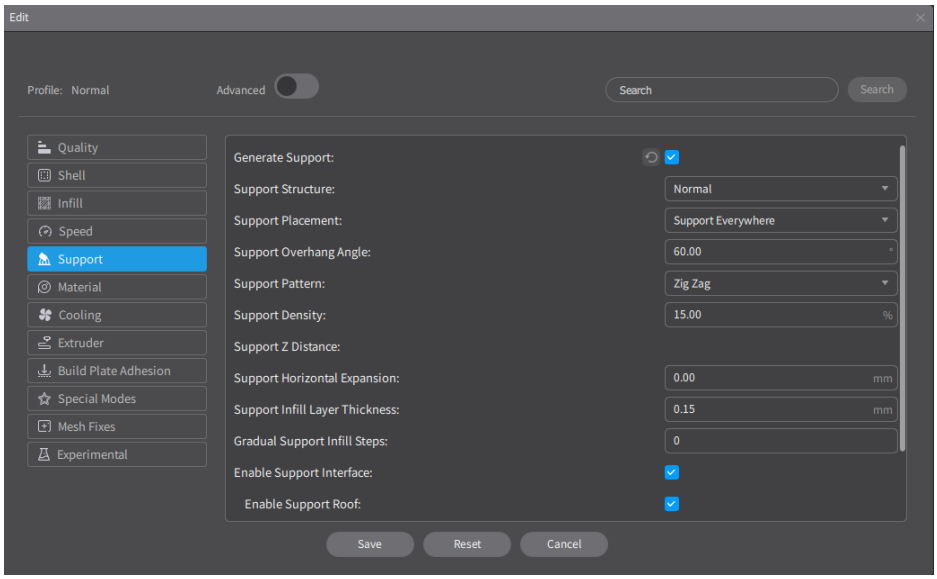
Εικόνα 3.2.9 Ρύθμιση Infill.

Στην Εικόνα 3.2.10 απεικονίζεται η ρύθμιση της θερμοκρασίας που λιώνει το υλικό. Στα πλαίσια της εκτύπωσης χρησιμοποιήθηκε το υλικό PLA, το οποίο λιώνει στους βαθμούς 195°C - 210°C, για αυτό και επιλέχθηκε η θερμοκρασία 200°C, όπου είναι μια ενδιάμεση θερμοκρασία για το υλικό PLA.



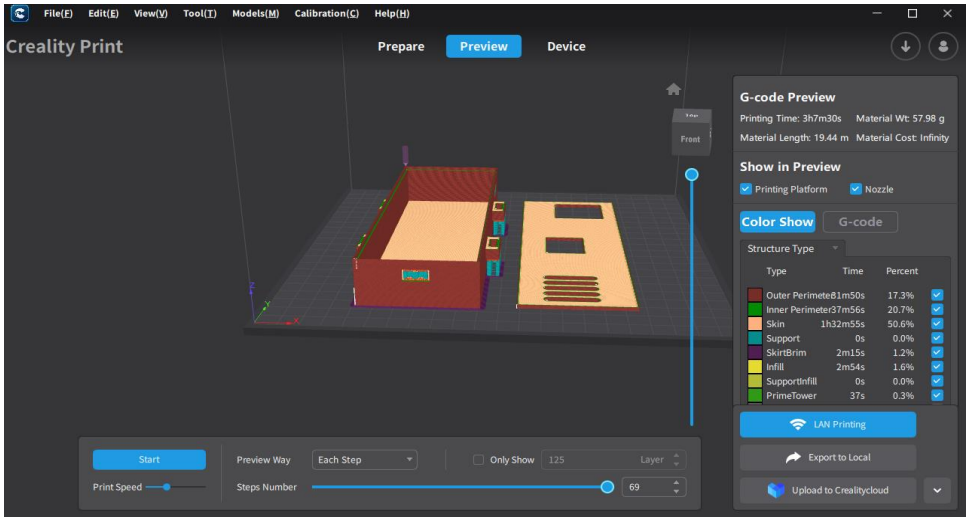
Εικόνα 3.2.10 Ρύθμιση Material.

Στην Εικόνα 3.2.11 που ακολουθεί, εμφανίζεται η ρύθμιση για την υποστήριξη (Support), με την οποία επιλέγεις αν είναι απαραίτητη η χρήση στηριγμάτων. Αξίζει να αναφερθεί ότι υπάρχουν τρία είδη στηριγμάτων. Το Normal, το οποίο σχηματίζει στηρίγματα, όπου κρίνεται απαραίτητο, το slim Tree, το οποίο είναι στήριγμα σε σχήμα δέντρου και τέλος το strong tree, το οποίο αποτελεί μια ισχυρότερη έκδοση του slim tree. Από αυτές τις επιλογές για την εκτύπωση της θήκης επιλέχθηκε το Normal, διότι δημιουργεί καλύτερα στηρίγματα σε οριζόντια πλαίσια.



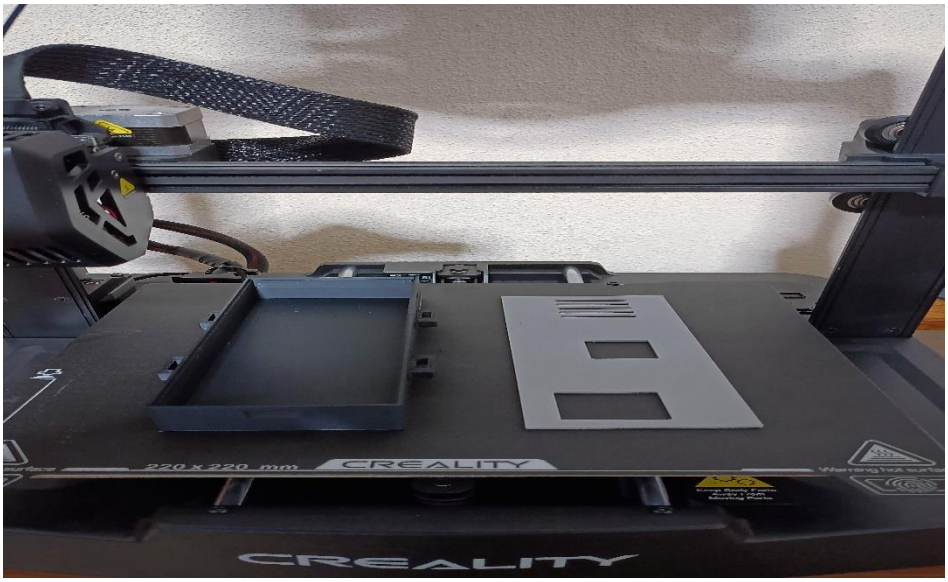
Εικόνα 3.2.11 Ρύθμιση Support

Αφού έγιναν όλες οι απαραίτητες ρυθμίσεις για την 3D εκτύπωση, με το πάτημα του κουμπιού Slice εμφανίζεται το preview του αρχείου G-code, στο οποίο αναδεικνύεται όλη η διαδρομή του ακροφυσίου κατά την διάρκεια της εκτύπωσης. Παράλληλα, στο preview του G-code εμφανίζεται η διάρκεια εκτύπωσης (Printing Time) καθώς και το ποσό του υλικού PLA που χρησιμοποιήθηκε. Στην Εικόνα 3.2.12 παρουσιάζεται το Preview του αρχείου G-code της συγκεκριμένης θήκης, η οποία εκτυπώθηκε σε 3 ώρες, 7 λεπτά και 30 δευτερόλεπτα και χρησιμοποιήθηκαν 57.98g γραμμάρια υλικού PLA.



Εικόνα 3.2.12 Απεικόνιση του αρχείου G-code.

Στην συνέχεια αφού έγινε η εξαγωγή του αρχείου G-code από το λογισμικό Creality Print, χρειάστηκε το αρχείο να μεταφερθεί σε μια ειδική SD κάρτα, η οποία θα συνδέθηκε με τον 3D εκτυπωτή. Η εκτύπωση έγινε στον εκτυπωτή Creality Ender-3 V3 SE και τα αποτελέσματα παρουσιάζονται στην Εικόνα 3.2.13.



Εικόνα 3.2.13 Τελικό αποτέλεσα της θήκης μετά από εκτύπωση σε 3D printer

ΚΕΦΑΛΑΙΟ 4

4. Περιγραφή του Firmware της συσκευής ανίχνευσης βημάτων

Σε αυτό το κεφάλαιο θα αναλυθεί η λογική πίσω από το κύριως πρόγραμμα (Firmware), που εκτελείται στον μικροελεγκτή για την υλοποίηση της συσκευής ανίχνευσης βημάτων.

4.1.1 Η λογική πίσω από το αρχείο OLED.C

Σε αυτό το αρχείο γίνεται η επικοινωνία της OLED οθόνης με το πρωτόκολλο I^2C. Αρχικά, η αναπαράσταση των χαρακτήρων γίνεται με την βοήθεια ενός πίνακα ascii_table_8x16, ο οποίος έχει διαστάσεις 94x16 και αντιστοιχεί στους χαρακτήρες ASCII από το 32 έως το 126, όπως απεικονίζεται και στον Εικόνα 4.1.1. Αυτοί οι χαρακτήρες περιλαμβάνουν όλα τα γράμματα της Αγγλικής αλφαβήτου σε κεφαλαία και πεζά μαζί με όλα τα σύμβολα. Για την απεικόνιση των χαρακτήρων στην οθόνη OLED χρησιμοποιήθηκε γραμματοσειρά 8x16. Κάθε χαρακτήρας αποτελείται από 16 bytes, όπως για παράδειγμα περιγράφεται και στον Πίνακα 4.1 ο χαρακτήρας A στην δεκαεξαδική του μορφή και αντίστοιχα στην δυαδική του μορφή. Αυτά τα 16 bytes στέλνονται με το πρωτόκολλο I^2C από τον μικροελεγκτή στην οθόνη και έτσι η οθόνη ανάβει ή σβήνει τα αντίστοιχα pixels.

Χαρακτήρας A σε δεκαεξαδική μορφή	Χαρακτήρας A σε δυαδική μορφή
`0xE0,0xF0,0x98,0x8C,0x98,0xF0,0xE0,0x00,` `0x0F,0x0F,0x00,0x00,0x00,0x0F,0x0F,0x00`	00001111 11100000 00001111 11110000 00000000 10011000 00000000 10001100 00000000 10011000 00001111 11110000 00001111 11100000 00000000 00000000

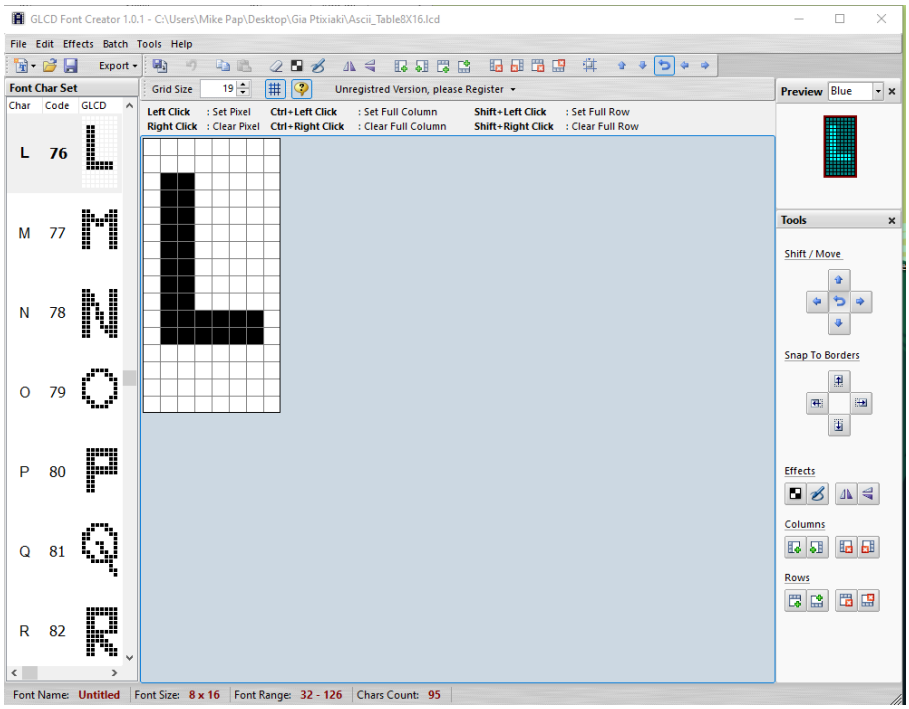
Πίνακας 4.1 Ο χαρακτήρας A σε δεκαεξαδική και σε δυαδική μορφή.

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

www.alpharithms.com

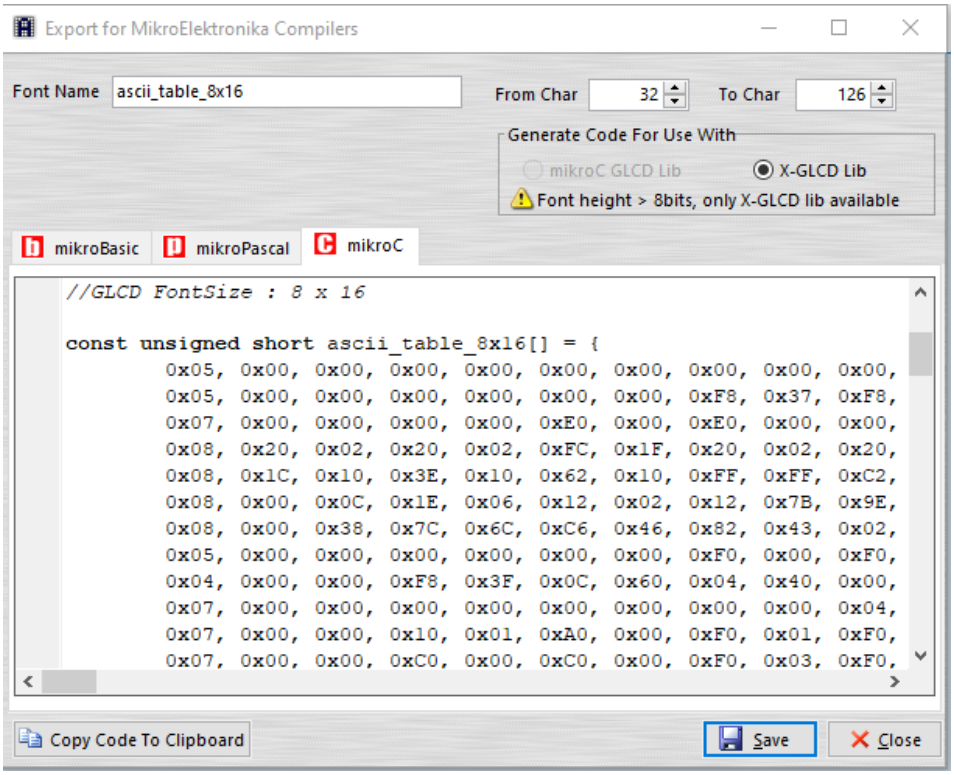
Εικόνα 4.1.1 πίνακας ASCII

Για να παραχθεί ο πίνακας ascii_table_8x16 χρησιμοποιήθηκε η εφαρμογή GLCD Font Creator, η οποία χρησιμοποιείται για σχεδίαση γραμματοσειρών pixel by pixel για την χρήση σε οθόνες που βασίζονται σε bitmap. Στην Εικόνα 4.1.2 παρουσιάζεται η εφαρμογή GLCD Font Creator.



Εικόνα 4.1.2 Η εφαρμογή GLCD Font Creator

Τέλος, η εφαρμογή μπορεί να παράγει κώδικα για διάφορες γλώσσες προγραμματισμού, όπως για παράδειγμα C, Basic και Pascal. Οι εξαγόμενες γραμματοσειρές και bitmap είναι έτοιμες για να ενσωματωθούνε σε οποιοδήποτε firmware για μικροελεγκτή. Στην Εικόνα 4.1.3 παρουσιάζεται ένα παράδειγμα εξαγωγής κώδικα από την εφαρμογή GLCD Font Creator.



Εικόνα 4.1.3 Εξαγωγή αρχείου από GLCD Font Creator.

4.1.2 Ανάλυση συναρτήσεων του αρχείου OLED.C

Ο στόχος της συνάρτησης **Set_Page_Address()** είναι να ρυθμίζει το page Address της οθόνης OLED, υπάρχουν 8 σελίδες κάθε σελίδα αντιπροσωπεύει ένα block 8 pixel στην κάθε διάσταση. Η συνάρτηση αυτή έχει ως είσοδο έναν χαρακτήρα. Αρχικά, η συνάρτηση ενεργοποιεί το πρωτόκολλο I^2C και στην συνέχεια στέλνει την διεύθυνση εγγραφής με την εντολή i2c_write(Write_Address), η οποία υποδηλώνει ότι θα σταλούν δεδομένα ή εντολές στην οθόνη. Στην συνέχεια με την εντολή i2c_write(0x80), δηλώνεται, ότι πρόκειται να σταλούν δεδομένα στην οθόνη και όχι εντολές. Η διεύθυνση 0x80 δηλώνει στον ελεγκτή, ότι θα ακολουθήσει μια εντολή η οποία αφορά τη ρύθμιση της σελίδας μνήμης του πίνακα. Εν συνεχεία με την εντολή add = 0xb0 | add, γίνεται η ρύθμιση της σελίδας. Για παράδειγμα αν δοθεί η τιμή 0x03 η τελική τιμή του add θα είναι add = 0xb3 και έτσι θα οριστεί το 3ο page για εγγραφή δεδομένων.

Η συνάρτηση **Set_Column_Address()** δημιουργήθηκε για να ορίζει την στήλη στην οποία θα γίνει η εγγραφή των δεδομένων. Η Oled SSD1306 διαθέτει 128 στήλες από (0x00 έως 0x7F). Οι στήλες ορίζονται από δύο βήματα, στο πρώτο βήμα ρυθμίζονται τα υψηλότερα bits με την εντολή i2c_write((0x10|add>>4)) και στην συνέχεια τα χαμηλότερα bits με την εντολή i2c_write((0x0f&add)). Αν για παράδειγμα δοθεί η τιμή 0x3A στην συνάρτηση, τότε με την πρώτη εντολή θα γίνει μετατόπιση της διεύθυνσης κατά 4 bits προς τα δεξιά και στην συνέχεια θα γίνει συνδυασμός με το σταθερό πρόθεμα 0x10, το οποίο έχει οριστεί από τον κατασκευαστή της οθόνης για να σηματοδοτεί ότι αφορά το MSB, άρα θα αποσταλθεί η διεύθυνση 0x13. Στην συνέχεια με την δεύτερη εντολή θα κρατηθεί μόνο το LSB δηλαδή το 0x0A και θα αποσταλθεί και αυτό. Έτσι η οθόνη αφού δεχθεί αυτές τις δύο τιμές μπορεί να αντιληφθεί σε ποιο από τα 128 column πρέπει να γίνει η εγγραφή. Στην περίπτωση του παραδείγματος θα η εγγραφή θα γίνει στην στήλη 58.

Στην συνέχεια ακολουθεί η συνάρτηση **Initial_1306()** η οποία δημιουργήθηκε για να ρυθμίσει την Oled οθόνη, ώστε να μπορεί να συνεργαστεί με τον μικροελεγκτή. Η συνάρτηση αυτή περιλαμβάνει μια σειρά από εντολές i2c, οι οποίες διαμορφώνουν κάποιες παραμέτρους όπως για παράδειγμα η φωτεινότητα, ο χρονισμός της οθόνης, το offset της οθόνης και πολλά άλλα. Αυτές οι ρυθμίσεις διασφαλίζουν τη σωστή λειτουργία της οθόνης πριν την εμφάνιση των δεδομένων.

Η συνάρτηση **lcd_clrscroled()** υλοποιήθηκε για να σβήνει ή να γεμίζει την οθόνη ανάλογα με την είσοδο που θα δοθεί, παραδείγματος χάρη αν δοθεί η τιμή 0x00 θα σβήσουν όλα τα led της οθόνης, ενώ αν δοθεί η τιμή 0xFF τότε θα ανάψουν όλα τα led της οθόνης. Αρχικά η συνάρτηση καλεί τις δύο συναρτήσεις, την **Set_Page_Address()** και την **Set_Column_Address()** μέσα σε έναν βρόγχο 8 επαναλήψεων όπου ακολουθείται από έναν εμφωλευμένο βρόχο 128 επαναλήψεων. Αυτό έγινε επειδή η οθόνη χωρίζεται σε 8 pages και καθένα από αυτά περιέχει 128 column και με αυτή την λογική μπορεί η συνάρτηση να σβήσει ή να ανάψει όλα τα led της οθόνης ταυτόχρονα.

Η συνάρτηση **show_font2l()** σχηματίζει έναν χαρακτήρα 8x16 στην οθόνη oled ssd1306. Γνωρίζουμε ότι η κάθε σελίδα (page) της oled οθόνης αντιπροσωπεύει 8 κάθετα pixel, οπότε η συνάρτηση διαχωρίζει τον χαρακτήρα σε δύο τμήματα, στο πρώτο τμήμα γράφει τα πρώτα 8 pixels της τρέχουσας σελίδας (page yp) και στο δεύτερο τμήμα γράφει τα επόμενα 8 pixels τα οποία γράφονται στην επόμενη σελίδα (page yp++). Τα βήματα που ακολουθεί η συνάρτηση είναι τα εξής:

1. Υπολογισμός της οριζόντιας θέσης της στήλης (column) με την εντολή i1= xp*8, στην οποία θα εμφανιστεί ο χαρακτήρας.
2. Στην συνέχεια γίνεται η ρύθμιση της διεύθυνσης της τρέχουσας στήλης και της σελίδας με τις συναρτήσεις **set_page_Address()** και **set_column_Address().**

3. Αφού γίνει ο προσδιορισμός της θέσης του χαρακτήρα, ξεκινάει επικοινωνία i2c και αποστέλλονται τα πρώτα 8 bytes από τον πίνακα ascii_table8x16, δηλαδή τα πρώτα 8 pixel για την δημιουργία του χαρακτήρα.
4. Στην συνέχεια γίνεται η μετακίνηση στην επόμενη σελίδα page(yp++) και αποστέλλονται τα υπόλοιπα 8 bytes του χαρακτήρα.

Αρχικά πριν την κλήση της συνάρτησης **show_string8x16()** για την εμφάνιση μια συμβολοσειράς στην οθόνη oled, καλείται η εντολή sprintf για να εντάξει την συμβολοσειρά που θέλουμε να εμφανίσουμε μέσα σε έναν πίνακα. Η συνάρτηση **show_string8x16()** με την βοήθεια του src_pointer δείκτη εμφανίζει έναν έναν τους χαρακτήρες που αποθηκευτήκαν στον πίνακα. Τέλος οι τιμές b και c όπου δέχεται η συνάρτηση είναι για τον προσδιορισμό που επιθυμούμε να εμφανιστεί η συμβολοσειρά.

4.2.1 Ανάλυση του αρχείου BNO055.C

Το αρχείο αυτό περιέχει τον κώδικα με τον οποίο γίνεται η επικοινωνία του μικροελεγκτή και του αισθητήρα BNO055, ο οποίος παρέχει δεδομένα κίνησης με την βοήθεια συναρτήσεων που θα αναλυθούν παρακάτω. Η επικοινωνία του μικροελεγκτή και του αισθητήρα BNO055 γίνεται με το πρωτόκολλο I^2C.

4.2.2 Συναρτήσεις του αρχείου BNO055.C

Η Συνάρτηση **BNO055_read_register_id()** διαβάζει τα CHIP_ID, ACC_ID, MAG_ID, GYR_ID από τον αισθητήρα BNO055 και τα εμφανίζει στην οθόνη OLED. Το καθένα από αυτά τα ID έχει μια συγκεκριμένη τιμή, όπως αναδεικνύονται στον Πίνακα 4.2.1. Με αυτές τις τιμές επιβεβαιώνουμε την σωστή λειτουργία του αισθητήρα.

ID	Τιμή
CHIP_ID	0xA0
ACC_ID	0xFB
MAG_ID	0x32
GYR_ID	0x0F

Πίνακας 4.2.1 Τιμές από τα ID του αισθητήρα BNO055.

Η συνάρτηση **BNO055_read_linear_accel()**, δημιουργήθηκε για να διαβάζει τα linear accelerometer data, δηλαδή τα δεδομένα γραμμικής επιτάχυνσης από τους τρείς άξονες x,y,z. Αρχικά, ορίζεται η διεύθυνση εκκίνησης (0x28) για τα δεδομένα γραμμικής επιτάχυνσης. Εν συνεχεία διαβάζονται οι χαμηλές τιμές (LSB) και οι υψηλές τιμές (MSB) και για τους τρεις άξονες και με την εντολή `x1 = ((int16)x_msb << 8) | x_lsb`, γίνεται η ενοποίηση των χαμηλών και των υψηλών τιμών για τον άξονα x, όπως αντίστοιχα γίνεται και για τους υπόλοιπους άξονες y και z. Τέλος, γίνεται η εμφάνιση των τιμών στην οθόνη OLED. Αντίστοιχα δημιουργήθηκε και η συνάρτηση **BNO055_read_accel(),** η οποία έχει τις ίδιες λειτουργίες με την **BNO055_read_linear_accel(),** με μόνη διαφορά, ότι διαβάζει τα ακατέργαστα δεδομένα επιτάχυνσης (raw data).

Συνοψίζοντας, ο κώδικας που περιλαμβάνει το αρχείο BNO055.C, θέτει τον BNO055 σε λειτουργία διαμόρφωσης, normal power mode, NDOF mode και στην συνέχεια διαβάζει τα ακατέργαστα δεδομένα επιτάχυνσης και γραμμικής επιτάχυνσης και τα εκτυπώνει στην OLED οθόνη.

4.3.1 Ανάλυση του αρχείου save_data.c

Σε αυτό το αρχείο υπάρχει μια μόνο συνάρτηση, η οποία είναι η **Write_paket().** Ο λόγος δημιουργίας αυτής της συνάρτησης είναι η αποθήκευση των δεδομένων επιτάχυνσης του αισθητήρα BNO055 σε πίνακες στην μνήμη RAM. Η συνάρτηση αυτή είναι void, δηλαδή δεν επιστρέφει κάποια τιμή, αλλά έχει ως είσοδο έναν ακέραιο 16 bit, ο οποίος χρησιμοποιείται ως δείκτης για τον προσδιορισμό της θέσης αποθήκευσης στους πίνακες save_acelx[],save_acely[], save_acelz[],save_acelxD[], save_acelyD[], save_acelzD[].

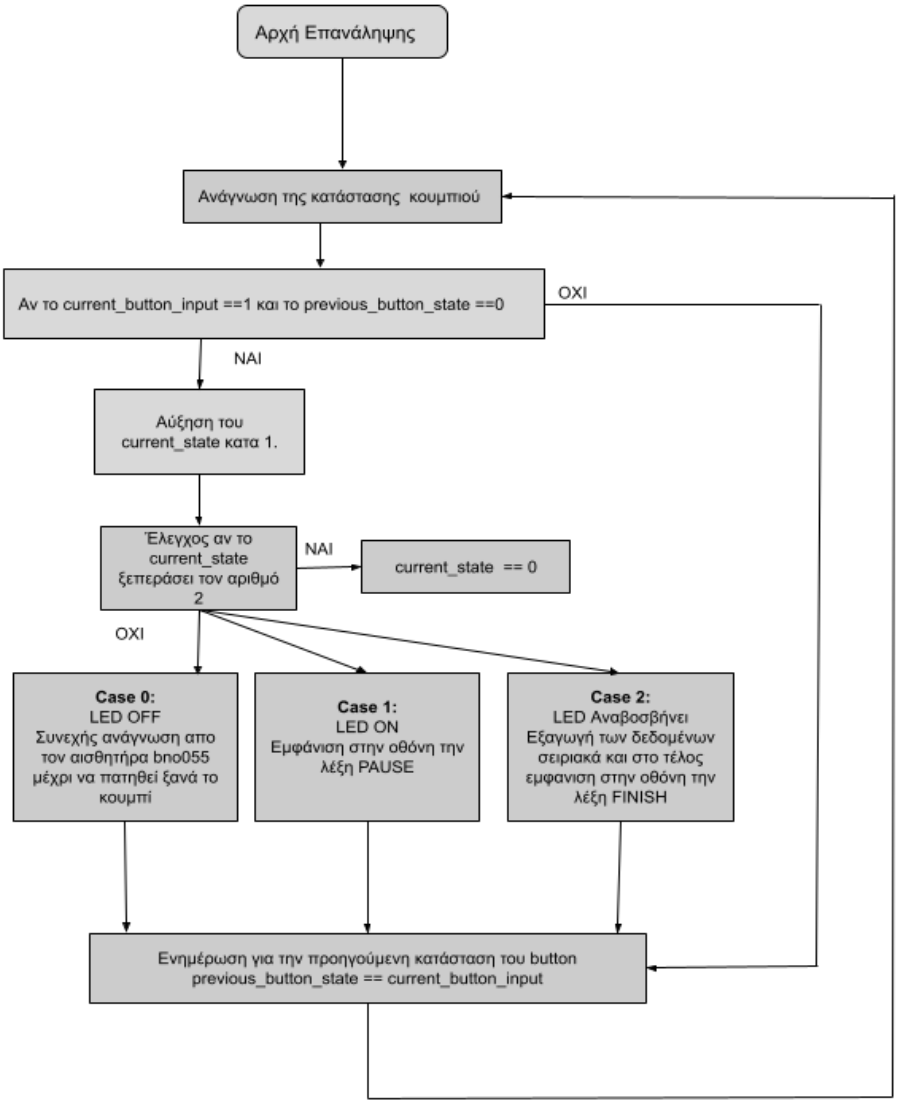
4.4.1 Ανάλυση του αρχείου main.c

Στο αρχείο **main.c** βρίσκεται το βασικό πρόγραμμα λειτουργίας της συσκευής ανίχνευσης βημάτων, το οποίο συντονίζει την επικοινωνία του μικροελεγκτή PIC18F57Q43, του αισθητήρα bno055 και της οθόνης OLED. Περιλαμβάνει την αρχικοποίηση, τη διαχείριση εισόδων και εξόδων καθώς και την υλοποίηση της λογικής λειτουργιών με την χρήση του button και led της αναπτυξιακής πλακέτας Curiosity nano.

Αρχικά, εισάγεται η βιβλιοθήκη **main.h,** στην οποία περιλαμβάνονται κάποιες βασικές ρυθμίσεις για τον μικροελεγκτή. Επιπρόσθετα, υπάρχουν και οι δηλώσεις για την επικοινωνία της οθόνης και του μικροελεγκτή με το πρωτόκολλο i2c καθώς επίσης περιλαμβάνεται και η δήλωση για την σειριακή επικοινωνία με το RS232. Στην συνέχεια ακολουθούν οι βιβλιοθήκες OLED.C, BNO055.C, save_data.c, οι οποίες έχουν τις συναρτήσεις για την oled οθόνη, τον αισθητήρα bno055 και την αποθήκευση των δεδομένων, όπως αναλύθηκαν στα παραπάνω κεφάλαια.

Εν συνεχεία ακολουθεί η συνάρτηση **init_port(),** η οποία περιλαμβάνει εντολές μέσω των οποίων δηλώνονται ποια Pins της αναπτυξιακής πλακέτας θα λειτουργούν σαν έξοδοι ή σαν είσοδοι.

Η επόμενη συνάρτηση είναι η **main,** μέσα από την οποία γίνεται η αρχικοποίηση της οθόνης και του αισθητήρα bno055 με τις εντολές Initial_1306 και BNO055_Config. Στην συνέχεια εκτελείται η συνάρτηση BNO055_read_register_id, η οποία όπως αναφέρθηκε και στο Κεφάλαιο 4.2.1, εμφανίζει κάποια id, τα οποία επαληθεύουν ότι τα δεδομένα που λαμβάνονται από τον αισθητήρα bno055 είναι έγκυρα. Τέλος, στον ατέρμονα βρόχο της main έχει υλοποιηθεί η λογική της συσκευής, όπου βασίζεται στη διαχείριση τριών καταστάσεων, ελεγχόμενων από το πάτημα ενός κουμπιού. Η κατάσταση ορίζεται από την μεταβλητή current_state, η οποία αλλάζει σε κάθε πάτημα του κουμπιού. Η κάθε κατάσταση έχει την δικιά της λειτουργία. Πιο συγκεκριμένα, στην **Κατάσταση 0** η συσκευή αντλεί δεδομένα επιτάχυνσης από τον αισθητήρα bno055 και να της αποθηκεύει στην μνήμη. Στην **Κατάσταση 1** γίνεται παύση από την αποθήκευση δεδομένων και εμφανίζει στην οθόνη την λέξη PAUSE για να ενημερώσει τον χρήστη ότι έχει σταματήσει. Τέλος, στην **Κατάσταση 2** εκτυπώνει τα αποθηκευμένα δεδομένα μέσω σειριακής επικοινωνίας και μόλις τελειώσει εμφανίζει στην oled οθόνη την λέξη FINISH για την ενημέρωση του χρήστη. Στο Διάγραμμα 1 που ακολουθεί, περιγράφεται η λογική του ατέρμονα βρόχου μαζί με τις 3 καταστάσεις σε διάγραμμα ροής, όπως προαναφέρθηκαν.



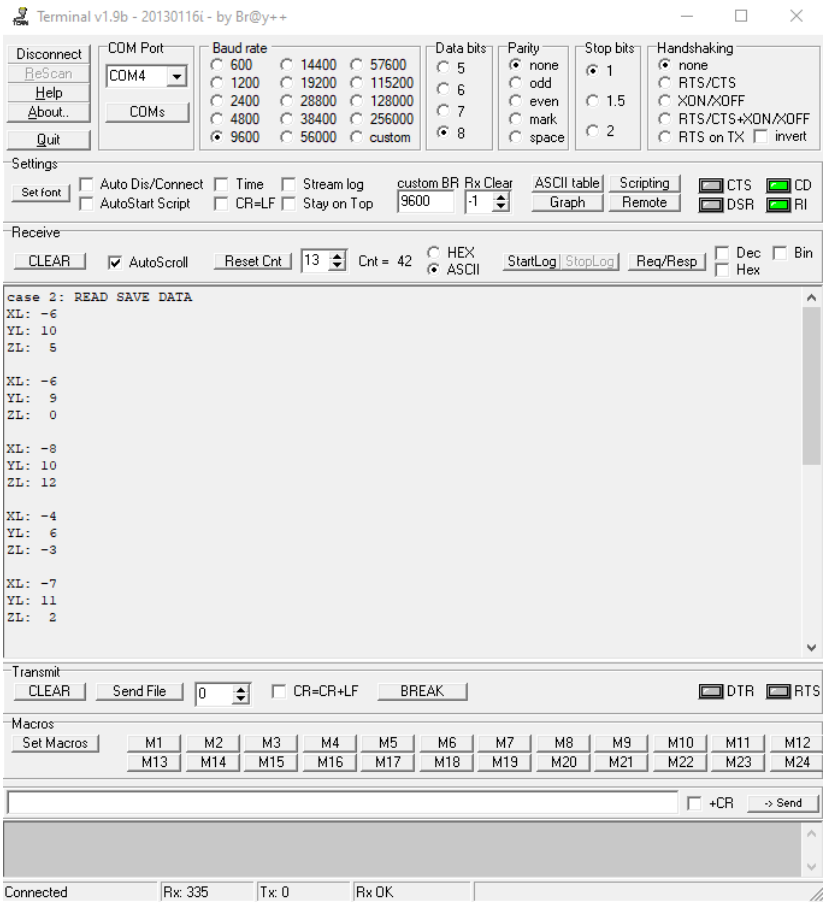
Διάγραμμα 1 Διάγραμμα ροής της while.

ΚΕΦΑΛΑΙΟ 5

5.1 Ανίχνευση Βημάτων μέσω Αλγορίθμου σε Python

Σε αυτό το κεφάλαιο παρουσιάζεται η διαδικασία αναγνώρισης των βημάτων με χρήση λογισμικού, που αναπτύχθηκε σε γλώσσα προγραμματισμού Python. Η επιλογή αυτής της γλώσσας προγραμματισμού έγινε λόγω της ευρείας χρήσης της στον τομέα της ανάλυσης σημάτων και της διαθεσιμότητας εξειδικευμένων βιβλιοθηκών, όπως οι NumPy, SciPy και Matplotlib, που προσφέρουν εργαλεία για αριθμητικούς υπολογισμούς και επεξεργασία σήματος.

Αρχικά, τα δεδομένα που συλλέχθηκαν από τον αισθητήρα bno055 μεταφέρθηκαν μέσω σειριακής επικοινωνίας στον υπολογιστή. Για την διευκόλυνση αυτής της διαδικασίας χρησιμοποιήθηκε η εφαρμογή Terminal v1.9b (Εικόνα 5.1.1). Αφότου ολοκληρώθηκε η διαδικασία της αποστολής των δεδομένων από την συσκευή στον υπολογιστή και συγκεκριμένα στην εφαρμογή Terminal, εξήχθησαν χειροκίνητα και αποθηκεύτηκαν σε αρχείο μορφής .txt, προκειμένου να υποβληθούν σε επεξεργασία από τον αλγόριθμο ανίχνευσης βημάτων.

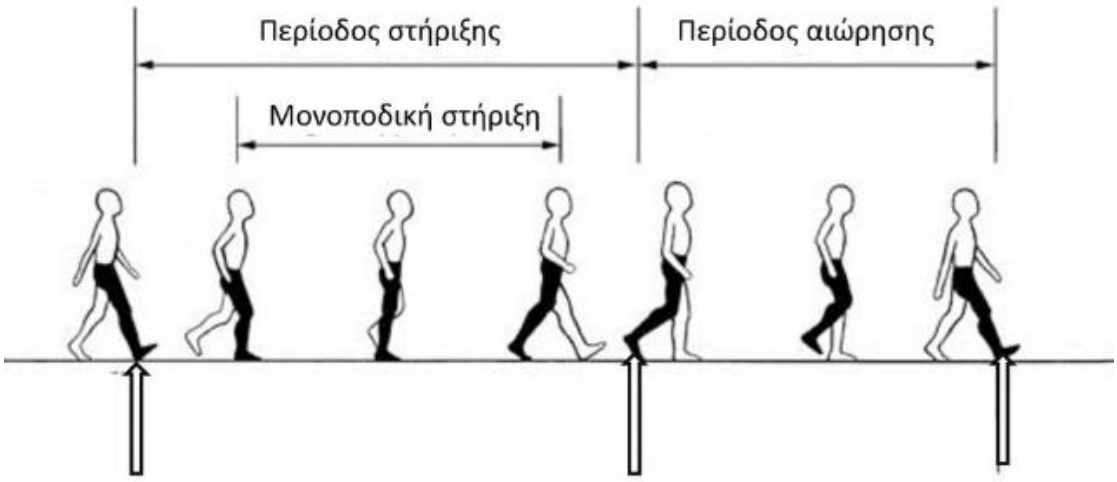


Εικόνα 5.1.1 Ανάγνωση σειριακή θύρας από τον υπολογιστή με την εφαρμογή Terminal v1.9b.

5.1.1 Αλγόριθμος Ανίχνευσης Βημάτων

Η υλοποίηση του αλγορίθμου έγινε με την βοήθεια μιας λογικής ανίχνευσης βημάτων, όπως παρουσιάζεται στο paper[13] των Ahmad Abadleh, Eshraq Al-Hawari, Esra'a Alkafaween, Hamad Al-Sawalqah. Ο αλγόριθμος χρησιμοποιεί δεδομένα γραμμικής επιτάχυνσης με στόχο την ακριβή αναγνώριση των χρονικών στιγμών που αντιστοιχεί σε ανθρώπινα βήματα.

Πριν από την υλοποίηση του αλγορίθμου, ήταν απαραίτητο να πραγματοποιηθεί ανάλυση του κύκλου βάδισης. Στην ανάλυση είναι σημαντικό να ξεχωρίσουμε τα διάφορα γεγονότα από τα οποία αποτελείται ο βηματισμός. Παρακάτω, στη Εικόνα5.1.2, φαίνεται ένας κανονικός κύκλος βηματισμού ενός ανθρώπου.



Εικόνα 5.1.2 Κύκλος βηματισμού ενός ανθρώπου Πηγή:(wsiat.on.ca).

Ο κύκλος βηματισμού ενός ανθρώπου χωρίζεται σε δύο περιόδους. Στην περίοδο στήριξης και στην περίοδο αιώρησης. Η περίοδος στήριξης αντιπροσωπεύει περίπου το 60% του συνολικού κύκλου και χαρακτηρίζεται από την στιγμή κατά την οποία το πέλμα του ποδιού παραμένει σε επαφή με το έδαφος, παρέχοντας στήριξη στο σώμα. Το υπόλοιπο 40% του κύκλου αφορά την περίοδο αιώρησης, κατά την οποία το πόδι δεν έρχεται σε επαφή με το έδαφος και κινείται προς τα εμπρός για να προετοιμάσει το επόμενο βήμα.

Οι δύο φάσεις αυτές αντικατοπτρίζονται στα δεδομένα του αισθητήρα με τη μορφή χαρακτηριστικών κορυφών (peaks), οι οποίες εντοπίζονται με κατάλληλη επεξεργασία του σήματος, όπως θα παρουσιαστεί στις επόμενες ενότητες.

5.1.2 Αναγνώριση και προ επεξεργασία δεδομένων

Αρχικά ο αλγόριθμος ανίχνευσης βημάτων ξεκινά με την ανάγνωση των δεδομένων γραμμικής επιτάχυνσης από ένα αρχείο μορφής .txt. Όπως προαναφέρθηκε παραπάνω, το αρχείο αυτό δημιουργήθηκε μέσω σειριακής επικοινωνίας της συσκευής και του υπολογιστή. Κάθε γραμμή του αρχείου περιέχει τιμές για τους τρείς άξονες το XL,YL,ZL και έναν μετρητή Counter_Data, ο οποίος προσδιορίζει τον αριθμό συνολικών δειγμάτων.

Για την αποτελεσματική ανίχνευση των βημάτων, κρίθηκε χρήσιμος ο υπολογισμός του Net Magnitude xy, ενός δείκτη που βασίζεται στις τιμές της γραμμικής επιτάχυνσης στους άξονες X και Y. Οι άξονες αυτοί επιλέχθηκαν, διότι αποτυπώνουν κυρίως την οριζόντια κίνηση του ποδιού κατά τη διάρκεια του βαδίσματος, ενώ ο άξονας Z επηρεάζεται συχνά από κραδασμούς ή τυχαίες κατακόρυφες μεταπτώσεις, που μπορεί να εισάγει θόρυβο στα δεδομένα. Αρχικά, για κάθε χρονική στιγμή i, υπολογίζεται το μέτρο της επιτάχυνσης στο οριζόντιο επίπεδο (XY) Εικόνα5.1.3 με τον τύπο (1).

(1) $$MagnitudeXY = \sqrt{X^2 + Y^2}$$

Στη συνέχεια, υπολογίζεται ο μέσος όρος όλων των μεγεθών XY.

(2) $$MeanXY = \frac{1}{N}\sum_{i=1}^{N} Magnitude\ XY, i$$

Με βάση το μέσο όρο, δημιουργείται ένα νέο σήμα που ονομάζεται Net magnitude XY, το οποίο εκφράζει την ενέργεια ή ένταση της στιγμιαίας επιτάχυνσης σε σχέση με την μέση κινητικότητα του χρήστη. Αυτό επιτυγχάνεται με τον παρακάτω τύπο.

(3) $$NetMagntudeXY = \frac{(MagntudeXY - MeanXY)^2}{N}$$

Ο λόγος που χρησιμοποιείται τετραγωνισμένη διαφορά, είναι για να δοθεί μεγαλύτερη έμφαση σε αιχμές του σήματος, δηλαδή σε στιγμές, όπου η επιτάχυνση αποκλίνει σημαντικά από τη μέση τιμή, φαινόμενο που συμβαίνει συνήθως κατά την πρόσκρουση της φτέρνας στο έδαφος, το οποίο αποτελεί και ένδειξη βήματος.

Η κανονικοποίηση ως προς το πλήθος των δειγμάτων N βοηθά στο να μην επηρεάζεται το τελικό μέγεθος του σήματος από το μήκος των δεδομένων. Το σήμα NetMagnitudeXY παρουσιάζει, σαφώς οριοθετημένες κορυφές, οι οποίες χρησιμοποιούνται στην συνέχεια για την αναγνώριση και καταμέτρηση των βημάτων, με την βοήθεια του αλγορίθμου find_peaks της βιβλιοθήκης SciPy.

Εικόνα 5.1.3

5.1.3 Φιλτράρισμα Κορυφών

Αφού γίνει ο εντοπισμός των κορυφών από το σήμα Net Magnitude των X και Y, παρατηρούνται κάποιες χαρακτηριστικές κορυφές που σχετίζονται με την βάδιση του

ανθρώπου. Πιο συγκεκριμένα, κάθε φυσικό βήμα συνοδεύεται από δύο διαδοχικές κορυφές. Η πρώτη κορυφή εμφανίζεται τη στιγμή που το πόδι σηκώνεται από το έδαφος, δηλαδή κατά την έναρξη της περιόδου αιώρησης. Η δεύτερη κορυφή εμφανίζεται, όταν το πόδι επανατοποθετείτε στο έδαφος, δηλαδή κατά τη έναρξη της περιόδου στήριξης. Αν και οι δύο κορυφές αντιστοιχούν στο ίδιο βήμα , η παρουσία τους στο σήμα προκαλεί την εμφάνιση διπλών αιχμών για κάθε βήμα. Επιπλέον, είναι πιθανό να εμφανιστούν ψευδείς κορυφές λόγω θορύβου ή μικροκινήσεων της συσκευής πάνω στο πόδι.

Για την αντιμετώπιση αυτών των προβλημάτων και για την ορθή αναγνώριση του πλήθους των βημάτων, υλοποιήθηκε μια συνάρτηση, η οποία ονομάζεται filter_close_peaks, η οποία εξαλείφει τις ψευδείς ή διπλές κορυφές που εμφανίζονται σε μικρές χρονικές αποστάσεις μεταξύ τους. Συγκεκριμένα, εμφανίζεται ένα κατώφλι ελάχιστης χρονικής απόστασης ίσο με 700 milliseconds (ms), το οποίο προσδιορίστηκε ύστερα από πειραματική παραμετροποίηση, Η τιμή αυτή προέκυψε ως κατάλληλη, καθώς ανταποκρίνεται τόσο στη συχνότητα της φυσιολογικής βάδισης όσο και στις αυξημένες ταχύτητες που εμφανίζονται κατά το τρέξιμο, εξασφαλίζοντας έτσι ισορροπία μεταξύ ακρίβειας και ευαισθησίας στον εντοπισμό βημάτων.

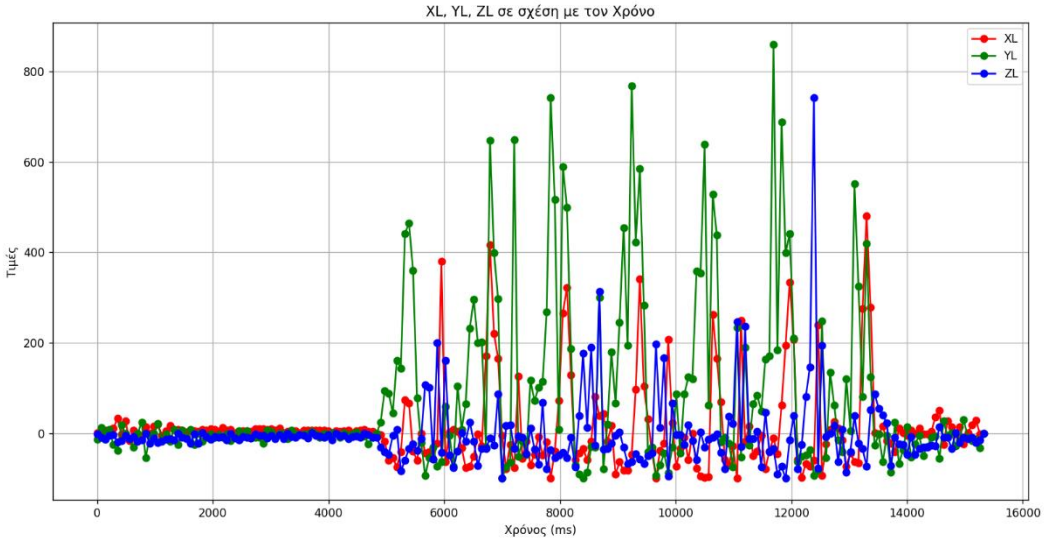
Στη συνέχεια, αναφέρεται ο τρόπος με τον οποίο λειτουργεί η συνάρτηση αυτή. Αρχικά, εξετάζεται κάθε κορυφή στην λίστα peak_time με την αντίστοιχη τιμή peak_values. Στη συνέχεια, για κάθε μια από τις κορυφές, ελέγχεται αν στις επόμενες χρονικές τιμές οι οποίες βρίσκονται στο διάστημα των 700ms υπάρχει άλλη κορυφή. Αν βρεθεί κοντινή κορυφή με μεγαλύτερη τιμή, τότε αυτή διατηρείται και απαλείφει τις υπόλοιπες, θεωρώντας ότι είναι η πιο αντιπροσωπευτική του βήματος. Αφού τελειώσει η αναζήτηση μέσα στο παράθυρο των 700ms, η πιο έντονη κορυφή αντιπροσωπεύει το βήμα. Αυτή η προσέγγιση εξασφαλίζει, ότι για κάθε φυσικό βήμα απομονώνεται μόνο με μία κορυφή, βελτιώνοντας σημαντικά την ακρίβεια της ανίχνευσης και περιορίζοντας τον αριθμό των ψευδών βημάτων. Η τελική φιλτραρισμένη λίστα των κορυφών χρησιμοποιείται για την καταμέτρηση των βημάτων.

5.2.1 Αποτελέσματα Ανίχνευσης Βημάτων

Στην Εικόνα 5.2.1 παρουσιάζονται τα ακατέργαστα δεδομένα ενός πειράματος βάδισης με την συσκευή ανίχνευσης βημάτων, στην οποία αποτυπώνονται 6 βήματα. Στον οριζόντιο άξονα αποτυπώνεται ο χρόνος σε χιλιοστά του δευτερολέπτου(ms), ενώ στον κατακόρυφο άξονα εμφανίζονται οι τιμές επιτάχυνσης στους τρεις άξονες (x,y,z), όπως προκύπτουν από το επιταχυνσιόμετρο bno055. Στην Εικόνα 5.2.1, παρατηρείται ότι στο χρονικό περιθώριο 0ms-5000ms δεν υπάρχουν μεταβολές στο διάγραμμα και οι τιμές επιτάχυνσης είναι μηδενικές. Αυτό οφείλεται στ' ότι σε αυτά τα χιλιοστά του δευτερολέπτου ο χρήστης δεν εκτελεί κάποιο βήμα και μένει σταθερός στο ίδιο σημείο. Μετά από αυτό το χρονικό σημείο, ο χρήστης ξεκινάει την βάδιση των 6 βημάτων και έτσι καταγράφονται και αποτυπώνονται με τιμές επιτάχυνσης τα βήματα. Αξίζει εδώ

να σημειωθεί ότι, από το συγκεκριμένο γράφημα δεν είναι εύκολο να εντοπιστεί με ακρίβεια ο βηματισμός, διότι υπάρχει θόρυβος στα δεδομένα, ο οποίος μπορεί να προκαλείται από πολλαπλές μικροκινήσεις της συσκευής κατά την ώρα της βάδισης.

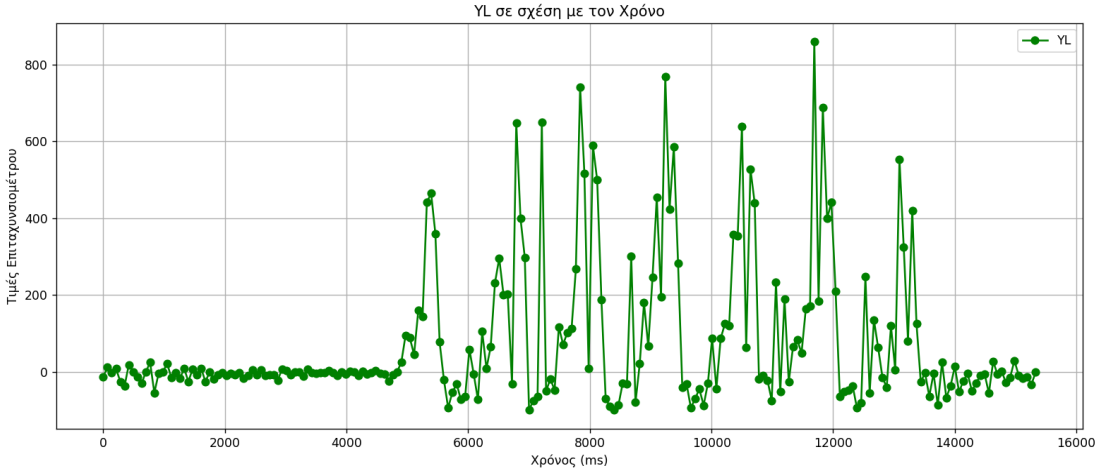
Για την επίλυση αυτού του προβλήματος, υπολογίστηκε το μέτρο της συνολικής επιτάχυνσης γνωστό και ως net magnitude για τους άξονες x,y, εξαιρώντας τον άξονα z, διότι δεν χρειαζόταν για την ανίχνευση των βημάτων και πολλές φορές προκαλούσε ψευδή στοιχεία, τα οποία ερμηνευόντουσαν ως βήματα.



Εικόνα 5.2.1 Αρχικό διάγραμμα των τιμών επιτάχυνσης στους άξονες XL,YL,ZL.

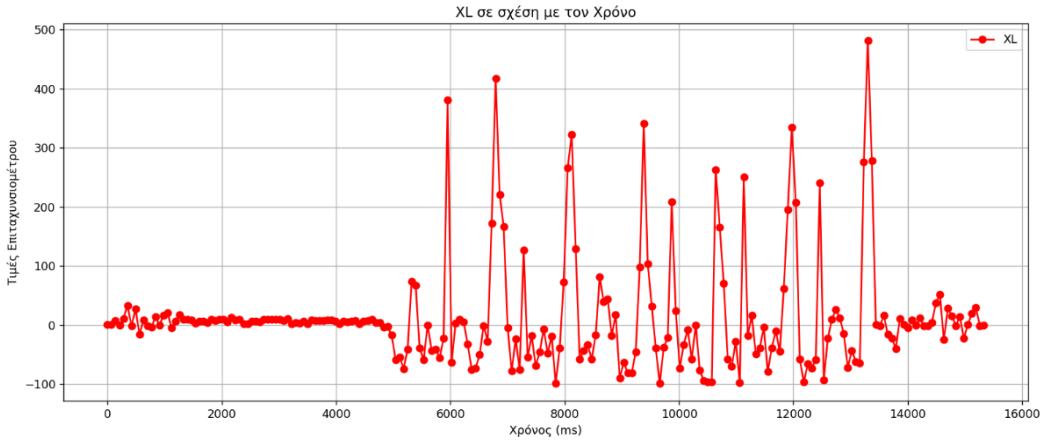
Επιπρόσθετα, στην Εικόνα 5.2.2 και Εικόνα 5.2.3, παρουσιάζονται ξεχωριστά τα σήματα από τους άξονες x,y του επιταχυνσιόμετρου.

Στην Εικόνα 5.2.2, απεικονίζεται το διάγραμμα των επιταχύνσεων κατά τον άξονα Y, ο οποίος συσχετίζεται με την κατεύθυνση της κίνησης του ποδιού προς τα εμπρός. Παρατηρούμε, ότι μετά τα 6000ms εμφανίζονται διαδοχικές, έντονες κορυφές με σχεδόν περιοδική μορφή. Επίσης, σε κάθε περίοδο παρατηρείται μια μεταβατική αύξηση της επιτάχυνσης. Αυτές οι κορυφές αντιπροσωπεύουν την κίνηση του ποδιού προς τα εμπρός σε κάθε βήμα.



Εικόνα 5.2.2 Διάγραμμα τιμών γραμμικής επιτάχυνσης του άξονα Y.

Στην Εικόνα 5.2.3, απεικονίζεται το διάγραμμα επιταχύνσεων του άξονα x, ο οποίος καταγράφει την επιτάχυνση του ποδιού κατά την άρση και κατά την πτώση του ποδιού. Στο συγκεκριμένο διάγραμμα είναι δύσκολο να απομονωθούν τα βήματα και αυτό οφείλεται στο ότι το σήμα παρουσιάζει μεγαλύτερο θόρυβο και ασυνεπείς κορυφές. Αυτό συμβαίνει, διότι η ανύψωση του ποδιού έχει πιο πολύπλοκη και λιγότερο σταθερή κίνηση σε σύγκριση με την προς τα εμπρός κίνηση.

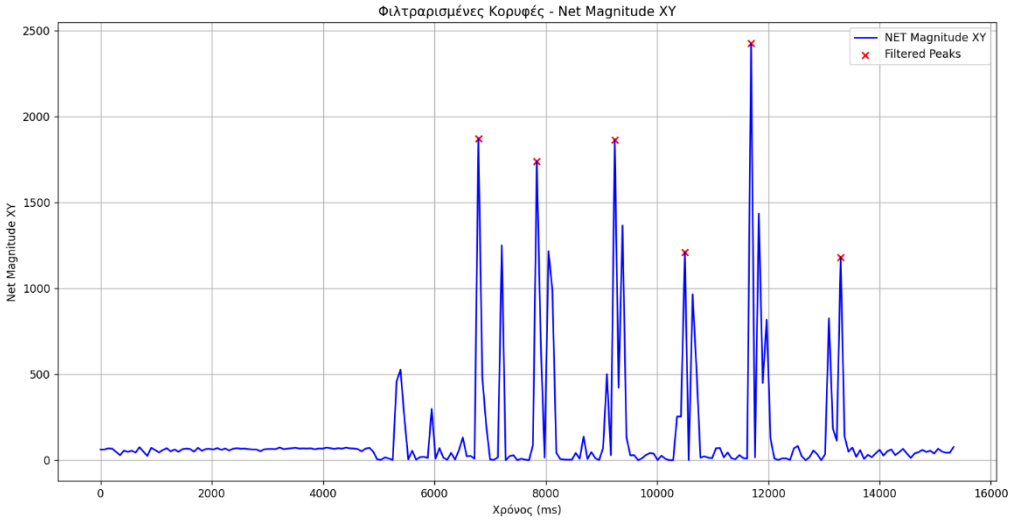


Εικόνα 5.2.3 Διάγραμμα τιμών γραμμικής επιτάχυνσης του άξονα X.

Τέλος, στην Εικόνα 5.2.4 παρουσιάζεται το τελικό αποτέλεσμα της επεξεργασίας του σήματος, όπου έχουν ανιχνευθεί τα σήματα κορυφής, τα οποία αντιστοιχούν σε βήματα. Το σήμα που απεικονίζεται στην Εικόνα 5.2.4, είναι το καθαρό σήμα επιτάχυνσης(net magnitude), στο οποίο έχει εφαρμοστεί φίλτρο χρονικού παραθύρου 700ms, ώστε να αποφεύγονται ψευδείς κορυφές, οι οποίες μπορούν να διαβαστούν ως βήματα.

Οι κορυφές που πληρούν τα κριτήρια του φίλτρου καταγράφονται ως ένα έγκυρο βήμα. Βάση των πειραμάτων που έγιναν, παρατηρήθηκε, ότι το πλήθος των κορυφών που εντοπίστηκαν είναι ακριβώς ίσα με των πραγματικών βημάτων. Αυτό αποδεικνύει και

την ακρίβεια του αλγορίθμου. Επίσης παρατηρείται, ότι το σήμα έχει περιορισμένο θόρυβο και οι κορυφές εμφανίζονται σε σταθερά χρονικά διαστήματα, αποτυπώνοντας τον ρυθμό της βάδισης. Αυτό αποδεικνύει ότι υπολογισμός του Net magnitude σε συνδυασμό με το κατάλληλο φιλτράρισμα οδηγεί σε αξιόπιστα αποτελέσματα.



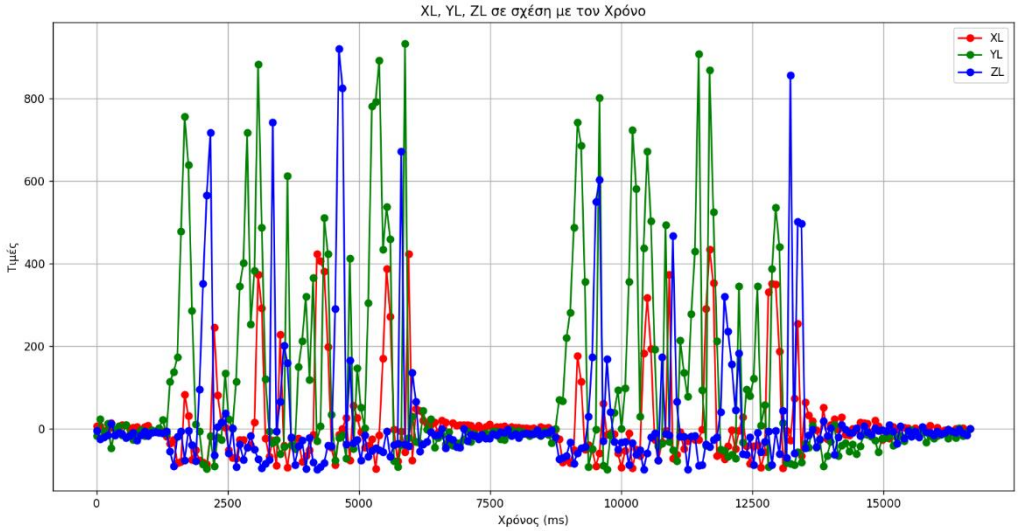
Εικόνα 5.2.4 Φιλτραρισμένο σήμα του Net magnitude και απεικόνιση των βημάτων.

Στην Εικόνα 5.2.5 απεικονίζονται τα αποτελέσματα του λογισμικού, δηλαδή το πλήθος των βημάτων και η θέση της κάθε κορυφής, η οποία εκφράζει ένα βήμα.

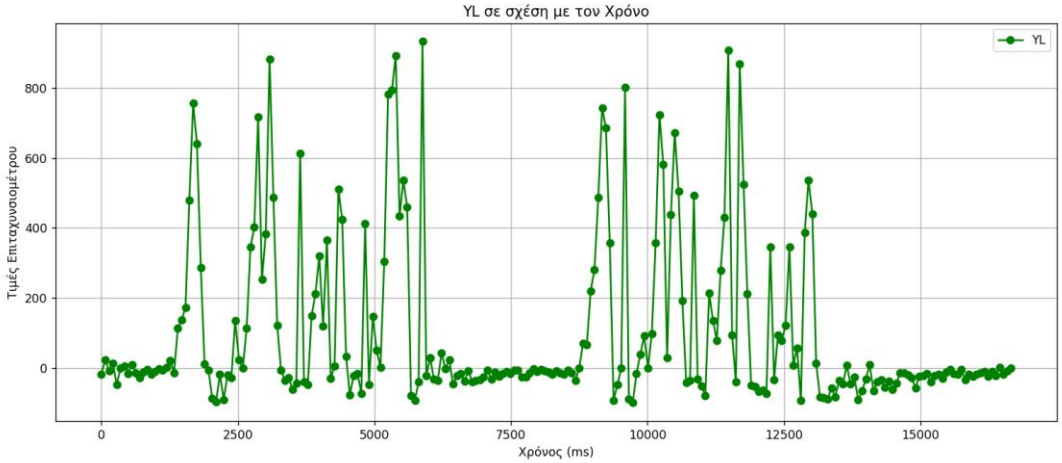
```
Number Peaks(Time, Net Magnitude XY):
[(np.int64(6790), np.float64(1873.0980765857814))]
[(np.int64(7840), np.float64(1741.0096982299713))]
[(np.int64(9240), np.float64(1866.066590127837))]
[(np.int64(10500), np.float64(1212.1420651597666))]
[(np.int64(11690), np.float64(2426.3633378093173))]
[(np.int64(13300), np.float64(1180.4965072172176))]
Number of steps: 6
```

Εικόνα 5.2.5 Αποτελέσματα Λογισμικού.

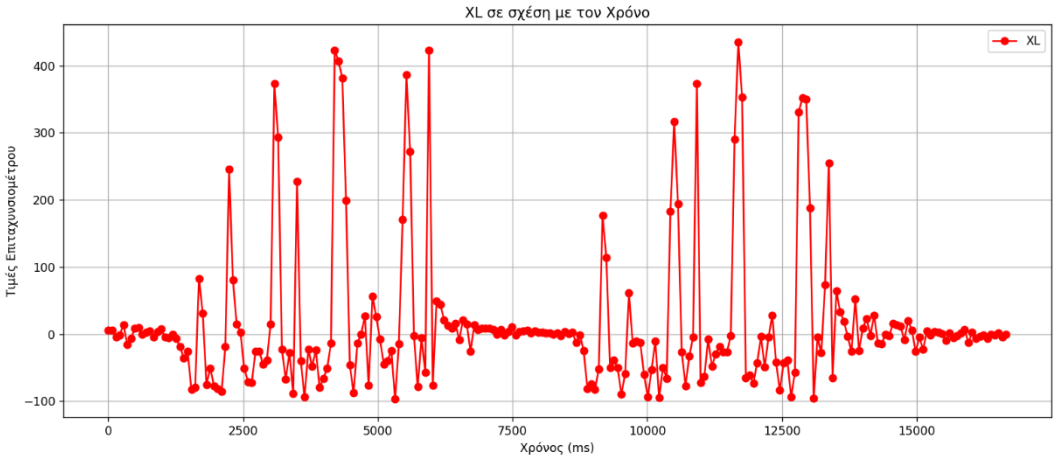
Για την διερεύνηση της αξιοπιστίας του συστήματος σε διαφορετικές συνθήκες, πραγματοποιήθηκαν και άλλα πειράματα, κατά τα οποία ο χρήστης περπάτησε με διαφορετικό ρυθμό. Στα επόμενα διαγράμματα απεικονίζονται τα δεδομένα του πειράματος. Παρότι ο ρυθμός βάδισης άλλαξε, παρατηρείται, ότι αλγόριθμος κατάφερε να ανίχνευσε με ακρίβεια τις κορυφές οι οποίες αντιστοιχούν σε βήματα.



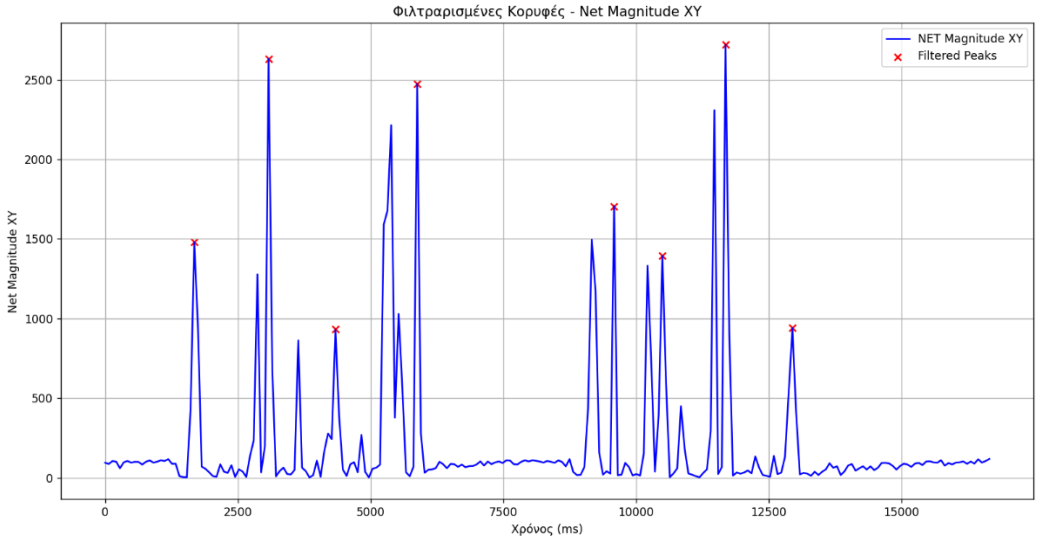
Εικόνα 5.2.6 Αρχικό διάγραμμα των τιμών επιτάχυνσης στους άξονες XL,YL,ZL (Πείραμα 2).



Εικόνα 5.2.7 Διάγραμμα τιμών γραμμικής επιτάχυνσης του άξονα Y (Πείραμα 2).



Εικόνα 5.2.8 Διάγραμμα τιμών γραμμικής επιτάχυνσης του άξονα X (πείραμα 2).



Εικόνα 5.2.9 Φιλτραρισμένο σήμα του Net magnitude και απεικόνιση των βημάτων (Πείραμα 2).

```
Number Peaks(Time, Net Magnitude XY):
[(np.int64(1680), np.float64(1479.5385901310901))]
[(np.int64(3080), np.float64(2631.6574426570105))]
[(np.int64(4340), np.float64(931.5660266715801))]
[(np.int64(5880), np.float64(2475.687697344664))]
[(np.int64(9590), np.float64(1705.073873927528))]
[(np.int64(10500), np.float64(1393.431975545138))]
[(np.int64(11690), np.float64(2720.4828254998706))]
[(np.int64(12950), np.float64(940.1018856565556))]
Number of steps: 8
```

Εικόνα 5.2.10 Αποτελέσματα Λογισμικού (Πείραμα 2).

ΚΕΦΑΛΑΙΟ 6

Συμπεράσματα

Ο σκοπός της παρούσας διπλωματικής εργασίας ήταν ο σχεδιασμός και η υλοποίηση μιας φορέσιμης συσκευής ανίχνευσης βημάτων, τοποθετώντας την στο πόδι του χρήστη και με στόχο την βελτιωμένη καταγραφή φυσικής δραστηριότητας. Η συσκευή βασίζεται στον μικροελεγκτή PIC18F57Q43, σε συνδυασμό με την αισθητήρα BNO055 και την οθόνη OLED SSD 1306, ενώ για την αναγνώριση βημάτων εφαρμόστηκε ένας αλγόριθμος ανίχνευσης κορυφών βασισμένος στον υπολογισμό του net magnitude σε γλώσσα python.

Τα πειραματικά αποτελέσματα έδειξαν ότι η προσέγγιση ήταν επιτυχείς. Ο αλγόριθμος κατάφερε να αναγνωρίσει με ακρίβεια το σύνολο των βημάτων, χωρίς την προσθήκη ψευδών αποτελεσμάτων. Η χρήση του net magnitude και του φίλτρου χρονικού παραθύρου αποδείχθηκε αποτελεσματική για την αποφυγή θορύβου και σφαλμάτων.

Η κατασκευή της συσκευής, τόσο σε επίπεδο κυκλώματος όσο και μηχανικά (3D εκτύπωση), αποδείχθηκε πλήρως λειτουργική και η φορεσιμότητα στο πόδι ενίσχυσε επίσης την αξιοπιστία της μέτρησης.

Ωστόσο η συσκευή παρότι πέτυχε τον στόχο της, υπήρξαν ορισμένοι περιορισμοί, όπως η ανάγκη χειροκίνητης μεταφοράς των δεδομένων μέσω usb και η έλλειψη απεικόνισης των αποτελεσμάτων σε πραγματικό χρόνο στο κινητό ή στο cloud.

Μελλοντικά, η συσκευή θα μπορούσε να εξελιχθεί με:

- Ενσωμάτωση τεχνολογίας Bluetooth για ασύρματη μεταφορά δεδομένων.
- Ανάπτυξη Mobile εφαρμογής για παρακολούθηση σε πραγματικό χρόνο.
- Ενσωμάτωση αλγορίθμων μηχανικής μάθησης για αναγνώριση πιο σύνθετων μοτίβων κίνησης.
- Σμίκρυνση της συσκευής για την βελτίωση της σε θέμα βάρους και ευχρηστίας για τον χρήστη.

Συνολικά η εργασία απέδειξε ότι είναι εφικτό να υλοποιηθεί μία απλή και ακριβής φορέσιμη συσκευή βηματομέτρησης με αξιοποιήσιμα αποτελέσματα και σημαντικές προοπτικές εξέλιξης.

ΒΙΒΛΙΟΓΡΑΦΙΑ

[1] Wikipedia Microcontroller https://en.wikipedia.org/wiki/Microcontroller/

[2] ΠΟΓΑΡΙΔΗΣ Δ. (2020) Ενσωματωμένα Συστήματα: Οι Μικροελεγκτές AVR και ARDUINO 3η Έκδοση ,Εκδόσεις ΔiΣΙΓΜΑ.

[3] PIC18F27/47/57Q43 Data Sheet https://ww1.microchip.com/downloads/en/DeviceDoc/PIC18F27-47-57Q43 Data-Sheet-DS40002147E.pdf

[4] PIC18F57Q43 Curiosity Nano Schematics https://www.microchip.com/en us/development-tool/DM164150

[5]https://www.ti.com/lit/an/sbaa565/sbaa565.pdf?ts=1730521398698&ref_url=https %253A%252F%252Fwww.google.com%252F

[6] https://adafru.it/f0H

[7] https://cdn-learn.adafruit.com/downloads/pdf/adafruit-bno055-absolute-orientation-sensor.pdf

[8] https://www.datasheethub.com/ssd1306-128x64-mono-0-96-inch-i2c-oled-display/

[9] https://www.pololu.com/docs/0j7/all

[10] https://link.springer.com/chapter/10.1007/978-3-540-70994-7_14

[11] https://github.com/M-J-Bauer/OLED-Display-SH1106-I2C-128x64-Graphics-Library-AVR8/tree/main

[12]https://elearning.cm.ihu.gr/pluginfile.php/54859/mod_resource/content/9/Microcontroller_Programming_08.pdf

[13] https://arxiv.org/pdf/1801.02336

Παράρτημα Α – Κώδικας Συσκευής ανίχνευσης βημάτων

OLED.C

```
#define SSD1306_LCDWIDTH 128
#define SSD1306_LCDHEIGHT 64

#define Write_Address 0x78
#define Read_Address  0x79

unsigned char Contrast_level=0x8F;


const unsigned  ascii_table_8x16[94][16]={
0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //- -
0x00,0x00,0x38,0xFC, 0xFC,0x38,0x00,0x00, 0x00,0x00,0x00,0x0D,
0x0D,0x00,0x00,0x00, //-!-
0x00,0x0E,0x1E,0x00, 0x00,0x1E,0x0E,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //-"-
0x20,0xF8,0xF8,0x20, 0xF8,0xF8,0x20,0x00, 0x02,0x0F,0x0F,0x02,
0x0F,0x0F,0x02,0x00, //-#-
0x38,0x7C,0x44,0x47, 0x47,0xCC,0x98,0x00, 0x06,0x0C,0x08,0x38,
0x38,0x0F,0x07,0x00, //-$-
0x30,0x30,0x00,0x80, 0xC0,0x60,0x30,0x00, 0x0C,0x06,0x03,0x01,
0x00,0x0C,0x0C,0x00, //-%-
0x80,0xD8,0x7C,0xE4, 0xBC,0xD8,0x40,0x00, 0x07,0x0F,0x08,0x08,
0x07,0x0F,0x08,0x00, //-&-
0x00,0x10,0x1E,0x0E, 0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //-'-
0x00,0x00,0xF0,0xF8, 0x0C,0x04,0x00,0x00, 0x00,0x00,0x03,0x07,
0x0C,0x08,0x00,0x00, //-(-
0x00,0x00,0x04,0x0C, 0xF8,0xF0,0x00,0x00, 0x00,0x00,0x08,0x0C,
0x07,0x03,0x00,0x00, //-)-
0x80,0xA0,0xE0,0xC0, 0xC0,0xE0,0xA0,0x80, 0x00,0x02,0x03,0x01,
0x01,0x03,0x02,0x00, //-*-
0x00,0x80,0x80,0xE0, 0xE0,0x80,0x80,0x00, 0x00,0x00,0x00,0x03,
0x03,0x00,0x00,0x00, //-+-
0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00, 0x00,0x00,0x10,0x1E,
0x0E,0x00,0x00,0x00, //-,-
0x80,0x80,0x80,0x80, 0x80,0x80,0x80,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //---
0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x0C,
0x0C,0x00,0x00,0x00, //-.-
0x00,0x00,0x00,0x80, 0xC0,0x60,0x30,0x00, 0x0C,0x06,0x03,0x01,
0x00,0x00,0x00,0x00, //-/-
```

```
0xF8,0xF8,0x0C,0xC4, 0x0C,0xF8,0xF0,0x00, 0x03,0x07,0x0C,0x08,
0x0C,0x07,0x03,0x00, //-0-
0x00,0x10,0x18,0xFC, 0xFC,0x00,0x00,0x00, 0x00,0x08,0x08,0x0F,
0x0F,0x08,0x08,0x00, //-1-
0x08,0x0C,0x84,0xC4, 0x64,0x3C,0x18,0x00, 0x0E,0x0F,0x09,0x08,
0x08,0x0C,0x0C,0x00, //-2-
0x08,0x0C,0x44,0x44, 0x44,0xFC,0xB8,0x00, 0x04,0x0C,0x08,0x08,
0x08,0x0F,0x07,0x00, //-3-
0xC0,0xE0,0xB0,0x98, 0xFC,0xFC,0x80,0x00, 0x00,0x00,0x00,0x08,
0x0F,0x0F,0x08,0x00, //-4-
0x7C,0x7C,0x44,0x44, 0x44,0xC4,0x84,0x00, 0x04,0x0C,0x08,0x08,
0x08,0x0F,0x07,0x00, //-5-
0xF0,0xF8,0x4C,0x44, 0x44,0xC0,0x80,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0F,0x07,0x00, //-6-
0x0C,0x0C,0x04,0x84, 0xC4,0x7C,0x3C,0x00, 0x00,0x00,0x0F,0x0F,
0x00,0x00,0x00,0x00, //-7-
0xB8,0xFC,0x44,0x44, 0x44,0xFC,0xB8,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0F,0x07,0x00, //-8-
0x38,0x7C,0x44,0x44, 0x44,0xFC,0xF8,0x00, 0x00,0x08,0x08,0x08,
0x0C,0x07,0x03,0x00, //-9-
0x00,0x00,0x00,0x30, 0x30,0x00,0x00,0x00, 0x00,0x00,0x00,0x06,
0x06,0x00,0x00,0x00, //-:-
0x00,0x00,0x00,0x30, 0x30,0x00,0x00,0x00, 0x00,0x00,0x08,0x0E,
0x06,0x00,0x00,0x00, //-;-
0x00,0x80,0xC0,0x60, 0x30,0x18,0x08,0x00, 0x00,0x00,0x01,0x03,
0x06,0x0C,0x08,0x00, //-<-
0x00,0x20,0x20,0x20, 0x20,0x20,0x20,0x00, 0x00,0x01,0x01,0x01,
0x01,0x01,0x01,0x00, //-=-
0x00,0x08,0x18,0x30, 0x60,0xC0,0x80,0x00, 0x00,0x08,0x0C,0x06,
0x03,0x01,0x00,0x00, //->-
0x18,0x1C,0x04,0xC4, 0xE4,0x3C,0x18,0x00, 0x00,0x00,0x00,0x0D,
0x0D,0x00,0x00,0x00, //-?-
0xF0,0xF0,0x08,0xC8, 0xC8,0xF8,0xF0,0x00, 0x07,0x0F,0x08,0x0B,
0x0B,0x0B,0x01,0x00, //-@-
0xE0,0xF0,0x98,0x8C, 0x98,0xF0,0xE0,0x00, 0x0F,0x0F,0x00,0x00,
0x00,0x0F,0x0F,0x00, //-A-
0x04,0xFC,0xFC,0x44, 0x44,0xFC,0xB8,0x00, 0x08,0x0F,0x0F,0x08,
0x08,0x0F,0x07,0x00, //-B-
0xF0,0xF8,0x0C,0x04, 0x04,0x0C,0x18,0x00, 0x03,0x07,0x0C,0x08,
0x08,0x0C,0x06,0x00, //-C-
0x04,0xFC,0xFC,0x04, 0x0C,0xF8,0xF0,0x00, 0x08,0x0F,0x0F,0x08,
0x0C,0x07,0x03,0x00, //-D-
0x04,0xFC,0xFC,0x44, 0xE4,0x0C,0x1C,0x00, 0x08,0x0F,0x0F,0x08,
0x08,0x0C,0x0E,0x00, //-E-
0x04,0xFC,0xFC,0x44, 0xE4,0x0C,0x1C,0x00, 0x08,0x0F,0x0F,0x08,
0x00,0x00,0x00,0x00, //-F-
0xF0,0xF8,0x0C,0x84, 0x84,0x8C,0x98,0x00, 0x03,0x07,0x0C,0x08,
0x08,0x07,0x0F,0x00, //-G-
0xFC,0xFC,0x40,0x40, 0x40,0xFC,0xFC,0x00, 0x0F,0x0F,0x00,0x00,
0x00,0x0F,0x0F,0x00, //-H-
0x00,0x00,0x04,0xFC, 0xFC,0x04,0x00,0x00, 0x00,0x00,0x08,0x0F,
0x0F,0x08,0x00,0x00, //-I-
0x00,0x00,0x00,0x04, 0xFC,0xFC,0x04,0x00, 0x07,0x0F,0x08,0x08,
0x0F,0x07,0x00,0x00, //-J-
0x04,0xFC,0xFC,0xC0, 0xE0,0x3C,0x1C,0x00, 0x08,0x0F,0x0F,0x00,
0x01,0x0F,0x0E,0x00, //-K-
```

```
0x04,0xFC,0xFC,0x04, 0x00,0x00,0x00,0x00, 0x08,0x0F,0x0F,0x08,
0x08,0x0C,0x0E,0x00, //-L-
0xFC,0xFC,0x38,0x70, 0x38,0xFC,0xFC,0x00, 0x0F,0x0F,0x00,0x00,
0x00,0x0F,0x0F,0x00, //-M-
0xFC,0xFC,0x38,0x70, 0xE0,0xFC,0xFC,0x00, 0x0F,0x0F,0x00,0x00,
0x00,0x0F,0x0F,0x00, //-N-
0xF8,0xFC,0x04,0x04, 0x04,0xFC,0xF8,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0F,0x07,0x00, //-O-
0x04,0xFC,0xFC,0x44, 0x44,0x7C,0x38,0x00, 0x08,0x0F,0x0F,0x08,
0x00,0x00,0x00,0x00, //-P
0xF8,0xFC,0x04,0x04, 0x04,0xFC,0xF8,0x00, 0x07,0x0F,0x08,0x0E,
0x3C,0x3F,0x27,0x00, //-Q-
0x04,0xFC,0xFC,0x44, 0xC4,0xFC,0x38,0x00, 0x08,0x0F,0x0F,0x00,
0x00,0x0F,0x0F,0x00, //-R-
0x18,0x3C,0x64,0x44, 0xC4,0x9C,0x18,0x00, 0x06,0x0E,0x08,0x08,
0x08,0x0F,0x07,0x00, //-S-
0x00,0x1C,0x0C,0xFC, 0xFC,0x0C,0x1C,0x00, 0x00,0x00,0x08,0x0F,
0x0F,0x08,0x00,0x00, //-T-
0xFC,0xFC,0x00,0x00, 0x00,0xFC,0xFC,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0F,0x07,0x00, //-U-
0xFC,0xFC,0x00,0x00, 0x00,0xFC,0xFC,0x00, 0x01,0x03,0x06,0x0C,
0x06,0x03,0x01,0x00, //-V-
0xFC,0xFC,0x00,0x00, 0x00,0xFC,0xFC,0x00, 0x07,0x0F,0x0E,0x03,
0x0E,0x0F,0x07,0x00, //-W-
0x0C,0x3C,0xF0,0xE0, 0xF0,0x3C,0x0C,0x00, 0x0C,0x0F,0x03,0x01,
0x03,0x0F,0x0C,0x00, //-X-
0x00,0x0C,0x7C,0xC0, 0xC0,0x7C,0x3C,0x00, 0x00,0x00,0x08,0x0F,
0x0F,0x08,0x00,0x00, //-Y-
0x1C,0x0C,0x84,0xC4, 0x64,0x3C,0x1C,0x00, 0x0E,0x0F,0x09,0x08,
0x08,0x0C,0x0E,0x00, //-Z-
0x00,0x00,0xFC,0xFC, 0x04,0x04,0x00,0x00, 0x00,0x00,0x0F,0x0F,
0x08,0x08,0x00,0x00, //-[-
0x38,0x70,0xE0,0xC0, 0x80,0x00,0x00,0x00, 0x00,0x00,0x00,0x01,
0x03,0x07,0x0E,0x00, //-\-
0x00,0x00,0x04,0x04, 0xFC,0xFC,0x00,0x00, 0x00,0x00,0x08,0x08,
0x0F,0x0F,0x00,0x00, //-]-
0x08,0x0C,0x06,0x03, 0x06,0x0C,0x08,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //-^-
0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00, 0x20,0x20,0x20,0x20,
0x20,0x20,0x20,0x20, //-_-
0x00,0x00,0x03,0x07, 0x04,0x00,0x00,0x00, 0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00, //-`-
0x00,0xA0,0xA0,0xA0, 0xE0,0xC0,0x00,0x00, 0x07,0x0F,0x08,0x08,
0x07,0x0F,0x08,0x00, //-a-
0x04,0xFC,0xFC,0x20, 0x60,0xC0,0x80,0x00, 0x00,0x0F,0x0F,0x08,
0x08,0x0F,0x07,0x00, //-b-
0xC0,0xE0,0x20,0x20, 0x20,0x60,0x40,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0C,0x04,0x00, //-c-
0x80,0xC0,0x60,0x24, 0xFC,0xFC,0x00,0x00, 0x07,0x0F,0x08,0x08,
0x07,0x0F,0x08,0x00, //-d-
0xC0,0xE0,0xA0,0xA0, 0xA0,0xE0,0xC0,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0C,0x04,0x00, //-e-
0x40,0xF8,0xFC,0x44, 0x0C,0x18,0x00,0x00, 0x08,0x0F,0x0F,0x08,
0x00,0x00,0x00,0x00, //-f-
0xC0,0xE0,0x20,0x20, 0xC0,0xE0,0x20,0x00, 0x27,0x6F,0x48,0x48,
0x7F,0x3F,0x00,0x00, //-g-
```

```
0x04,0xFC,0xFC,0x40, 0x20,0xE0,0xC0,0x00, 0x08,0x0F,0x0F,0x00,
0x00,0x0F,0x0F,0x00, //-h-
0x00,0x00,0x20,0xEC, 0xEC,0x00,0x00,0x00, 0x00,0x00,0x08,0x0F,
0x0F,0x08,0x00,0x00, //-i-
0x00,0x00,0x00,0x00, 0x20,0xEC,0xEC,0x00, 0x00,0x30,0x70,0x40,
0x40,0x7F,0x3F,0x00, //-j-
0x04,0xFC,0xFC,0x80, 0xC0,0x60,0x20,0x00, 0x08,0x0F,0x0F,0x01,
0x03,0x0E,0x0C,0x00, //-k-
0x00,0x00,0x04,0xFC, 0xFC,0x00,0x00,0x00, 0x00,0x00,0x08,0x0F,
0x0F,0x08,0x00,0x00, //-l-
0xE0,0xE0,0x60,0xC0, 0x60,0xE0,0xC0,0x00, 0x0F,0x0F,0x00,0x07,
0x00,0x0F,0x0F,0x00, //-m-
0x20,0xE0,0xC0,0x20, 0x20,0xE0,0xC0,0x00, 0x00,0x0F,0x0F,0x00,
0x00,0x0F,0x0F,0x00, //-n-
0xC0,0xE0,0x20,0x20, 0x20,0xE0,0xC0,0x00, 0x07,0x0F,0x08,0x08,
0x08,0x0F,0x07,0x00, //-o-
0x20,0xE0,0xC0,0x20, 0x20,0xE0,0xC0,0x00, 0x40,0x7F,0x7F,0x48,
0x08,0x0F,0x07,0x00, //-p-
0xC0,0xE0,0x20,0x20, 0xC0,0xE0,0x20,0x00, 0x07,0x0F,0x08,0x48,
0x7F,0x7F,0x40,0x00, //-q-
0x20,0xE0,0xC0,0x60, 0x20,0xE0,0xC0,0x00, 0x08,0x0F,0x0F,0x08,
0x00,0x00,0x00,0x00, //-r-
0x40,0xE0,0xA0,0x20, 0x20,0x60,0x40,0x00, 0x04,0x0C,0x09,0x09,
0x0B,0x0E,0x04,0x00, //-s-
0x20,0x20,0xF8,0xFC, 0x20,0x20,0x00,0x00, 0x00,0x00,0x07,0x0F,
0x08,0x0C,0x04,0x00, //-t-
0xE0,0xE0,0x00,0x00, 0xE0,0xE0,0x00,0x00, 0x07,0x0F,0x08,0x08,
0x07,0x0F,0x08,0x00, //-u-
0x00,0xE0,0xE0,0x00, 0x00,0xE0,0xE0,0x00, 0x00,0x03,0x07,0x0C,
0x0C,0x07,0x03,0x00, //-v-
0xE0,0xE0,0x00,0x80, 0x00,0xE0,0xE0,0x00, 0x07,0x0F,0x0C,0x07,
0x0C,0x0F,0x07,0x00, //-w-
0x20,0x60,0xC0,0x80, 0xC0,0x60,0x20,0x00, 0x08,0x0C,0x07,0x03,
0x07,0x0C,0x08,0x00, //-x-
0xE0,0xE0,0x00,0x00, 0x00,0xE0,0xE0,0x00, 0x47,0x4F,0x48,0x48,
0x68,0x3F,0x1F,0x00, //-y-
0x60,0x60,0x20,0xA0, 0xE0,0x60,0x20,0x00, 0x0C,0x0E,0x0B,0x09,
0x08,0x0C,0x0C,0x00, //-z-
0x00,0x40,0x40,0xF8, 0xBC,0x04,0x04,0x00, 0x00,0x00,0x00,0x07,
0x0F,0x08,0x08,0x00, //-{-
0x00,0x00,0x00,0xBC, 0xBC,0x00,0x00,0x00, 0x00,0x00,0x00,0x0F,
0x0F,0x00,0x00,0x00, //-|-
0x00,0x04,0x04,0xBC, 0xF8,0x40,0x40,0x00, 0x00,0x08,0x08,0x0F,
0x07,0x00,0x00,0x00  //-}-
};


void Set_Page_Address(unsigned char add)
{
// Εεκινά την επικοινωνία I2C
   i2c_start();
// Αποστέλλει τη διεύθυνση εγγραφής (Write Address).
   i2c_write(Write_Address);
// Αποστέλλει την εντολή 0x80 (Σημαίνει ότι θέλουμε να
ρυθμίσουμε //την "σελίδα").
   i2c_write(0x80);
```

```
// Τοποθετούμε το bit 0xB0 στην τιμή του add (σελίδα 0-7).
    add=0xb0|add;
// Αποστέλλουμε την τιμή της σελίδας (add) στην οθόνη.
   i2c_write(add);
   delay_us(10);
   i2c_stop();
   return;
}

void Set_Column_Address(unsigned char add)
{
   i2c_start();
   i2c_write(Write_Address);
// Αποστέλλει την εντολή για ρύθμιση της διεύθυνσης της στήλης.
   i2c_write(0x80);
// Αποστέλλει τα Higher 4 bits της διεύθυνσης στήλης (με //την
εντολή 0x10).
   i2c_write((0x10|(add>>4)));
// Αποστέλλει ξανά την εντολή για ρύθμιση της στήλης.
   i2c_write(0x80);
// Αποστέλλει τα Lower 4 bits της διεύθυνσης στήλης.
   i2c_write((0x0f&add));
// Τερματίζει την επικοινωνία I2C.
   i2c_stop();
   return;
}


void Initial_1306(void)
{

   delay_ms(200);

   i2c_start();
   i2c_write(Write_Address);
   i2c_write(0x80);
// Απενεργοποιεί την οθόνη OLED (turn off OLED panel).
   i2c_write(0xae); //DISPLAY OFF
   i2c_write(0x80);
// Ορίζει την αναλογία διαίρεσης του ρολογιού και τη συχνότητα
//του ταλαντωτή.
   i2c_write(0xd5); //SET DISPLAY CLOCK DIV
   i2c_write(0x80);
// Ορίζει τη διαίρεση του ρολογιού.
   i2c_write(0x80);
   i2c_write(0x80);
// Ορίζει τον λόγο πολλαπλασιασμού (multiplex ratio(1 to 64).
   i2c_write(0xa8);  //SET DISPLAY OFFSET
   i2c_write(0x80);
// Ορίζει τον λόγο πολλαπλασιασμού (1/64 duty).
   i2c_write(0x3f);
   i2c_write(0x80);
// Ορίζει την εκτροπή της οθόνης (display offset).
   i2c_write(0xd3); //SET DISPLAY OFFSET
   i2c_write(0x80);
   i2c_write(0x00);//Not offset
```

```
   i2c_write(0x80);
// Ορίζει την ενεργοποίηση ή απενεργοποίηση της αντλίας φόρτισης
//(charge pump).
   i2c_write(0x8d); //CHATGE PUMP
   i2c_write(0x80);
//Ρυθμίζει την αντλία φόρτισης σε ενεργοποιημένη κατάσταση (0x10
= απενεργοποιημένη, 0x14 = ενεργοποιημένη).
   i2c_write(0x14);
   i2c_write(0x80);
// Ορίζει τη διεύθυνση γραμμής εκκίνησης.
   i2c_write(0x40); //SET START LINE
   i2c_write(0x80);
// Ορίζει την κανονική εμφάνιση (normal display).
   i2c_write(0xa6); //NORMAL DISPLAY
   i2c_write(0x80);
// Απενεργοποιεί την οθόνη όλης της περιοχής (disable entire
//display on).
   i2c_write(0xa4); //DISABLE Entire Display ON
   i2c_write(0x80);
// Ορίζει την αναστροφή των σημείων(segment remap 128 to 0).
   i2c_write(0xa1);
   i2c_write(0x80);
// Ορίζει την κατεύθυνση του σάρωσης των COM (COM Output Scan
//Direction).
   i2c_write(0xC8); // COMSCANDEC
   i2c_write(0x80);
// Ορίζει την υλική διάταξη των ακίδων COM (COM Pins hardware
//configuration).
   i2c_write(0xda); // SETCOMPINS
   i2c_write(0x80);
// Ορίζει την υλική διάταξη των ακίδων COM.
   i2c_write(0x12);
   i2c_write(0x80);
// Ορίζει τη ρύθμιση της αντίθεσης (contrast control register).
   i2c_write(0x81); //SET CONTRAST
   i2c_write(0x80);
   i2c_write(Contrast_level);
   i2c_write(0x80);
// Ορίζει την περίοδο προ φόρτισης (pre-charge period).
   i2c_write(0xd9); // SET PRECARGE
   i2c_write(0x80);
   i2c_write(0xf1);
   i2c_write(0x80);
//Ορίζει την τιμή vcomh
   i2c_write(0xdb); // SET VCOMDETECT
   i2c_write(0x80);
   i2c_write(0x40); // SET START LINE
   i2c_write(0x80);
//Ενεργοποιεί την οθόνη OLED (oled panel)
   i2c_write(0xaf); // SET DISPLAY ON

   i2c_stop();
}


void lcd_clrscroled(unsigned char nn)
```

```
{
    unsigned char i,j;
    for(i=0;i<8;i++)
   {
      Set_Page_Address(i);
      Set_Column_Address(0);
      i2c_start();
      i2c_write(Write_Address);
// Δηλώνει ότι θα ακολουθήσουν δεδομένα (όχι εντολές).
      i2c_write(0x40);
// Κάθε σελίδα έχει 8 στήλες (ή 8*128 για την οθόνη).
      for(j=0;j<128;j++)
      {
// Διατρέχουμε τις στήλες από 0 έως 127
// Στέλνουμε την τιμή nn (π.χ., 0x00 για να καθαρίσουμε την
οθόνη).
            i2c_write(nn);
      }
       i2c_stop();
   }
    return;
}


void Show_font21(unsigned char xp,unsigned char yp,unsigned char
ch)
{
     char i,i1;

          i1 = xp*8;

         Set_Page_Address(yp);
         Set_Column_Address(i1);

         i2c_start();
         i2c_write(Write_Address);
         i2c_write(0x40);

     for(i=0; i<8; i++) {
        i2c_write(ascii_table_8x16[ch-32][i]);
     }
          yp++;
          i1 = xp*8;
          Set_Page_Address(yp);
          Set_Column_Address(i1);

          i2c_start();
          i2c_write(Write_Address);
          i2c_write(0x40);

     for(i=8; i<16; i++) {
        i2c_write(ascii_table_8x16[ch-32][i]);
     }
      i2c_stop();
    return;
}
```

```
void Show_String8x16(unsigned char *Data_Pointer, unsigned char
b, unsigned char c)
{
   unsigned char *Src_Pointer;

    Src_Pointer=Data_Pointer;

   while(true)
   {
      Show_font21(b,c,*Src_Pointer);
      Src_Pointer++;
       b++;
      if(*Src_Pointer == 0) break;
   }
}
```

BNO055.C

```
// Ενεργοποίηση του πρωτοκόλλου I2C στα pin C3,C4
#use i2c(master, scl=PIN_c3, sda=PIN_c4,stream = BNO)

/** BNO055 Address A **/
#define BNO055_ADDRESS_A 0x28
/** BNO055 Address B **/
#define BNO055_ADDRESS_B 0x29
/** BNO055 ID **/
#define BNO055_ID 0xA0

/* Linear acceleration data registers */
#define BNO055_LINEAR_ACCEL_DATA_X_LSB_ADDR = 0X28
#define BNO055_LINEAR_ACCEL_DATA_X_MSB_ADDR = 0X29
#define BNO055_LINEAR_ACCEL_DATA_Y_LSB_ADDR = 0X2A
#define BNO055_LINEAR_ACCEL_DATA_Y_MSB_ADDR = 0X2B
#define BNO055_LINEAR_ACCEL_DATA_Z_LSB_ADDR = 0X2C
#define BNO055_LINEAR_ACCEL_DATA_Z_MSB_ADDR = 0X2D


#define  ACC_DATA_X_LSB 0x08
#define  ACC_DATA_X_MSB 0x09
#define  ACC_DATA_Y_LSB 0x0A
#define  ACC_DATA_Y_MSB 0x0B
#define  ACC_DATA_Z_LSB 0x0C
#define  ACC_DATA_Z_MSB 0x0D

int16 bno_x,bno_y,bno_z,bno_xD,bno_yD,bno_zD;


void BNO055_Config()
```

```
{   i2c_start(BNO);
//  WRITE MODE
    i2c_write(BNO, BNO055_ADDRESS_A <<1);
// Επιλογή καταχωρητή BNO055_OPR_MODE_ADDR
    i2c_write(BNO, 0x3D);
// Θέτει τον αισθητήρα σε λειτουργία Config mode
    i2c_write(BNO, 0x00);
    i2c_stop(BNO);
// Καθυστέρηση για σταθεροποίηση
    delay_ms(500);

    // Set to normal power mode
    i2c_start(BNO);
    i2c_write(BNO, BNO055_ADDRESS_A <<1); //WRITE MODE
// Επιλογή καταχωρητή BNO055_PWR_MODE_ADDR
    i2c_write(BNO, 0x3E);
// θέτει τον αισθητήρα σε λειτουργία Normal mode
    i2c_write(BNO, 0x00);
    i2c_stop(BNO);
    delay_ms(500);

    // Set to NDOF mode
    i2c_start(BNO);
//  WRITE MODE
    i2c_write(BNO, BNO055_ADDRESS_A <<1);
//  Επιλογή καταχωρητή  BNO055_OPR_MODE_ADDR
    i2c_write(BNO, 0x3D);
//  θέτει τον αισθητήρα σε λειτουργία NDOF mode(λειτουργία 9
//Degreess of Freedom)
    i2c_write(BNO, 0x0C);
    i2c_stop(BNO);
    delay_ms(500);
}

void BNO055_read_register_id()
{
    unsigned char gyr_id, acc_id, chip_id, mag_id;

    i2c_start(BNO);
// Μπαίνει σε λειτουργία Write mode
    i2c_write(BNO, BNO055_ADDRESS_A << 1);
// Εκκινάει από την διεύθυνση 0x00 όπου βρίσκονται τα ID
//registers
    i2c_write(BNO, 0x00);
    i2c_start(BNO);
// Μπαίνει σε λειτουργία Read mode
    i2c_write(BNO, (BNO055_ADDRESS_A << 1) | 1);
// Read CHIP_ID with ACK Address 0x00
    chip_id = i2c_read(BNO, 1);
// Read ACC_ID with ACK Address 0x01
    acc_id = i2c_read(BNO, 1);
//read MAG_ID with ACK Address 0x02
    mag_id = i2c_read(BNO, 1);
// Read GYR_ID with NACK Address 0x03
    gyr_id = i2c_read(BNO, 0);
```

```
    i2c_stop(BNO);

  // Εμφάνιση αποτελεσμάτων στην οθόνη OLED
    sprintf(master_mask,"CHPID:%02X ",chip_id);
     Show_String8x16(master_mask,0,0);
    sprintf(master_mask,"ACC:%02X",acc_id);
     Show_String8x16(master_mask,0,2);
    sprintf(master_mask,"MAG:%02X",mag_id);
     Show_String8x16(master_mask,0,4);
    sprintf(master_mask,"GYR:%02X",gyr_id);
     Show_String8x16(master_mask,0,6);


    delay_ms(4000);

  }


  void BNO055_read_linear_accel()
  {
      unsigned char x_lsb, x_msb, y_lsb, y_msb, z_lsb, z_msb;
      signed  int16 x1, y1, z1;


      // Set start address for reading linear acceleration data
      i2c_start(BNO);
      i2c_write(BNO, BNO055_ADDRESS_A << 1); // Write mode
      i2c_write(BNO, 0x28); // Start address
      i2c_start(BNO);
      i2c_write(BNO, (BNO055_ADDRESS_A << 1) | 1); // Read mode

      // Διαβάζει τα  linear acceleration data
      x_lsb = i2c_read(BNO, 1); // Διάβασμα X LSB με ACK
      x_msb = i2c_read(BNO, 1); // Διάβασμα X MSB με ACK
      y_lsb = i2c_read(BNO, 1); // Διάβασμα Y LSB με ACK
      y_msb = i2c_read(BNO, 1); // Διάβασμα Y MSB με ACK
      z_lsb = i2c_read(BNO, 1); // Διάβασμα Z LSB με ACK
      z_msb = i2c_read(BNO, 0); // Διάβασμα Z MSB με NACK

      i2c_stop(BNO);

      // Συνδυασμός των MSB και LSB
      x1 = ((int16)x_msb << 8) | x_lsb;
      y1 = ((int16)y_msb << 8) | y_lsb;
      z1 = ((int16)z_msb << 8) | z_lsb;
      // Εκτύπωση των δεδομένων στην οθόνη OLED
      sprintf(master_mask, "X data  : %3Ld ", x1);
       Show_String8x16(master_mask, 0, 2);

      sprintf(master_mask, "Y data  : %3Ld ", y1);
       Show_String8x16(master_mask, 0, 4);

      sprintf(master_mask, "Z data  : %3Ld ", z1);
       Show_String8x16(master_mask, 0, 6);
```

```
 // Αποστολή τιμών στην RAM.
      bno_x= x1;
      bno_y= y1;
      bno_z= z1;
//Σειριακή Εκτύπωση των δεδομένων
    printf("XL:%3Ld \r\n",x1);
    printf("YL:%3Ld \r\n",y1);
    printf("ZL:%3Ld \r\n\r\n",z1);

// Εμφάνιση του δείκτη ptr_save ο οποίος μας δείχνει το πλήθος των
//μετρήσεων
    sprintf(master_mask, "RECORT:%4Lu ",ptr_save);
    Show_String8x16(master_mask, 11, 0);

    delay_ms(50);
}



void BNO055_read_accel()
{

    unsigned char xD_lsb, xD_msb, yD_lsb, yD_msb, zD_lsb, zD_msb;
    signed int16 xD, yD, zD;


    // Set start address for reading acceleration data
    i2c_start(BNO);
    i2c_write(BNO, BNO055_ADDRESS_A << 1); // Write mode
    // Εκκινάει από την διέυθυνση 0x08
    i2c_write(BNO, ACC_DATA_X_LSB);
    i2c_start(BNO);
    i2c_write(BNO, (BNO055_ADDRESS_A << 1) | 1); // Read mode

    // Διάβασμα των acceleration data
    xD_lsb = i2c_read(BNO, 1); // Διάβασμα X LSB με ACK
    xD_msb = i2c_read(BNO, 1); // Διάβασμα X MSB με ACK
    yD_lsb = i2c_read(BNO, 1); // Διάβασμα Y LSB με ACK
    yD_msb = i2c_read(BNO, 1); // Διάβασμα Y MSB με ACK
    zD_lsb = i2c_read(BNO, 1); // Διάβασμα Z LSB με ACK
    zD_msb = i2c_read(BNO, 0); // Διάβασμα Z MSB με NACK

    i2c_stop(BNO);

    // Συνδυασμός MSB και LSB
    xD = ((int16)xD_msb << 8) | xD_lsb;
    yD = ((int16)yD_msb << 8) | yD_lsb;
    zD = ((int16)zD_msb << 8) | zD_lsb;

    // Εκτύπωση των δεδομένων στην οθόνη OLED
    sprintf(master_mask, "X data  : %3Ld ", xD);
     Show_String8x16(master_mask, 0, 2);

    sprintf(master_mask, "Y data  : %3Ld ", yD);
     Show_String8x16(master_mask, 0, 4);
```

```
    sprintf(master_mask, "Z data  : %3Ld ", zD);
     Show_String8x16(master_mask, 0, 6);
    // Αποθήκευση των δεδομένων στην RAM
     bno_xD= xD;
     bno_yD= yD;
     bno_zD= zD;

    printf("XD:%3Ld \r\n",xD);
    printf("YD:%3Ld \r\n",yD);
    printf("ZD:%3Ld \r\n\r\n",zD);
    delay_ms(100);
}
```

Save_data.C

```
// Δηλώσεις μεταβλητών και πινάκων
signed int16 save_acelx[512];
signed int16 seve_acely[512];
signed int16 save_acelz[512];
signed int16 save_acelxD[512];
signed int16 seve_acelyD[512];
signed int16 save_acelzD[512];

void  write_paket(int16  ptr)
{
// Αποθήκευση των δεδομένων γραμμικής επιτάχυσνης
//στους πίνακες
    save_acelx[ptr_save]= bno_x;
    seve_acely[ptr_save]= bno_y;
    save_acelz[ptr_save]= bno_z;
// Αποθήκευση των δεδομένων απλής επιτάχυσνης
//στους πίνακες
    save_acelxD[ptr_save]= bno_xD;
    seve_acelyD[ptr_save]= bno_yD;
    save_acelzD[ptr_save]= bno_zD;
 // Αύξηση του δείκτη κατά 1
    ptr_save++;
    }
```

Main.h

```
// Περιέχει όλους τους καταχωρητές του μικροελεγκτή PIC18F57Q43
#include <18F57Q43.h>

// Ορίζει ότι ο Analog-to-Digital-Converter θα λειτουργεί με 10bit
// ανάλυση
#device  ADC=10
//Απενεργοποίηση του Watch Dog Timer και του Brown-Out-Reset
#FUSES   NOWDT , NOBROWNOUT

// Ρυθμίζει τον αρχικό ταλαντωτή του μικροελεγκτή στα 64MHz
#FUSES RSTOSC_HFINTRC_64MHZ

// Ρύθμιση του συστήματος καθυστέρησης του κώδικα στην συχνότητα
//32MHz, χρησιμοποιείται η μισή ταχύτητα από τον ταλαντωτή για
//καλύτερη απόδοση.
#use delay(internal=32MHz)

// Ρύθμιση για επικοινωνία με το πρωτόκολλο i2c και την oled
#use i2c(master, scl=PIN_b1, sda=PIN_b2 ,FAST)

// Ρύθμιση της σειριακής επικοινωνίας με την θύρα UART3 του
//μικροελεγκτή με baud rate 9600.
#use rs232(uart3, baud=9600,xmit=PIN_a3,rcv=PIN_a4,bits=8, errors
)
```

Main.c

```
#include <main.h>

#define LED      PIN_F3
#define BUTTON    PIN_B4

// Δηλώσεις μεταβλητών και πινάκων
char  i;
char master_mask[32];
int16  ptr_save = 0;


#include <OLED.c>
#include <BNP055.c>
#include <save_data.c>

#zero_ram


VOID init_ports( void )
   {
// pin A0,A1,A2,A4 ΕΙΣΟΔΟΣ //PIN A3,A5,A6,A7 ΕΞΟΔΟΣ
       set_TRIS_A (0b00010111);
// PIN B0,B1,B2,B4 ΕΙΣΟΔΟΙ //PIN B3,B5,B6,B7 ΕΞΟΔΟΣ
       set_TRIS_B (0b00010111);
// PIN F1,F4 ΕΙΣΟΔΟΣ // PIN F0,F2,F3,F5,F6,F7 ΕΞΟΔΟΣ
       set_TRIS_F (0b11110111);
```

```
   }


VOID main()
   {
      init_ports () ;
    // Ενεργοποίηση των pull up αντιστάσεων στα pin
RB0,RB1,RB2,RB4
      port_b_pullups (0b00010111);


      // Αρχικοποίηση της oled  οθόνης
      Initial_1306 () ;
      delay_ms (200) ;
     // καθαρισμός της οθόνης OLED
      lcd_clrscroled (0x00);
     // Ενεργοποίηση του BNO055
      BNO055_Config () ;
      delay_ms (400) ;
     // Διαβάζουμε τα Register ID από τον BNO055
      BNO055_read_register_id();
      delay_ms(500);
      lcd_clrscroled (0x00);


int previous_button_state = 0;
int button_state = 0;
int current_state = 0;
int current_button_input = 0;

ptr_save = 0;

while(TRUE) {
// Διαβάζει την τωρινή κατάσταση που βρίσκεται το button
    current_button_input = input(BUTTON);
// Ελεγχος αν το button έχει πατηθεί
    if(current_button_input == 1 && previous_button_state == 0) {
        delay_ms(20);
// Αφού πατηθεί το button το current_state αυξάνεται κατά ένα
        if(input(BUTTON) == 1) {
            button_state = 1;
            current_state++;
// Επαναφέρει το current state στο 0 για να ξεκινήσει πάλι από το
//case 0
            if(current_state > 2) {
                current_state = 0;
            }
// Έναρξη της switch case, η οποία παίρνει τιμές ανάλογα με την
μεταβλητή current_state.
            switch(current_state) {
// Στο case 0 πρώτα σβήνει το led της αναπτυξιακής πλακέτας και
στην συνέχεια αποθηκεύουμε τις τιμές από τον bno055 στην ram.

                case 0:
                    lcd_clrscroled(0x00);
```

```
                    output_low(LED);
                    printf("case 0: Continuously reading sensor
data\n\r");

                    delay_ms(500);
//Συνεχής καταγραφή από τον αισθητήρα μέχρι να ξανά πατηθεί το
//button και το current_state γίνει 1.
              while(current_state == 0) {
                        BNO055_read_linear_accel();
                        write_paket(ptr_save);
                        delay_ms(50);
                        if(input(BUTTON)!= 1){
                           break;
                        }
                }

                  break;
// Στο case 1 το led της αναπτυξιακής πλακέτας curiosity nano
ανάβει και εκτυπώνουμε στην οθόνη την λέξη PUAUSE.
                case 1:

                    lcd_clrscroled(0x00);
                    output_high(LED);
                    delay_ms(100);
                    printf("case 1: State has changed.\n\r");

                    sprintf(master_mask, "PAUSE");
                    Show_String8x16(master_mask, 5, 2);
                    break;
// Στο case 2 το led αναβοσβήνει μια φορά και στην συνέχεια
στέλνει τις αποθηκευμένες τιμές στην σειριακή θύρα και μόλις
τελειώσει εμφανίζει στην οθόνη την λέξη FINISH.
                case 2:
                    output_high(LED);
                    delay_ms(500);
                    output_low(LED);
                    delay_ms(500);
                    output_high(LED);
                    delay_ms(500);
                    lcd_clrscroled(0x00);

                     printf("case 2: READ SAVE DATA\n\r");
// Με την βοήθεια του δείκτη ptr_save εκτυπώνουμε συριακά τις
τιμές //που αποθηκεύτηκαν στους πίνακες
save_acelx,save_acely,save_acelz
                     for(i=0; i<=ptr_save; i++ )
                      {
                       sprintf (master_mask,"EXPORT DATA:%3u",i);
                       Show_String8x16(master_mask,1, 2);

                       printf("XL:%3Ld \r\n",save_acelx[i]);
                       printf("YL:%3Ld \r\n",seve_acely[i]);
                       printf("ZL:%3Ld \r\n\r\n",save_acelz[i]);

                       delay_ms(100);
                       lcd_clrscroled (0x00);
```

```
                        }
                    printf("Counter_Data: %4Lu\n\r",ptr_save);
                    ptr_save = 0;
                    sprintf(master_mask,"FINISH");
                    Show_String8x16(master_mask,5, 2);
                    break;
                }
            }
        }

        // Ενημέρωση για την προηγούμενη κατάσταση του button
        previous_button_state = current_button_input;
    }
}
```

Παράρτημα Β – Firmware αναγνώρισης βημάτων

StepTracker.py

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.signal as signal
import math
import re


# Συνάρτηση για την εξαγωγή των τιμών από το αρχείο
def extract_values(file_path):

    data_XL_YL_ZL = {'XL': [], 'YL': [], 'ZL': []}

    with open(file_path, 'r') as file:
        for line in file:
            line = line.strip()
            if line.startswith('XL:'):

data_XL_YL_ZL['XL'].append(float(line.split(':')[1]))
            elif line.startswith('YL:'):

data_XL_YL_ZL['YL'].append(float(line.split(':')[1]))
            elif line.startswith('ZL:'):

data_XL_YL_ZL['ZL'].append(float(line.split(':')[1]))

            match = re.search(r'Counter_Data:\s*(\d+)', line)
            if match:
             Count = int(match.group(1))
```

```
    return  data_XL_YL_ZL,Count

# Συνάρτηση για τον υπολογισμό του magnitude
def calculate_magnitude(x, y, z):
    return math.sqrt(x**2 + y**2 + z**2)
def calculate_magnitudexy(x, y):
    return math.sqrt(x**2 + y**2)


# Διαβάζουμε τα δεδομένα από το αρχείο example2.txt
file_path = 'example2.txt'
data_XL_YL_ZL,Count = extract_values(file_path)
print(f"Data number is: {Count}")


mag_XL_YL_ZL = [
    calculate_magnitude(x, y, z)
    for x, y, z in zip(data_XL_YL_ZL['XL'], data_XL_YL_ZL['YL'],
data_XL_YL_ZL['ZL'])
]

mag_XL_YL = [
    calculate_magnitudexy(x, y)
    for x, y in zip(data_XL_YL_ZL['XL'], data_XL_YL_ZL['YL'])
]

avgmag2 = sum(mag_XL_YL) / len(mag_XL_YL)
netmags2 = [(mag2 - avgmag2)**2/Count for mag2 in mag_XL_YL]
peaks2, _ = signal.find_peaks(netmags2, height=900, distance=2)

# Ο χρόνος (σε ms), αυξάνεται ανά 150ms για κάθε σετ μετρήσεων
time = [i * 70 for i in range(len(mag_XL_YL_ZL))]

def filter_close_peaks(peak_times, peak_values,
min_distance=700):
    filtered_peaks = []  # Αποθήκευση των φιλτραρισμένων κορυφών
    i = 0

    while i < len(peak_times):
        current_peak = (peak_times[i], peak_values[i])
        j = i + 1

        # Έλεγχος αν υπάρχουν κοντινές κορυφές
        while j < len(peak_times) and (peak_times[j] -
peak_times[i] < min_distance):
            if peak_values[j] > current_peak[1]:  # Αν η νέα
κορυφή είναι μεγαλύτερη, αντικαθιστούμε
                current_peak = (peak_times[j], peak_values[j])
            j += 1

        filtered_peaks.append(current_peak)  # Αποθήκευση της
μεγαλύτερης κορυφής
        i = j  # Προχωράμε στην επόμενη απομακρυσμένη κορυφή

    return filtered_peaks
```

```
peakXY = filter_close_peaks(np.array(time)[peaks2],
np.array(netmags2)[peaks2], min_distance=700)

number_of_Steps = len(peakXY)

# Εκτύπωση των κορυφών (χρόνος, τιμή netmags2)
print("Number Peaks(Time, Net Magnitude XY):")
for peak2 in peakXY:
    print([peak2])

 # Διαχωρισμός χρόνου και τιμής κορυφών
filtered_peak_times, filtered_peak_values = zip(*peakXY)

# Δημιουργία διαγράμματος για XL, YL, ZL
plt.figure(figsize=(15, 6))
plt.plot(time, data_XL_YL_ZL['XL'], marker='o', label='XL',
color='r')
plt.plot(time, data_XL_YL_ZL['YL'], marker='o', label='YL',
color='g')
plt.plot(time, data_XL_YL_ZL['ZL'], marker='o', label='ZL',
color='b')
plt.title('XL, YL, ZL σε σχέση με τον Χρόνο')
plt.xlabel('Χρόνος (ms)')
plt.ylabel('Τιμές')
plt.legend()
plt.grid(True)

# Δημιουργία διαγράμματος
plt.figure(figsize=(15, 6))
plt.plot(time, netmags2, label='NET Magnitude XY', color='b')

# Σχεδίαση των φιλτραρισμένων κορυφών
plt.scatter(filtered_peak_times, filtered_peak_values,
color='r', marker='x', label='Filtered Peaks')
plt.title('Φιλτραρισμένες Κορυφές - Net Magnitude XY')
plt.xlabel('Χρόνος (ms)')
plt.ylabel('Net Magnitude XY')
plt.legend()
plt.grid(True)
plt.show()

# Εκτύπωση του αριθμού των βημάτων
print(f'Number of steps: {number_of_Steps}')
```

Παράρτημα Γ – Συμπληρωματικό Υλικό

Για την πληρέστερη κατανόηση της λειτουργίας της συσκευής που αναπτύχθηκε στο πλαίσιο της παρούσας διπλωματικής εργασίας, έχει δημιουργηθεί ένα επεξηγηματικό βίντεο, το οποίο είναι διαθέσιμο στον ακόλουθο σύνδεσμο:

Link: https://youtu.be/hNOQP1DH3xc?si=v2_sdOumNYu0JMTt